

Algoritmos

De la lógica a la programación

Contenido

1	Créditos Editoriales	1
2	Acerca de este volumen	3
3	Presentación del curso	5
3.1	Objetivo general de la asignatura	5
3.2	Aprendizajes esperados	5
4	¿Qué es un algoritmo?	7
4.1	Elementos básicos de un algoritmo	7
4.2	Ejemplo cotidiano	7
5	¿Cómo desarrollar un algoritmo?	9
6	¿Cómo se describe un algoritmo?	11
7	Variables: de neuronas a espacios de memoria	13
7.1	Algoritmo para sumar dos números	13
7.2	Pseudocódigo	14
7.3	Explicación paso a paso	14
8	Ejercicio guiado: multiplicar tres números	15
8.1	Análisis	15
8.2	Pseudocódigo	15
9	Diagramas de flujo	17
9.1	Nomenclatura básica	17
9.2	Ejemplo gráfico del flujo para sumar dos números	18
9.3	Ejemplo textual del flujo para sumar dos números	18
10	Bifurcaciones y decisiones	21
10.1	Ejemplo: mayor de edad	21
10.2	Diagrama de flujo	22
10.3	Flujo lógico	22
11	Operadores	25
12	Operadores aritméticos	27
12.1	Módulo o residuo	27

13 Operadores relacionales	29
14 Operadores lógicos	31
14.1 Operador Y: &&	31
14.2 Operador O: 	31
14.3 Operador NO: !	32
15 Casos de análisis	33
16 Caso 1: operador &&	35
17 Caso 2: combinación de y &&	37
18 Caso 3: operador !	39
19 Prioridad de operadores	41
20 Introducción a ciclos	43
20.1 ¿Cuándo se aplican los ciclos?	43
21 Ejemplo: sumar N números	45
21.1 Análisis	45
21.2 Diagrama de flujo	46
21.3 Pseudocódigo	47
22 Ejemplo en C++: sumar del 1 hasta N	49
23 Ejemplo en C++: encontrar el mayor de N números	51
24 Programa de aplicación: ecuación de primer grado	53
24.1 Diagrama de flujo	54
24.2 Pseudocódigo	54
24.3 Código en C++	55
25 Ejercicios para resolver en clase	57
25.1 Ejercicio 1	57
25.2 Ejercicio 2	57
25.3 Ejercicio 3	57
25.4 Ejercicio 4	57
25.5 Ejercicio 5	57
25.6 Ejercicio 6	57
26 Resumen de la clase	59
27 Tarea sugerida	61
28 Cierre	63
29 Clase 2. Primeros pasos en C++	65
29.1 Objetivos	65
29.2 Herramienta de trabajo	65

29.3 Nuestro primer programa	65
29.3.1 Explicación	66
29.4 Entrada y salida	66
29.5 Tipos de datos	66
29.6 Ejercicio guiado	66
29.7 Laboratorio	67
29.7.1 Ejercicio 1	67
29.7.2 Ejercicio 2	67
29.7.3 Ejercicio 3	67
29.7.4 Ejercicio 4	67
29.8 Introducción a <code>while</code>	67
29.9 Resumen	68
29.10 Tarea	68
30 Laboratorio 1. Desarrollo de una solución informática	69
31 Laboratorio 1	71
31.1 Desarrollo de una solución informática	71
31.1.1 Objetivo	71
32 Requerimiento	73
33 Actividades a desarrollar	75
33.1 Parte 1. Análisis del requerimiento	75
33.1.1 Entradas	75
33.1.2 Procesos	75
33.1.3 Salidas	76
34 Parte 2. Diseño del algoritmo	77
34.1 Pseudocódigo	77
34.2 Diagrama de flujo	77
35 Parte 3. Implementación	79
36 Parte 4. Casos de prueba	81
36.1 Caso de prueba No. 1	81
36.2 Caso de prueba No. 2	81
37 Evidencia	83
38 Criterios de evaluación	85
39 Conclusión	87
40 Capítulo 3. Expresiones lógicas y estructuras condicionales	89
40.1 Presentación de la clase	89
40.2 Distribución de los 80 minutos	89
41 Objetivos de aprendizaje	91

42 Repaso rápido: expresiones y operadores	93
42.1 Operadores aritméticos	93
42.2 Operadores relacionales	93
42.3 Operadores lógicos	94
42.4 Precedencia simplificada	94
43 Estructura <code>if...else if...else</code>	95
43.1 Sintaxis	95
44 Ejercicio guiado 1: calculadora con <code>if</code>	97
44.1 Requerimiento	97
44.2 Parte A. Análisis	97
44.2.1 Entradas	97
44.2.2 Procesos	97
44.2.3 Salidas	98
44.3 Parte B. Diagrama de flujo	99
44.4 Parte C. Pseudocódigo	100
44.5 Parte D. Código C++	100
45 El mismo ejercicio con <code>switch...case</code>	103
45.1 Sintaxis	103
45.2 Código C++	103
46 Comparación entre <code>if</code> y <code>switch</code>	105
47 Ejercicio guiado 2: clasificación de edad	107
47.1 Requerimiento	107
47.2 Diagrama de flujo	108
47.3 Código C++	109
48 Actividad rápida en parejas	111
49 Errores frecuentes	113
49.1 Usar <code>=</code> en lugar de <code>==</code>	113
49.2 Colocar condición después de <code>else</code>	113
49.3 Olvidar <code>break</code>	113
49.4 Usar <code>switch</code> para rangos	113
50 Resumen de la clase	115
51 Tarea	117
51.1 Problema 1. Menú de conversión	117
51.1.1 Entregables	117
51.2 Problema 2. Clasificación de notas	117
51.2.1 Validación	118
51.2.2 Entregables	118
52 Cierre	119

53 Laboratorio No. 3: ciclos, <code>enum</code> y simulación de un cajero automático	121
53.1 Propósito del laboratorio	121
53.2 Ruta de trabajo	121
53.3 Recordatorio rápido	122
53.3.1 Estructura secuencial	122
53.3.2 Decisión con <code>if</code>	122
53.3.3 Selección múltiple con <code>switch</code>	123
53.3.4 Repetición con <code>while</code>	123
53.4 Ciclo <code>for</code>	124
53.4.1 Ejercicio 1: mostrar los números del 1 al 10	124
53.4.2 Ejercicio 2: tabla de multiplicar	124
53.5 Uso de <code>enum</code>	125
53.5.1 Ventaja principal	126
53.6 Proyecto del laboratorio: cajero automático	127
53.6.1 Situación	127
53.6.2 Operaciones requeridas	127
53.6.3 Reglas	127
53.7 4.1 ¿Por qué utilizar <code>do...while</code> ?	127
53.8 Diseño previo	128
53.9 Diagrama de flujo	129
53.10 Definición de las opciones con <code>enum</code>	130
53.11 Construcción paso a paso	130
53.11.1 Paso 1: variables principales	130
53.11.2 Paso 2: mostrar el menú	130
53.11.3 Paso 3: procesar la opción	130
53.11.4 Paso 4: repetir el menú	131
53.12 5. Programa completo	131
53.13 Pruebas del programa	133
53.14 Reto opcional	133
53.15 Cierre	133
54 Capítulo 5. Integración de conceptos: construyendo un programa completo	135
55 Introducción	137
56 Objetivos	139
57 Caso de estudio	141
57.1 Sistema de Control de Parqueos	141
58 Análisis del problema	143
59 Variables	145
60 Pseudocódigo	147
61 Diagrama: Sistema de control de parqueos	149
62 Paso 1	151

63 Paso 2	153
64 Paso 3. Entrada	155
65 Paso 4. Tarifa	157
66 Paso 5	159
67 Paso 6	161
68 Reflexión	163
69 Resumen	165
70 Paso 7. Mostrando los resultados	167
70.1 Ejemplo de ejecución	168
70.2 Reflexionemos	168
71 Paso 8. Analizando el programa	169
72 Paso 8. ¿Cómo podemos mejorarlo?	171
72.1 Lo que construiremos a continuación	171
72.2 Preparándonos para la siguiente clase	171
73 Capítulo 6. Organizando nuestros programas mediante funciones y procedimientos	173
73.1 Caso de estudio	173
73.1.1 Evolucionando el Sistema de Control de Parqueos	173
74 Introducción	175
75 Objetivos	177
76 ¿Por qué necesitamos organizar nuestros programas?	179
77 La analogía de la construcción de una casa	181
78 Procedimientos y funciones	183
79 ¿Qué es un procedimiento?	185
80 ¿Qué es una función?	187
81 Procedimiento o función	189
82 ¿Qué son los parámetros?	191
83 La analogía de la casa	193
84 Transformando nuestro programa	195
85 Primera mejora: mover el título del programa	197
85.1 Objetivo	197

86 Segunda mejora: separar la lectura de datos	199
86.1 Objetivo	199
87 Tercera mejora: crear una función para determinar la tarifa	201
87.1 Objetivo	201
88 Cuarta mejora: crear una función para calcular el total	203
89 Quinta mejora: crear un procedimiento para mostrar el reporte	205
90 Nuestro nuevo programa	207
91 ¿Qué ganamos con esta organización?	209
92 Primera modificación: crear el procedimiento <code>mostrarTitulo()</code>	211
93 1. Identificamos el código que vamos a mover	213
94 2. Creamos el procedimiento	215
95 3. Modificamos <code>main()</code>	217
96 4. Código después de la primera modificación	219
97 Segunda modificación: crear el procedimiento <code>leerDatos()</code>	223
98 1. Identificamos el código que vamos a mover	225
99 2. Creamos el procedimiento	227
100 3. Modificamos <code>main()</code>	229
101 4. ¿Qué ganamos?	231
101.1 Antes	231
101.2 Después	231
101.3 Preparándonos para la siguiente modificación	232
102 Tercera modificación: crear la función <code>determinarTarifa()</code>	233
103 1. Identificamos el código que vamos a mover	235
104 2. Creamos la función	237
104.1 Analicemos el encabezado	237
105 3. Modificamos <code>main()</code>	239
106 4. ¿Qué sucede internamente?	241
107 5. ¿Qué ganamos?	243
107.1 Antes	243
107.2 Después	243
107.3 Preparándonos para la siguiente modificación	243

108Cuarta modificación: crear la función <code>calcularTotal()</code>	245
1091. Identificamos el código que vamos a mover	247
1102. Creamos la función	249
1113. Modificamos <code>main()</code>	251
1124. ¿Qué sucede internamente?	253
1135. ¿Qué ganamos?	255
113.1Antes	255
113.2Después	255
113.3Preparándonos para la siguiente modificación	256
114Quinta modificación: crear el procedimiento <code>mostrarResultados()</code>	257
1151. Identificamos el código que vamos a mover	259
1162. Creamos el procedimiento	261
1173. Modificamos <code>main()</code>	263
1184. Nuestro nuevo <code>main()</code>	265
1195. ¿Qué cambió?	267
119.1Antes	267
119.2Después	267
120Comprobación final	269
121Lo que aprendimos	271
122Preparándonos para el siguiente capítulo	273
123Laboratorio guiado: desarrollo de una solución	275
124El reto de hoy	277
1251. Comprobemos nuestro análisis	279
125.1Veamos el problema completo	280
125.2Ahora veamos el flujo de la solución	281
1262. Primer código: ¿qué necesitamos para controlar el menú?	283
1273. Construyamos el menú	285
127.1Probemos este primer bloque	286
1284. ¿Qué datos necesita la persona?	287
1295. Provocamos un problema	289

1306. Construyamos nuestras validaciones	291
130.16.1 Actualicemos las librerías	291
130.26.2 Procedimiento para limpiar el buffer	292
130.36.3 Validar un entero	293
130.46.4 Validar un decimal	293
130.56.5 Validar texto	294
130.66.6 Validar un carácter	294
130.76.7 Validar un valor lógico	295
1317. Hagamos funcionar la opción Registrar	297
131.1Probemos otra vez	297
1328. ¿Llenamos <code>main()</code> de instrucciones?	299
1339. Ahora necesitamos obtener un resultado	301
13410. Consultemos el estado	303
13511. Nuestro programa funciona... ¿pero cualquiera puede entrar?	305
135.111.1 Validar usuario	305
135.211.2 Validar usuario y contraseña juntos	305
13612. Control de intentos	307
13713. ¿Cómo quedó organizado nuestro programa?	309
13814. Código completo del laboratorio	311
13915. ¿Qué acabamos de utilizar?	319
14016. Cierre del laboratorio	321
140.1De una variable a un programa completo	321
140.2Reto rápido	321
141Arreglos y vectores: muchas casillas, una sola estructura	323
142El reto de hoy	325
1431. Antes de programar: imaginemos una fila de Excel	327
1442. Dos conceptos que no debemos confundir	329
1453. Llenemos las casillas manualmente	331
1464. Ahora recorramos la fila manualmente	333
1475. De las casillas de Excel a un arreglo en C++	335
1486. Hagamos exactamente lo mismo que hicimos a mano	337
1497. Aparece el ciclo <code>for</code>	339
149.1La conexión importante	339

1508. Ahora llenemos el arreglo con un ciclo	341
1519. Nuestro primer programa completo	343
15210. Ahora que los datos los ingrese el usuario	345
15311. Hagamos operaciones con todos los elementos	347
15412. Actividad guiada en clase	349
15513. Mini reto	351
15614. Cierre de la clase	353
156.1Si comprendimos esto, estamos listos para el laboratorio	353
15715. Para recordar	355
158Síntesis visual del capítulo	357
159Laboratorio: Arreglos y Vectores	361
160Laboratorio: Analizador de notas	363
1611. Situación	365
1622. Requerimiento	367
1633. Ejemplo de funcionamiento	369
1644. Antes de programar	371
164.1¿Qué necesitamos almacenar?	371
164.2¿Cuántos recorridos necesitamos?	371
1655. Plan de trabajo sugerido	373
165.1Etapa 1 — Registrar las notas	373
165.2Etapa 2 — Mostrar el arreglo	373
165.3Etapa 3 — Suma y promedio	373
165.4Etapa 4 — Mayor y menor	374
165.5Etapa 5 — Aprobados y reprobados	374
1666. Tiempo de desarrollo	375
1677. Casos de prueba	377
167.1Prueba 1	377
167.2Prueba 2	377
1688. Preparémonos para explicar	379
1699. Reto adicional	381
17010. Alto aquí	383

171Solución propuesta	385
17211. Analicemos la solución	387
172.1El arreglo	387
172.2El ingreso	387
172.3Mayor y menor	387
172.4Aprobados y reprobados	388
17312. Pregunta final	389
17413. Cierre del laboratorio	391
174.1Misión cumplida	391
175Laboratorio: Modularización del analizador de notas	393
1761. El reto de hoy	395
1772. Del programa completo a un programa modular	397
1783. La estructura que queremos alcanzar	401
1794. Resultado esperado	403
1805. Antes de escribir código	405
1816. Procedimientos y funciones	407
181.1Procedimiento	407
181.2Función	407
1827. ¿Qué información debe viajar entre los módulos?	409
1838. Plan de construcción	411
183.1Etapa 1 — ingresarNotas()	411
183.2Etapa 2 — mostrarNotas()	411
183.3Etapa 3 — calcularPromedio()	412
183.4Etapa 4 — buscarMayor()	412
183.5Etapa 5 — buscarMenor()	412
183.6Etapa 6 — contarAprobados()	413
183.7Etapa 7 — contarReprobados()	413
183.8Etapa 8 — mostrarResultados()	413
1849. Prototipos: el mapa de nuestros módulos	415
18510. Construyamos y probemos	417
18611. Caso de prueba	419
18712. Preparémonos para explicar	421
18813. Compare los dos programas	423
188.1La idea que buscamos descubrir	423

18914. Alto aquí	425
19015. Solución propuesta	427
19116. Analicemos la solución	431
191.1Procedimientos	431
191.2Funciones	431
191.3Parámetros	431
19217. Cierre del laboratorio	433
192.1Lo más importante de hoy	433
193Vectores en C++	435
194El reto de hoy	437
1951. Recordemos el arreglo tradicional	439
1962. ¿Por qué necesitamos otra estructura?	441
1973. ¿Qué es vector?	443
1984. Arreglo y vector	445
1995. <code>push_back()</code> : agregar elementos	447
2006. <code>size()</code> : conocer la cantidad	449
2017. Acceder y recorrer	451
2028. Agregar datos ingresados por el usuario	453
2039. Primera prueba	455
20410. Regresemos al Analizador de notas modularizado	457
20511. El arreglo se convierte en vector	459
20612. Pasar el vector a una función	461
20713. Funciones que solamente consultan	463
20814. Transformemos las funciones	465
208.1Promedio	465
208.2Mayor	465
208.3Menor	465
208.4Aprobados	466
208.5Reprobados	466
20915. El nuevo <code>main()</code>	467
209.1Observa	467

21016. Resultado esperado	469
21117. Solución completa	471
21218. La evolución de nuestra solución	475
21319. Cinco instrucciones para recordar	477
21420. Mini reto	479
21521. Tarea para el viernes	481
215.1Sistema de registro de temperaturas	481
215.1.1Requerimientos	481
215.2Entregables	482
21622. Cierre	483
216.1De tamaño fijo a tamaño dinámico	483
217Vectores paralelos: del control de estudiantes a las matrices	485
218Antes de comenzar	487
219Objetivos de aprendizaje	489
220El problema: control de estudiantes	491
221Primera solución: vectores paralelos	493
222Paso 1. Registrar estudiantes	495
222.1¿Qué está ocurriendo?	495
223Paso 2. Mostrar el listado completo	497
224Paso 3. Buscar un estudiante por carnet	499
224.1El detalle más importante	500
225Paso 4. Mostrar estudiantes aprobados	501
226Paso 5. Calcular el promedio del grupo	503
227Paso 6. Encontrar al estudiante con la nota más alta	505
228Programa integrador	507
229Detengámonos a pensar	511
230El problema cambia de forma	513
231De una dimensión a dos dimensiones	515
232La puerta hacia las matrices	517
233Conexión con los vectores paralelos	519

234Actividad de comprensión	521
235Reto final de la clase	523
236Lo que hemos construido	525
237Próxima clase: matrices	527
238Cierre	529
Laboratorio de Integración I	531
Del problema al programa	531
1. La ruta que seguiremos	531
1.1 Antes de programar	532
Ejercicio 1. Empecemos por lo esencial: variables y operaciones	533
El requerimiento	533
Antes de programar	533
Análisis	533
Pensemos en las variables	534
Pseudocódigo	534
Ahora sí: C++	534
Caso de prueba	535
Ejercicio 2. Ahora debemos tomar una decisión	537
El requerimiento	537
Analicemos antes de escribir código	537
¿Qué cambió respecto al ejercicio anterior?	538
Antes de continuar	538
Pseudocódigo esperado	538
Reto de programación	538
Casos de prueba obligatorios	539
Ejercicio 3. Una decisión puede tener varios caminos	541
El requerimiento	541
No programe todavía	541
Construya la lógica	541
Pruebas mínimas	542
Ejercicio 4. Cuando una tarea debe repetirse	543
El requerimiento	543
Analicemos el problema	543
Preguntas de análisis	544
Simulación manual	544
Primera solución	544
Reto adicional	544
Ejercicio 5. Construyamos un menú	547
El requerimiento	547

Aquí aparecen dos problemas diferentes	547
Antes de programar	547
Diseño general	548
Pregunta importante	548
Reto avanzado	548
Ejercicio 6. Usemos <code>for</code> cuando conocemos las repeticiones	549
El requerimiento	549
Pensemos antes de programar	549
Identifique las variables necesarias	549
Pregunta de razonamiento	550
Reto	550
Reto avanzado	550
Ejercicio 7. Dividamos el problema: procedimientos	551
El requerimiento	551
La pregunta cambia	551
Analicemos responsabilidades	552
Primer objetivo	552
Detengámonos aquí	553
Preguntas de comprensión	553
Ejercicio 8. Funciones y parámetros: piezas reutilizables	555
El problema de nuestra solución anterior	555
8.1 De procedimiento cerrado a procedimiento reutilizable	556
¿Qué cambió?	556
8.2 Ahora necesitamos devolver un resultado	557
8.3 Pensemos en la función como una pequeña máquina	557
8.4 El reto integrador	558
Organización propuesta	558
Estructura inicial	558
8.5 Segundo reto: función para determinar el mayor	559
Preguntas	560
8.6 Tercer reto: función para determinar si un número es par	560
Pregunta importante	560
8.7 Cuarto reto: combine funciones	560
Antes de escribir código	561
Desafío final. Sistema de evaluación de hábitos e IMC	563
Situación	563
El requerimiento	563
Cálculo del IMC	564
Pero el IMC no es todo el problema	564
Recomendaciones que puede generar el programa	565
Ejemplo del reporte esperado	565
Antes de programar: analice	567
A. Comprensión del requerimiento	567
B. Entradas	567

C. Validaciones	567
D. Procesos	567
E. Decisiones	568
F. Repeticiones	568
G. Descomposición del problema	568
Requisitos mínimos de programación	569
Entregables	571
Preparación para la exposición	573
¿Qué hemos recuperado?	575
La idea que debe quedar completa	577
Semáforo de aprendizaje	579
Próximo paso: arreglos y vectores	581
239Matrices y arreglos multidimensionales en C++	583
240Antes de comenzar	585
241El nuevo requerimiento	587
242Pensemos primero en Excel	589
243¿Qué es una matriz?	591
244Comparación con un vector	593
245Recordemos: los índices comienzan en cero	595
246Inicialización de una matriz	597
247¿Cómo llenamos una matriz con datos del usuario?	599
248Sigamos el recorrido paso a paso	601
249Mostrar una matriz como una tabla	603
250Caso de la vida real: registro de calificaciones	605
251Ejemplo 1: promedio por estudiante	607
252Pensemos antes de continuar	609
253Comparación: arreglo, vector y matriz	611
254Ejercicio guiado	613
254.1Paso 1: declarar la matriz	613
254.2Paso 2: ingresar los datos	613

254.3	Paso 3: procesar los datos	614
254.4	Paso 4: calcular el promedio	614
254.5	Paso 5: mostrar resultados	614
255	Buenas prácticas	615
256	Más de dos dimensiones	617
257	Comprobemos la comprensión	619
258	El reto para el laboratorio	621
259	Cierre	623
260	Cierre del capítulo	625
260.1	Próximo paso	625

1 Créditos Editoriales

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas

ALGORITMOS

Introducción y Fundamentos

Editorial Académica

Ingeniería en Sistemas

Ing. Jorge Luis García Chacón

Segundo Semestre

Edición 2026

Diseñar una vez.

Enseñar toda la vida.

2 Acerca de este volumen

Este libro forma parte de la colección **Editorial Académica de Ingeniería en Sistemas**.

Su propósito es proporcionar material de estudio moderno, visual y práctico para apoyar el proceso de enseñanza y aprendizaje.

Cada capítulo ha sido diseñado para combinar teoría, ejemplos, diagramas, laboratorios y ejercicios que permitan al estudiante comprender los conceptos y aplicarlos en situaciones reales.

3 Presentación del curso

Bienvenidos al curso de **Algoritmos** para estudiantes de Ingeniería en Sistemas.

Este documento inicia una ruta de aprendizaje que busca que el estudiante no solo aprenda instrucciones, símbolos o código, sino que desarrolle una forma ordenada de pensar para resolver problemas.

Idea central del curso

Al finalizar el curso, el estudiante debe ser capaz de plantear una solución a un problema mediante un algoritmo y, posteriormente, expresarlo como un programa ejecutado por un computador.

3.1 Objetivo general de la asignatura

Resolver problemas básicos a través de la construcción de programas basados en algoritmos y un lenguaje de programación, generando acciones hacia la búsqueda de propuestas pertinentes.

3.2 Aprendizajes esperados

Al finalizar la asignatura, el estudiante será capaz de:

- Descomponer un problema en subproblemas para resolverlo de forma más sencilla.
- Interpretar algoritmos y programas para resolver problemas básicos de programación.
- Construir algoritmos y programas aplicados a problemas matemáticos sencillos.
- Trabajar colaborativamente, cumpliendo un rol y responsabilizándose de él.
- Comprender situaciones problemáticas que tienen un componente informático.

Consejo del Ingeniero

Antes de escribir código, primero piense cómo resolvería el problema con lápiz y papel. Programar es más fácil cuando el algoritmo ya está claro.

4 ¿Qué es un algoritmo?

Un **algoritmo** es un procedimiento detallado para resolver un problema mediante una serie de pasos ordenados, precisos y finitos.

En términos sencillos:

Un algoritmo indica qué hacer, en qué orden hacerlo y cuándo terminar.

4.1 Elementos básicos de un algoritmo

Todo algoritmo puede analizarse con la siguiente estructura:

Elemento	Pregunta que responde	Ejemplo
Entrada	¿Qué datos necesito?	Dos números
Proceso	¿Qué operaciones realizo?	Sumar los números
Salida	¿Qué resultado entrego?	Resultado de la suma

! Importante

Un algoritmo debe llevarnos desde un **estado inicial** hasta un **estado final**. Si no sabemos dónde empieza o dónde termina, el algoritmo estará incompleto.

4.2 Ejemplo cotidiano

Pensemos en preparar una taza de café.

Entrada

- Agua
- Café
- Azúcar
- Taza

Proceso

1. Calentar el agua.

2. Colocar café en la taza.
3. Agregar agua caliente.
4. Agregar azúcar si se desea.
5. Mezclar.

Salida

- Una taza de café lista para tomar.

Aunque este ejemplo parece simple, ya contiene los tres elementos fundamentales: entrada, proceso y salida.

5 ¿Cómo desarrollar un algoritmo?

Para desarrollar algoritmos se necesita práctica, imaginación y orden.

Algunas ideas clave son:

- **Imaginación:** buscar una forma de resolver el problema.
- **No reinventar la rueda:** aprovechar soluciones conocidas cuando sea posible.
- **Dividir para conquistar:** partir un problema grande en problemas más pequeños.
- **Practicar constantemente:** la lógica se fortalece con ejercicios.

Consejo del Ingeniero

Cuando un problema parezca difícil, no intente resolverlo completo de una vez. Divídalo en partes pequeñas y resuelva una por una.

6 ¿Cómo se describe un algoritmo?

Un algoritmo puede describirse de varias formas.

Forma	Descripción	Uso principal
Lenguaje natural	Explicación con palabras comunes	Primer análisis del problema
Pseudocódigo	Instrucciones ordenadas parecidas a código	Diseño del algoritmo
Diagrama de flujo	Representación gráfica del algoritmo	Visualizar decisiones y procesos
Lenguaje de programación	Código ejecutable por la computadora	Implementación final

Precisión

Un algoritmo no puede ser ambiguo. Todos deben entender lo mismo: el docente, los estudiantes y, finalmente, el computador.

7 Variables: de neuronas a espacios de memoria

Para entender las variables, imaginemos cómo resolveríamos mentalmente una suma de dos números.

Primero pedimos el primer valor y lo recordamos. Luego pedimos el segundo valor y también lo recordamos. Después sumamos ambos valores y guardamos el resultado.

En nuestro cerebro podríamos imaginar tres espacios:

- Espacio 1: primer número.
- Espacio 2: segundo número.
- Espacio 3: resultado.

En programación, esos espacios se llaman **variables**.

Definición

Una variable es un espacio de memoria que permite almacenar un dato para utilizarlo durante la ejecución de un algoritmo o programa.

7.1 Algoritmo para sumar dos números

Problema: Crear un algoritmo que sume dos números ingresados por el usuario.

Entrada

- Primer número.
- Segundo número.

Proceso

- Sumar ambos números.

Salida

- Resultado de la suma.

7.2 Pseudocódigo

```
Algoritmo SumarDosNumeros
Variables
    Entero var1, var2, var3
Inicio
    Escribir "Ingrese el primer valor"
    Leer var1

    Escribir "Ingrese el segundo valor"
    Leer var2

    var3 = var1 + var2

    Escribir "El resultado es: ", var3
Fin
```

7.3 Explicación paso a paso

Paso	Acción	Resultado
1	Definir variables	var1, var2, var3 existen en memoria
2	Leer primer valor	var1 almacena el primer número
3	Leer segundo valor	var2 almacena el segundo número
4	Sumar	var3 almacena var1 + var2
5	Mostrar resultado	Se imprime var3

8 Ejercicio guiado: multiplicar tres números

Problema: Crear un algoritmo que multiplique tres números ingresados por teclado.

8.1 Análisis

Entrada

- Primer número.
- Segundo número.
- Tercer número.

Proceso

- Multiplicar los tres valores.

Salida

- Resultado de la multiplicación.

8.2 Pseudocódigo

```
Algoritmo MultiplicarTresNumeros
Variables
    Entero var1, var2, var3, var4
Inicio
    Escribir "Ingresa el primer valor"
    Leer var1

    Escribir "Ingresa el segundo valor"
    Leer var2

    Escribir "Ingresa el tercer valor"
    Leer var3
```

```
var4 = var1 * var2 * var3  
Escribir "El resultado es: ", var4  
Fin
```

 Observe

Para multiplicar tres números necesitamos cuatro variables: tres para guardar las entradas y una para guardar el resultado.

9 Diagramas de flujo

Un **diagrama de flujo** es una representación gráfica de la lógica de un algoritmo.

Permite observar el orden de las instrucciones, las entradas, los procesos, las salidas y las decisiones.

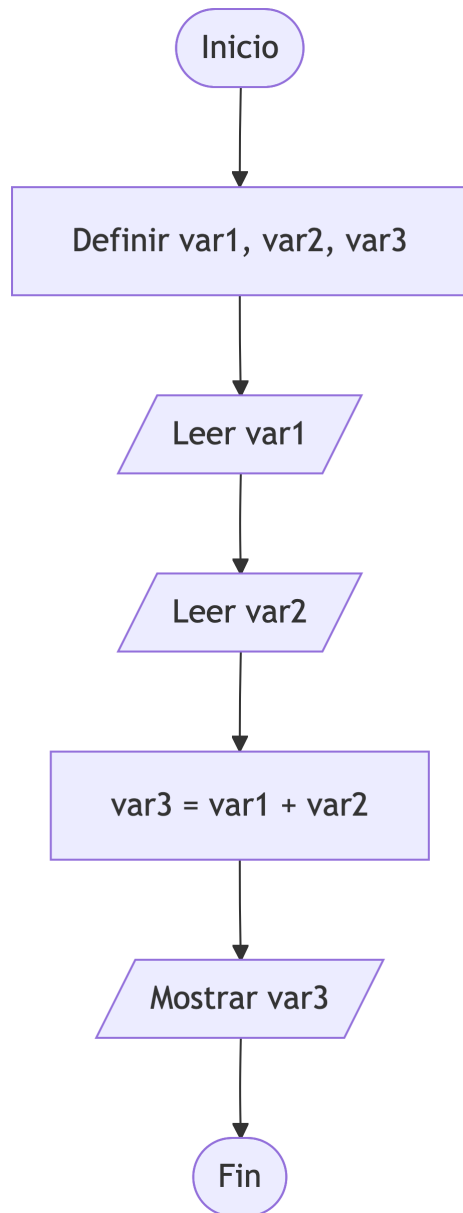
9.1 Nomenclatura básica

Símbolo	Nombre	Significado
□	Óvalo	Inicio o fin del algoritmo
□	Paralelogramo	Entrada o salida de datos
□	Rectángulo	Proceso u operación
□	Rombo	Decisión o bifurcación
↓ / →	Flechas	Dirección del flujo
•	Círculo pequeño	Conector

i Nota para el docente

En el PDF se utilizan símbolos simples para que el documento sea liviano y fácil de renderizar. En clase, estos mismos símbolos se explican con dibujos más grandes en la pizarra o en diagramas de flujo.

9.2 Ejemplo gráfico del flujo para sumar dos números



9.3 Ejemplo textual del flujo para sumar dos números

```
Inicio
↓
Definir var1, var2, var3
↓
Leer var1
```

```
↓  
Leer var2  
↓  
var3 = var1 + var2  
↓  
Mostrar var3  
↓  
Fin
```

i Para la clase

Primero puede mostrarse el diagrama gráfico y luego el flujo textual. La idea es que el estudiante relacione cada símbolo con una acción concreta del algoritmo.

10 Bifurcaciones y decisiones

Una **bifurcación** permite que el algoritmo tome caminos diferentes según una condición.

Las condiciones siempre se evalúan como:

- Verdadero.
- Falso.

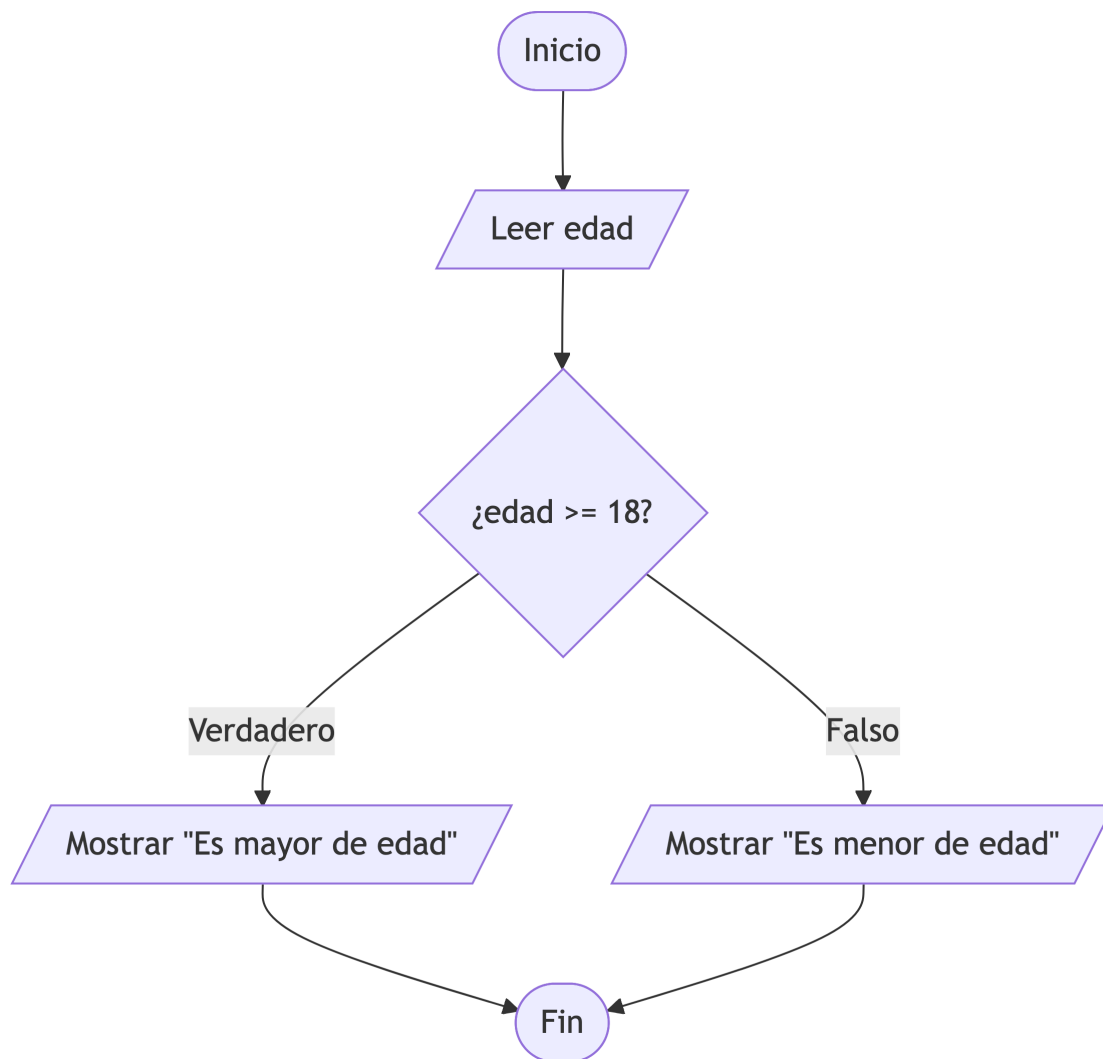
10.1 Ejemplo: mayor de edad

Problema: Crear un algoritmo donde una persona ingrese su edad y se muestre si es mayor de edad.

```
Algoritmo MayorDeEdad
Variables
    Entero edad
Inicio
    Escribir "Ingrese su edad"
    Leer edad

    Si edad >= 18 Entonces
        Escribir "Es mayor de edad"
    Sino
        Escribir "Es menor de edad"
    FinSi
Fin
```

10.2 Diagrama de flujo



10.3 Flujo lógico

```
Inicio
↓
Leer edad
↓
¿edad >= 18?
├── Verdadero → Mostrar "Es mayor de edad"
└── Falso     → Mostrar "Es menor de edad"
↓
Fin
```

! Idea clave

En las bifurcaciones, el algoritmo no ejecuta todo. Ejecuta solamente el camino que corresponde al resultado de la condición.

11 Operadores

Las condiciones se construyen utilizando operadores.

En esta primera clase revisaremos tres grupos:

- Operadores aritméticos.
- Operadores relacionales.
- Operadores lógicos.

12 Operadores aritméticos

Operador	Significado	Ejemplo	Resultado
*	Multiplicación	5 * 2	10
/	División	10 / 2	5
+	Suma	4 + 3	7
-	Resta	8 - 5	3
%	Módulo o residuo	5 % 2	1

12.1 Módulo o residuo

El operador % devuelve el residuo de una división entera.

```
5 % 2 = 1
4 % 2 = 0
```

Uso frecuente

El operador módulo se utiliza mucho para saber si un número es par o impar.
Si `numero % 2 == 0`, entonces el número es par.

13 Operadores relacionales

Los operadores relacionales comparan valores y devuelven un resultado lógico: verdadero o falso.

Operador	Significado	Ejemplo
>	Mayor que	edad > 18
<	Menor que	edad < 18
>=	Mayor o igual que	edad >= 18
<=	Menor o igual que	nota <= 60
==	Igual que	a == b
!=	Diferente de	a != b

Error común

No es lo mismo = que ==.

- = se utiliza para asignar.
- == se utiliza para comparar.

14 Operadores lógicos

Los operadores lógicos permiten combinar condiciones.

Operador	Nombre	Significado práctico
& &	Y / AND	Exige que ambas condiciones sean verdaderas
	O / OR	Basta con que una condición sea verdadera
!	NO / NOT	Invierte el valor lógico

14.1 Operador Y: &&

El operador && es verdadero solamente cuando ambas condiciones son verdaderas.

A	B	A && B
V	V	V
V	F	F
F	V	F
F	F	F

Forma de recordarlo

El operador && es **exigente**: pide que todo se cumpla.

14.2 Operador O: | |

El operador | | es verdadero cuando al menos una condición es verdadera.

A	B	A B
V	V	V
V	F	V
F	V	V
F	F	F

i Forma de recordarlo

El operador `||` es **flexible**: acepta que se cumpla una condición o la otra.

14.3 Operador NO: `!`

El operador `!` invierte el valor lógico.

A	<code>!A</code>
V	F
F	V

i Forma de recordarlo

El operador `!` voltea la respuesta: lo verdadero se vuelve falso y lo falso se vuelve verdadero.

15 Casos de análisis

A continuación se presentan casos para practicar el análisis de expresiones lógicas.

La idea no es memorizar la respuesta, sino resolver paso a paso.

16 Caso 1: operador &&

Datos iniciales:

```
var1 = 2  
var2 = 0  
var3 = 4
```

Operación:

```
var4 = (var1 * var1) / var3
```

Sustituyendo valores:

```
var4 = (2 * 2) / 4  
var4 = 4 / 4  
var4 = 1
```

Condición:

```
var4 >= 1 && var1 != 0
```

Análisis:

```
var4 >= 1 → 1 >= 1 → Verdadero  
var1 != 0 → 2 != 0 → Verdadero
```

Resultado final:

```
Verdadero && Verdadero = Verdadero
```

Conclusión

En este caso, el resultado de la condición es **Verdadero**.

17 Caso 2: combinación de || y &&

Datos iniciales:

```
var1 = 2  
var2 = 0  
var3 = 4
```

Operación:

```
var4 = var3 % var1
```

Sustituyendo valores:

```
var4 = 4 % 2  
var4 = 0
```

Condición:

```
(var4 != 0 || var2 > 0) && (var3 == 4)
```

Análisis:

```
var4 != 0 → 0 != 0 → Falso  
var2 > 0 → 0 > 0 → Falso
```

Primera parte:

```
Falso || Falso = Falso
```

Segunda parte:

```
var3 == 4 → 4 == 4 → Verdadero
```

Resultado final:

```
Falso && Verdadero = Falso
```

 Conclusión

En este caso, el resultado de la condición es **Falso**.

18 Caso 3: operador !

Datos iniciales:

```
var1 = 2  
var4 = 1  
var3 = 4
```

Operación:

```
var4 = var4 + var1
```

Sustituyendo valores:

```
var4 = 1 + 2  
var4 = 3
```

Condición original:

```
!(var1 && (var4 >= var1))
```

Para estudiantes que están iniciando, conviene escribirla de forma más explícita:

```
!(var1 != 0 && (var4 >= var1))
```

Análisis:

```
var1 != 0      → 2 != 0 → Verdadero  
var4 >= var1   → 3 >= 2 → Verdadero
```

Primera parte:

```
Verdadero && Verdadero = Verdadero
```

Aplicando el operador NO:

```
!Verdadero = Falso
```

Conclusión

En este caso, el resultado de la condición es **Falso**.

Nota de C++

En C++:

- 0 se interpreta como falso.
- Cualquier valor distinto de 0 se interpreta como verdadero.

Por eso una expresión como `if (var1)` puede funcionar, aunque para iniciar es mejor escribir condiciones explícitas como `var1 != 0`.

19 Prioridad de operadores

Es importante notar que existen prioridades entre operadores.

Por ejemplo, no siempre se evalúa de izquierda a derecha sin reglas. Por eso, cuando una expresión tenga varias operaciones, es recomendable usar paréntesis.

! Recomendación

Use paréntesis cuando una condición combine varios operadores. Ayuda al computador y también ayuda a quien lee el algoritmo.

Ejemplo más claro:

```
(var4 != 0 || var2 > 0) && (var3 == 4)
```

Ejemplo menos claro:

```
var4 != 0 || var2 > 0 && var3 == 4
```


20 Introducción a ciclos

Un **ciclo** es la repetición de un conjunto de instrucciones.

El ciclo termina cuando se cumple una condición de finalización.

20.1 ¿Cuándo se aplican los ciclos?

Los ciclos se aplican cuando queremos ejecutar un conjunto de instrucciones varias veces.

Por ejemplo:

- Sumar varios números.
- Pedir datos de varios estudiantes.
- Buscar el número mayor de una lista.
- Repetir un menú hasta que el usuario decida salir.

i Idea clave

Si una acción debe repetirse, probablemente necesitamos un ciclo.

21 Ejemplo: sumar N números

Problema: Crear un algoritmo que sume N números ingresados por teclado.

21.1 Análisis

Entrada

- Cantidad de números a sumar: N .
- Cada uno de los números.

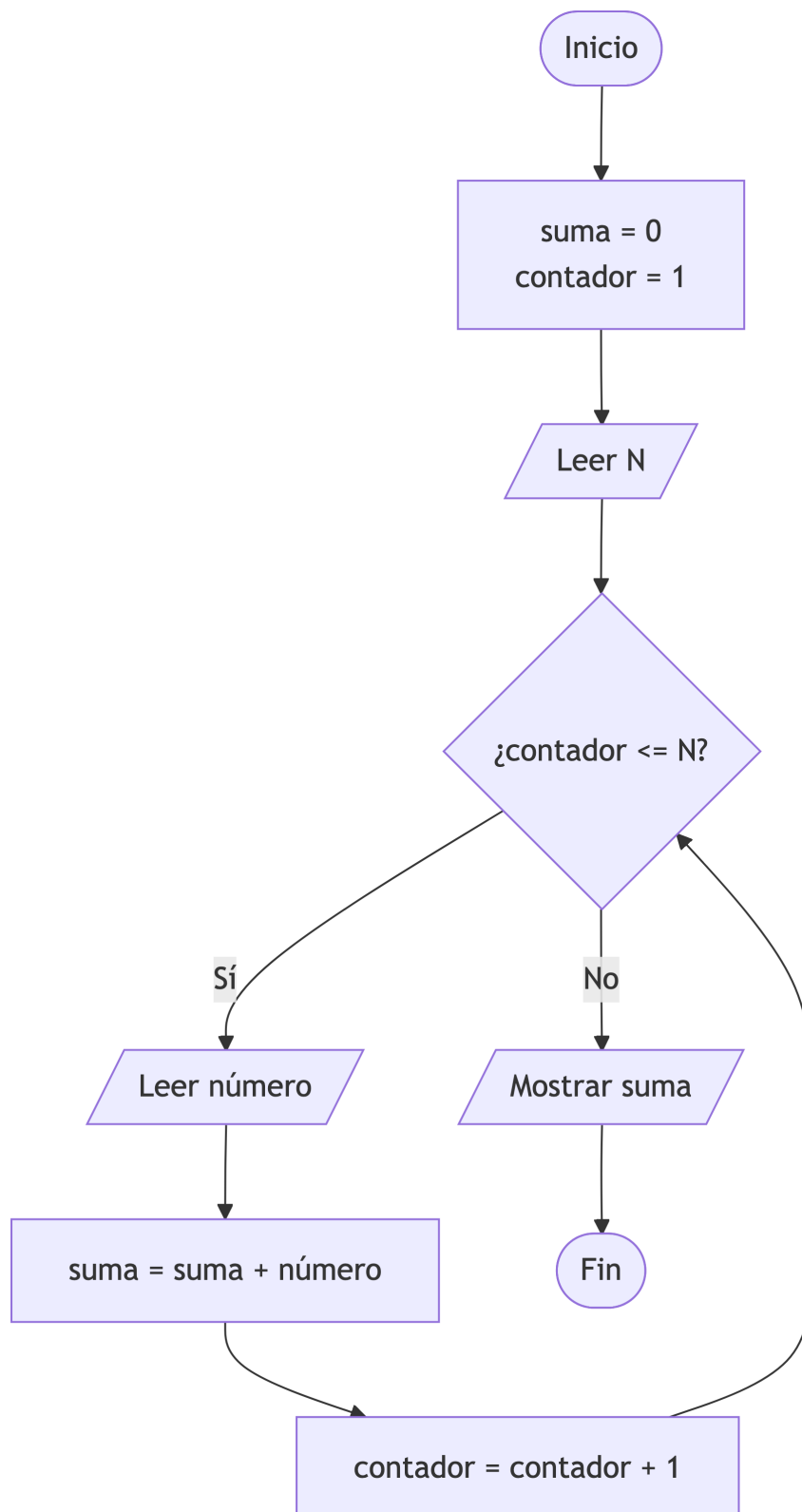
Proceso

- Repetir la lectura de números.
- Acumular la suma.

Salida

- Total de la suma.

21.2 Diagrama de flujo



21.3 Pseudocódigo

```
Algoritmo SumarNNumeros
Variables
    Entero N, contador, numero, suma
Inicio
    suma = 0
    contador = 1

    Escribir "Ingrese cuántos números desea sumar"
    Leer N

    Mientras contador <= N Hacer
        Escribir "Ingrese un número"
        Leer numero
        suma = suma + numero
        contador = contador + 1
    FinMientras

    Escribir "La suma total es: ", suma
Fin
```


22 Ejemplo en C++: sumar del 1 hasta N

```
#include <iostream>
using namespace std;

int main() {
    int N, suma = 0, cont = 1;

    cout << "Ingrese hasta qué número desea sumar: ";
    cin >> N;

    while (cont <= N) {
        suma += cont;
        cont++;
    }

    cout << "El resultado es: " << suma << endl;
    return 0;
}
```

i Nota

En C++, endl permite hacer un salto de línea. También puede usarse "\n".

23 Ejemplo en C++: encontrar el mayor de N números

```
#include <iostream>
using namespace std;

int main() {
    int N, var, mayor, cont = 1;

    cout << "Ingrese cuántos números va a ingresar: ";
    cin >> N;

    cout << "Ingrese el valor 1: ";
    cin >> var;
    mayor = var;

    while (cont < N) {
        cout << "Ingrese el valor " << cont + 1 << ": ";
        cin >> var;

        if (var > mayor) {
            mayor = var;
        }

        cont++;
    }

    cout << "El número mayor es: " << mayor << endl;

    return 0;
}
```

Observe

El primer número se guarda inicialmente como `mayor`. Luego, cada nuevo número se compara contra ese valor.

24 Programa de aplicación: ecuación de primer grado

Resolver una ecuación de primer grado de la forma:

$$a x + b = 0$$

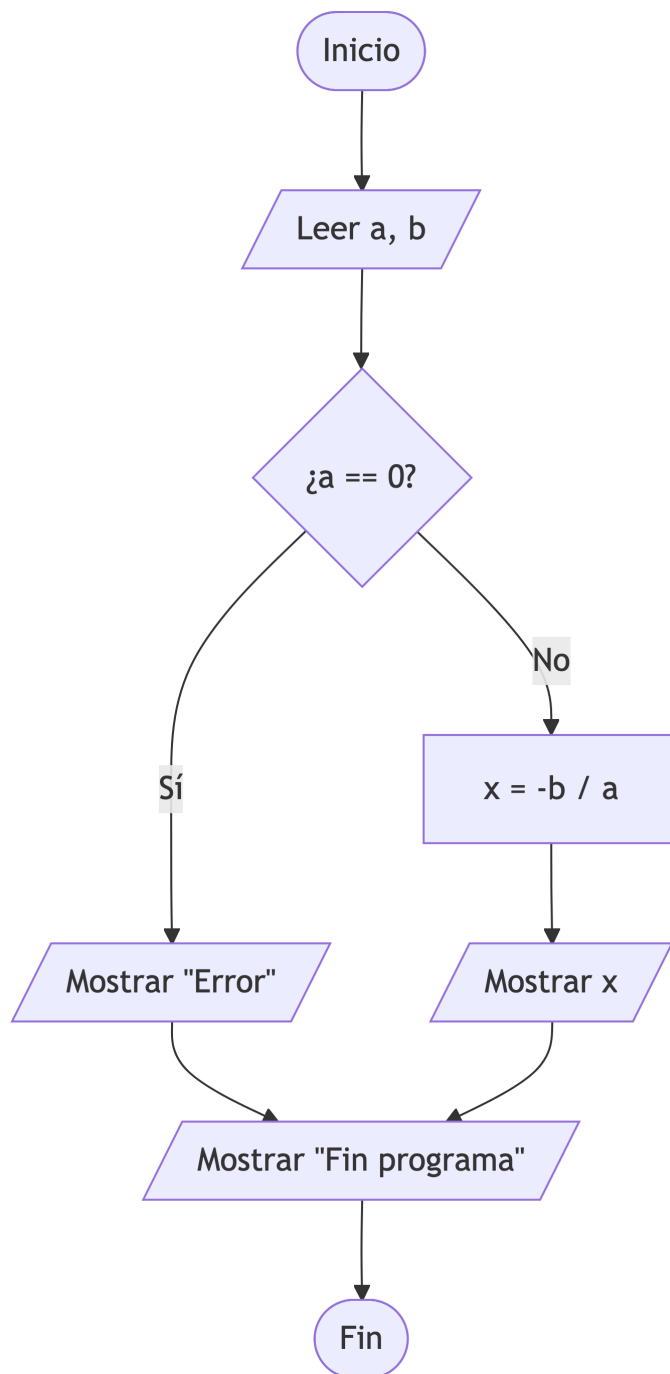
La solución es:

$$x = -b / a$$

Pero existe un caso especial:

Si $a = 0$, la división no puede realizarse.

24.1 Diagrama de flujo



24.2 Pseudocódigo

```
Algoritmo EcuacionPrimerGrado
Variables
    Entero a, b, x
Inicio
    Escribir "Ingrese valores de a y b"
    Leer a, b

    Si a == 0 Entonces
        Escribir "Error: no se puede dividir entre cero"
    Sino
        x = -b / a
        Escribir "La solución es: ", x
    FinSi

    Escribir "Fin del programa"
Fin
```

24.3 Código en C++

```
#include <iostream>
using namespace std;

int main() {
    int a, b, x;

    cout << "Ingrese valores de EC (a y b): ";
    cin >> a >> b;

    if (a == 0) {
        cout << "Error" << endl;
    } else {
        x = -b / a;
        cout << "La solución es: " << x << endl;
    }

    cout << "Fin programa" << endl;
    return 0;
}
```

Observación importante

En este ejemplo `x` está declarada como entero. Más adelante puede mejorarse usando variables reales para obtener resultados decimales.

25 Ejercicios para resolver en clase

25.1 Ejercicio 1

Cree un algoritmo que multiplique tres números ingresados por teclado y muestre el resultado.

25.2 Ejercicio 2

Cree un algoritmo que divida dos números ingresados por teclado y muestre el resultado.

Tome en cuenta que no se debe dividir entre cero.

25.3 Ejercicio 3

Cree un algoritmo que permita sumar tres números ingresados por teclado y muestre el resultado.

25.4 Ejercicio 4

Cree un algoritmo donde una persona ingrese su edad y muestre si es mayor de edad.

25.5 Ejercicio 5

Cree un algoritmo que muestre en forma ordenada tres números enteros ingresados desde teclado, de mayor a menor.

25.6 Ejercicio 6

Cree un algoritmo que permita sumar N números y muestre el resultado. El valor de N debe ser ingresado por teclado, al igual que los números que se sumarán.

26 Resumen de la clase

En esta clase aprendimos que:

- Un algoritmo es una secuencia finita, ordenada y precisa de pasos.
- Todo algoritmo trabaja con entradas, procesos y salidas.
- Las variables permiten almacenar datos en memoria.
- El pseudocódigo permite expresar soluciones antes de programarlas.
- Los diagramas de flujo ayudan a visualizar la lógica del algoritmo.
- Las bifurcaciones permiten tomar decisiones.
- Los operadores aritméticos, relacionales y lógicos permiten construir condiciones.
- Los ciclos permiten repetir instrucciones.

Consejo final del Ingeniero

Un buen programador no es quien escribe código más rápido, sino quien piensa mejor la solución antes de programarla.

27 Tarea sugerida

Para reforzar la clase, resuelva los siguientes problemas en pseudocódigo y diagrama de flujo:

1. Calcular el área de un rectángulo.
2. Calcular el promedio de tres notas.
3. Determinar si un número es par o impar.
4. Determinar el mayor de dos números.
5. Sumar los números del 1 hasta N.

28 Cierre

Algoritmos – De la lógica a la programación

Material preparado para el curso de Algoritmos, Ingeniería en Sistemas, Universidad Mariano Gálvez.

Docente: Ing. Jorge Luis García Chacón

29 Clase 2. Primeros pasos en C++

Laboratorio No. 1

29.1 Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Utilizar OnlineGDB para escribir y ejecutar programas.
- Comprender la estructura básica de un programa en C++.
- Declarar variables.
- Leer datos desde el teclado.
- Mostrar resultados en pantalla.

29.2 Herramienta de trabajo

Trabajaremos utilizando:

https://www.onlinegdb.com/online_c++_compiler

29.3 Nuestro primer programa

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hola Ingeniería en Sistemas" << endl;
    return 0;
}
```

29.3.1 Explicación

Línea	Descripción
<code>#include <iostream></code>	Biblioteca para entrada y salida.
<code>using namespace std;</code>	Permite utilizar <code>cout</code> y <code>cin</code> sin escribir <code>std::</code> .
<code>int main()</code>	Punto de inicio del programa.
<code>{ }</code>	Delimitan el bloque principal.
<code>return 0;</code>	Finaliza correctamente el programa.

29.4 Entrada y salida

```
#include <iostream>
using namespace std;

int main()
{
    int edad;

    cout << "Ingrese su edad: ";
    cin >> edad;

    cout << "Usted tiene " << edad << " años." << endl;

    return 0;
}
```

29.5 Tipos de datos

Tipo	Ejemplo
int	25
float	25.5
double	3.14159265
char	'A'
string	"Guatemala"
bool	true

29.6 Ejercicio guiado

Crear un programa que solicite:

- Nombre
- Edad
- Carrera

y muestre la información ingresada.

29.7 Laboratorio

29.7.1 Ejercicio 1

Solicite dos números enteros y muestre su suma.

29.7.2 Ejercicio 2

Solicite tres números y muestre su multiplicación.

29.7.3 Ejercicio 3

Calcule el área de un rectángulo.

29.7.4 Ejercicio 4

Calcule el promedio de tres notas.

29.8 Introducción a while

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;

    while(i <= 5)
    {
        cout << i << endl;
        i++;
    }

    return 0;
}
```

Observe: Si elimina `i++`; el ciclo nunca finalizará.

29.9 Resumen

En esta clase aprendimos:

- La estructura de un programa en C++.
- Uso de `cout` y `cin`.
- Declaración de variables.
- Tipos de datos.
- Primer ciclo `while`.

29.10 Tarea

1. Programa que calcule el perímetro de un rectángulo.
2. Programa que calcule el promedio de cuatro notas.
3. Programa que determine si un número es positivo o negativo.
4. Programa que convierta grados Celsius a Fahrenheit.
5. Modifique el ejemplo del ciclo `while` para mostrar los números del 1 al 10.

30 Laboratorio 1. Desarrollo de una solución informática

Análisis, diseño e implementación de algoritmos

31 Laboratorio 1

31.1 Desarrollo de una solución informática

31.1.1 Objetivo

Aplicar los conocimientos adquiridos durante las primeras clases del curso mediante el análisis, diseño e implementación de una solución informática utilizando pseudocódigo, diagrama de flujo y el lenguaje de programación C++.

! Competencias que se evaluarán

Al finalizar este laboratorio el estudiante será capaz de:

- Analizar un requerimiento.
- Identificar entradas, procesos y salidas.
- Diseñar un algoritmo mediante pseudocódigo.
- Representar la solución utilizando un diagrama de flujo.
- Implementar el algoritmo en C++.
- Verificar el funcionamiento de la solución mediante casos de prueba.

32 Requerimiento

Una academia de capacitación desea automatizar el cálculo del promedio de sus estudiantes.

Para ello necesita un programa que solicite la siguiente información:

- Nombre del estudiante.
- Primera nota.
- Segunda nota.
- Tercera nota.

Posteriormente el sistema deberá:

1. Calcular el promedio de las tres notas.
2. Mostrar el promedio obtenido.
3. Indicar si el estudiante **Aprobó** o **Reprobó**.

Considere que:

- Un estudiante aprueba con un promedio mayor o igual a **61 puntos**.
 - En cualquier otro caso el estudiante reprueba.
-

33 Actividades a desarrollar

33.1 Parte 1. Análisis del requerimiento

Antes de escribir cualquier algoritmo complete la siguiente información.

33.1.1 Entradas

Escriba las entradas que necesita el sistema.

No.	Entrada
1	
2	
3	
4	

33.1.2 Procesos

Describa las operaciones que realizará el algoritmo.

No.	Proceso
1	
2	
3	

33.1.3 Salidas

Indique qué información mostrará el programa.

No.	Salida
1	
2	
3	

34 Parte 2. Diseño del algoritmo

34.1 Pseudocódigo

Diseñe el algoritmo utilizando pseudocódigo.

34.2 Diagrama de flujo

Diseñe el diagrama de flujo correspondiente.

35 Parte 3. Implementación

Desarrolle el programa utilizando el lenguaje **C++**.

El programa debe cumplir con todos los requerimientos indicados anteriormente.

Recuerde

Utilice únicamente los temas vistos en clase:

- Variables
- Entrada de datos (`cin`)
- Salida de datos (`cout`)
- Operadores aritméticos
- Operadores relacionales
- Estructuras de decisión (`if/else`)

No es necesario utilizar ciclos.

36 Parte 4. Casos de prueba

Ejecute el programa al menos dos veces utilizando diferentes datos.

36.1 Caso de prueba No. 1

Dato	Valor
Nombre	
Nota 1	
Nota 2	
Nota 3	
Promedio esperado	
Resultado esperado	

36.2 Caso de prueba No. 2

Dato	Valor
Nombre	
Nota 1	
Nota 2	
Nota 3	
Promedio esperado	
Resultado esperado	

37 Evidencia

Adjunte una captura de pantalla del programa en ejecución.

38 Criterios de evaluación

Criterio	Punteo
Identificación de entradas, procesos y salidas	15
Pseudocódigo	20
Diagrama de flujo	20
Programa en C++	35
Casos de prueba	10
Total	100

39 Conclusión

Describa brevemente qué aprendió durante el desarrollo del laboratorio.

Reflexión

En el desarrollo de software, escribir el código representa únicamente una parte del trabajo. Antes de programar, un desarrollador debe comprender el requerimiento, analizar la información disponible, diseñar una solución y posteriormente implementarla. Este laboratorio busca fortalecer esa forma de pensar, la cual será utilizada durante todo el curso.

40 Capítulo 3. Expresiones lógicas y estructuras condicionales

40.1 Presentación de la clase

En las clases anteriores aprendimos a identificar entradas, procesos y salidas; declarar variables; utilizar operadores aritméticos; trabajar con `DIV` y `MOD`; construir expresiones; y representar una solución mediante pseudocódigo, diagrama de flujo y código C++.

En esta clase daremos continuidad a esos temas incorporando las **expresiones lógicas** y las estructuras de decisión:

- `if`;
- `if...else`;
- `if...else if...else`;
- `switch...case`.

La meta no es memorizar sintaxis. La meta es aprender a decidir **qué estructura conviene usar según el problema**.

Idea central

Una estructura condicional permite que el algoritmo tome caminos diferentes según el valor de una condición.

40.2 Distribución de los 80 minutos

Tiempo	Actividad
12 minutos	Introducción y repaso conceptual
18 minutos	Ejercicio guiado 1 con <code>if...else if...else</code>
18 minutos	El mismo ejercicio con <code>switch...case</code>
20 minutos	Ejercicio guiado 2: condiciones compuestas
7 minutos	Comparación, errores frecuentes y cierre
5 minutos	Explicación de la tarea

 Metodología de la clase

La explicación representa aproximadamente el 15 % de la sesión. El resto se dedica a analizar, diseñar, programar y probar soluciones.

41 Objetivos de aprendizaje

Al finalizar la clase, el estudiante será capaz de:

1. Evaluar expresiones lógicas.
2. Aplicar la precedencia de operadores.
3. Utilizar `if`, `else if` y `else`.
4. Utilizar `switch`, `case`, `break` y `default`.
5. Diferenciar cuándo conviene usar `if` y cuándo `switch`.
6. Elaborar el análisis de un problema.
7. Diseñar la solución mediante pseudocódigo y diagrama de flujo.
8. Implementar y probar la solución en C++.

42 Repaso rápido: expresiones y operadores

Una **expresión** es una combinación de variables, constantes y operadores que produce un valor.

```
int resultado = 3 + 4 * 2;
```

La multiplicación se evalúa antes que la suma:

```
3 + 4 * 2
3 + 8
11
```

42.1 Operadores aritméticos

Operador	Descripción	Ejemplo
+	Suma	10 + 3 = 13
-	Resta	10 - 3 = 7
*	Multiplicación	10 * 3 = 30
/	División	10 / 3 = 3 si ambas variables son enteras
%	Residuo	10 % 3 = 1

42.2 Operadores relacionales

Operador	Significado
>	Mayor que
<	Menor que
>=	Mayor o igual que
<=	Menor o igual que
==	Igual que
!=	Diferente de

42.3 Operadores lógicos

Operador	Nombre	Uso
& &	AND	Todas las condiciones deben ser verdaderas
	OR	Al menos una condición debe ser verdadera
!	NOT	Invierte el resultado lógico

42.4 Precedencia simplificada

Nivel	Operadores
1	()
2	!
3	*, /, %
4	+, -
5	<, >, <=, >=
6	==, !=
7	& &
8	
9	=

Consejo

Cuando una expresión sea larga, utilice paréntesis. Aunque el resultado sea correcto, los paréntesis hacen que la intención sea más clara.

43 Estructura `if...else if...else`

Se utiliza cuando necesitamos evaluar rangos, varias condiciones, operadores relacionales u operadores lógicos.

43.1 Sintaxis

```
if (condicion1) {  
    // instrucciones  
} else if (condicion2) {  
    // instrucciones  
} else {  
    // instrucciones  
}
```


44 Ejercicio guiado 1: calculadora con `if`

44.1 Requerimiento

Crear un programa que:

1. Solicite dos números.
2. Solicite una operación: +, -, *, / o %.
3. Realice la operación seleccionada.
4. Evite dividir entre cero.
5. Muestre un mensaje si la operación no es válida.

44.2 Parte A. Análisis

44.2.1 Entradas

Dato	Tipo
Primer número	Entero
Segundo número	Entero
Operación	Carácter

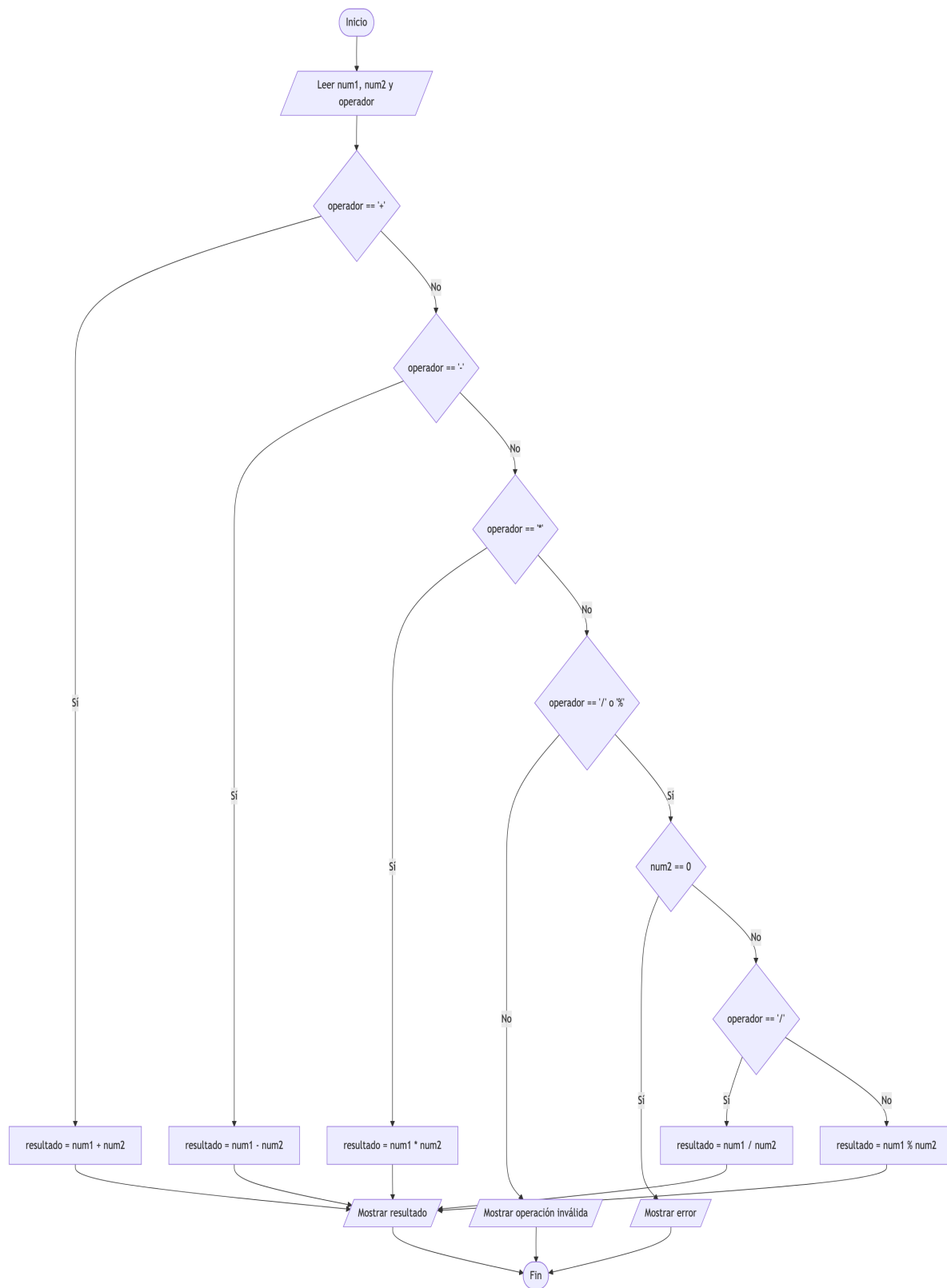
44.2.2 Procesos

1. Leer los dos números.
2. Leer la operación.
3. Comparar la operación ingresada.
4. Ejecutar el cálculo correspondiente.
5. Validar división y residuo entre cero.

44.2.3 Salidas

- Resultado de la operación.
- Mensaje de error cuando corresponda.

44.3 Parte B. Diagrama de flujo



44.4 Parte C. Pseudocódigo

```
Algoritmo CalculadoraConIf
Variables
    Entero num1, num2, resultado
    Caracter operador

Inicio
    Leer num1
    Leer num2
    Leer operador

    Si operador == '+' Entonces
        resultado = num1 + num2
        Escribir resultado
    Sino Si operador == '-' Entonces
        resultado = num1 - num2
        Escribir resultado
    Sino Si operador == '*' Entonces
        resultado = num1 * num2
        Escribir resultado
    Sino Si operador == '/' Entonces
        Si num2 == 0 Entonces
            Escribir "Error"
        Sino
            resultado = num1 / num2
            Escribir resultado
        FinSi
    Sino Si operador == '%' Entonces
        Si num2 == 0 Entonces
            Escribir "Error"
        Sino
            resultado = num1 % num2
            Escribir resultado
        FinSi
    Sino
        Escribir "Operación no válida"
    FinSi
Fin
```

44.5 Parte D. Código C++

```
#include <iostream>
using namespace std;
```



```
int main() {
    int num1, num2, resultado;
    char operador;

    cout << "Ingrese el primer numero: ";
    cin >> num1;

    cout << "Ingrese el segundo numero: ";
    cin >> num2;

    cout << "Ingrese la operacion (+, -, *, /, %): ";
    cin >> operador;

    if (operador == '+') {
        resultado = num1 + num2;
        cout << "Resultado: " << resultado << endl;
    } else if (operador == '-') {
        resultado = num1 - num2;
        cout << "Resultado: " << resultado << endl;
    } else if (operador == '*') {
        resultado = num1 * num2;
        cout << "Resultado: " << resultado << endl;
    } else if (operador == '/') {
        if (num2 == 0) {
            cout << "Error: no se puede dividir entre cero." << endl;
        } else {
            resultado = num1 / num2;
            cout << "Resultado: " << resultado << endl;
        }
    } else if (operador == '%') {
        if (num2 == 0) {
            cout << "Error: no se puede obtener residuo entre cero."
                << endl;
        } else {
            resultado = num1 % num2;
            cout << "Resultado: " << resultado << endl;
        }
    } else {
        cout << "Operacion no valida." << endl;
    }

    return 0;
}
```

}

45 El mismo ejercicio con `switch...case`

`switch` es apropiado cuando una misma variable se compara contra valores constantes exactos.

45.1 Sintaxis

```
switch (expresion) {  
    case valor1:  
        // instrucciones  
        break;  
  
    case valor2:  
        // instrucciones  
        break;  
  
    default:  
        // instrucciones  
}
```

45.2 Código C++

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int num1, num2, resultado;  
    char operador;  
  
    cout << "Ingrese el primer numero: ";  
    cin >> num1;  
  
    cout << "Ingrese el segundo numero: ";  
    cin >> num2;
```

```
cout << "Ingrese la operacion (+, -, *, /, %): ";
cin >> operador;

switch (operador) {
    case '+':
        resultado = num1 + num2;
        cout << "Resultado: " << resultado << endl;
        break;

    case '-':
        resultado = num1 - num2;
        cout << "Resultado: " << resultado << endl;
        break;

    case '*':
        resultado = num1 * num2;
        cout << "Resultado: " << resultado << endl;
        break;

    case '/':
        if (num2 == 0) {
            cout << "Error: no se puede dividir entre cero."
                << endl;
        } else {
            resultado = num1 / num2;
            cout << "Resultado: " << resultado << endl;
        }
        break;

    case '%':
        if (num2 == 0) {
            cout << "Error: no se puede obtener residuo entre cero."
                << endl;
        } else {
            resultado = num1 % num2;
            cout << "Resultado: " << resultado << endl;
        }
        break;

    default:
        cout << "Operacion no valida." << endl;
}

return 0;
}
```

46 Comparación entre `if` y `switch`

Característica	<code>if...else if...else</code>	<code>switch...case</code>
Evalúa rangos	Sí	No directamente
Usa condiciones lógicas	Sí	No
Compara valores exactos	Sí	Sí
Claridad en menús	Buena	Muy buena
Uso recomendado	Rangos y condiciones complejas	Opciones exactas

! Regla práctica

Use `if` cuando la decisión dependa de rangos o expresiones lógicas. Use `switch` cuando una variable se compare contra opciones exactas.

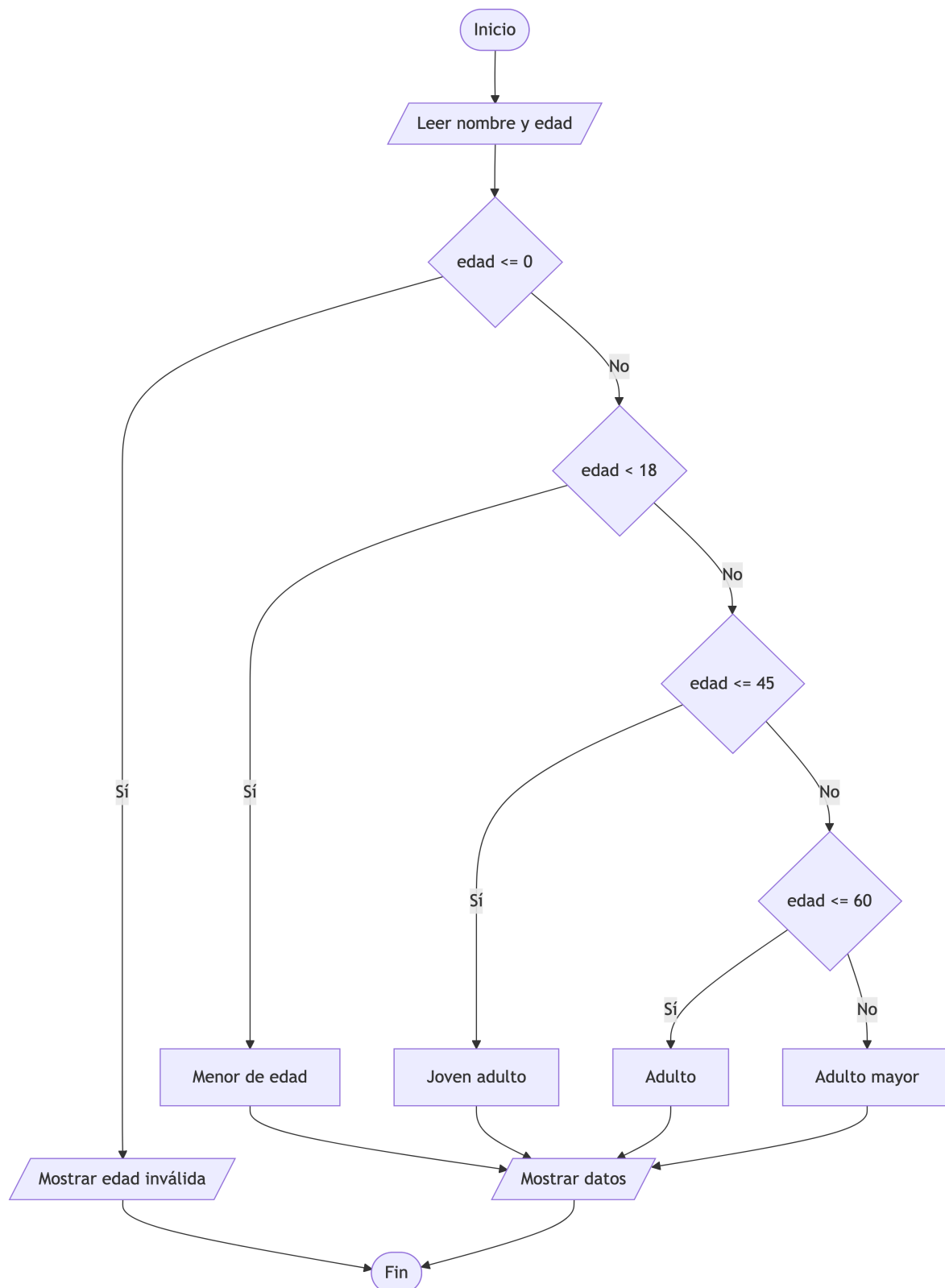
47 Ejercicio guiado 2: clasificación de edad

47.1 Requerimiento

Crear un programa que:

1. Solicite el nombre.
2. Solicite la edad.
3. No permita una edad igual o menor que cero.
4. Clasifique:
 - menor de edad: menos de 18;
 - joven adulto: de 18 a 45;
 - adulto: de 46 a 60;
 - adulto mayor: más de 60.
5. Muestre nombre, edad y clasificación.

47.2 Diagrama de flujo



47.3 Código C++

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombre;
    string clasificacion;
    int edad;

    cout << "Ingrese su nombre: ";
    getline(cin, nombre);

    cout << "Ingrese su edad: ";
    cin >> edad;

    if (edad <= 0) {
        cout << "Edad invalida." << endl;
    } else if (edad < 18) {
        clasificacion = "Menor de edad";
    } else if (edad <= 45) {
        clasificacion = "Joven adulto";
    } else if (edad <= 60) {
        clasificacion = "Adulto";
    } else {
        clasificacion = "Adulto mayor";
    }

    if (edad > 0) {
        cout << "\nNombre: " << nombre << endl;
        cout << "Edad: " << edad << endl;
        cout << "Clasificacion: " << clasificacion << endl;
    }

    return 0;
}
```


48 Actividad rápida en parejas

Modifique la calculadora para que también muestre el nombre de la operación seleccionada.

Cada pareja debe:

1. modificar la versión con `if`;
2. modificar la versión con `switch`;
3. probar al menos tres casos;
4. explicar cuál versión considera más clara.

49 Errores frecuentes

49.1 Usar = en lugar de ==

Incorrecto:

```
if (opcion = 1)
```

Correcto:

```
if (opcion == 1)
```

49.2 Colocar condición después de else

Incorrecto:

```
else (edad > 60)
```

Correcto:

```
else
```

49.3 Olvidar break

```
case 1:  
    cout << "Opcion 1";  
    break;
```

49.4 Usar switch para rangos

switch trabaja mejor con valores exactos.

50 Resumen de la clase

En esta clase aprendimos que:

- Las expresiones lógicas producen valores verdaderos o falsos.
- La precedencia determina el orden de evaluación.
- `if` permite evaluar rangos y condiciones complejas.
- `switch` facilita la construcción de menús y opciones exactas.
- El mismo problema puede resolverse con estructuras diferentes.
- Una buena solución comienza con análisis y diseño.
- Los casos de prueba permiten verificar el comportamiento del programa.

51 Tarea

51.1 Problema 1. Menú de conversión

Crear un programa que solicite una cantidad y muestre este menú:

```
1. Convertir metros a centímetros
2. Convertir centímetros a metros
3. Convertir kilómetros a metros
4. Salir
```

Debe resolverse utilizando `switch`.

51.1.1 Entregables

1. Análisis:
 - entradas;
 - procesos;
 - salidas.
2. Diagrama de flujo.
3. Pseudocódigo.
4. Código C++.
5. Tres casos de prueba.

51.2 Problema 2. Clasificación de notas

Solicitar una nota entre 0 y 100.

Clasificar:

Rango	Resultado
90 a 100	Excelente
80 a 89	Muy bueno
70 a 79	Bueno

Rango	Resultado
61 a 69	Aprobado
0 a 60	Reprobado

Debe resolverse utilizando `if...else if...else`.

51.2.1 Validación

Si la nota es menor que 0 o mayor que 100, mostrar:

```
Nota inválida
```

51.2.2 Entregables

1. Análisis:

- entradas;
- procesos;
- salidas.

2. Diagrama de flujo.

3. Pseudocódigo.

4. Código C++.

5. Cinco casos de prueba.

 Condición de entrega

No se calificará únicamente el código. La tarea debe incluir análisis, diagrama, pseudocódigo, programa y pruebas.

52 Cierre

Consejo final del Ingeniero

Primero comprenda la decisión que debe tomar el algoritmo. Después elija `if` o `switch`. La sintaxis viene al final.

53 Laboratorio No. 3: ciclos, enum y simulación de un cajero automático

53.1 Propósito del laboratorio

En este laboratorio repasaremos brevemente las estructuras que ya conocemos y construiremos un programa completo que combine decisiones, repetición y opciones de menú.

Al finalizar, el estudiante podrá:

- reconocer cuándo utilizar `if`, `switch`, `while`, `for` y `do...while`;
- construir ciclos `for` para repeticiones conocidas;
- representar opciones mediante un tipo enumerado `enum`;
- integrar un menú repetitivo con `switch` y `do...while`;
- validar depósitos y retiros en una simulación de cajero automático.

i Duración total

80 minutos. El propósito no es estudiar muchas estructuras nuevas, sino aprender a combinarlas en un programa claro y funcional.

53.2 Ruta de trabajo

Momento	Actividad	Tiempo
1	Recordatorio de estructuras básicas	10 min
2	Ciclo <code>for</code> : explicación y ejercicios	15 min
3	Uso de <code>enum</code>	10 min
4	Construcción del cajero automático	40 min
5	Prueba y cierre	5 min

53.3 Recordatorio rápido

Un programa puede imaginarse como una persona que sigue instrucciones:

- una **secuencia** indica qué hacer paso a paso;
- una **decisión** permite escoger entre varios caminos;
- un **ciclo** permite repetir una actividad;
- un **menú** reúne varias operaciones y espera la selección del usuario.

53.3.1 Estructura secuencial

Las instrucciones se ejecutan en el mismo orden en que aparecen.

```
#include <iostream>
using namespace std;

int main() {
    int numero1, numero2, suma;

    cout << "Ingrese el primer numero: ";
    cin >> numero1;

    cout << "Ingrese el segundo numero: ";
    cin >> numero2;

    suma = numero1 + numero2;

    cout << "La suma es: " << suma << endl;

    return 0;
}
```

Analogía: preparar café implica seguir una secuencia: calentar agua, agregar café, servir y endulzar.

53.3.2 Decisión con `if`

`if` se utiliza cuando una acción depende de una condición.

```
if (edad >= 18) {
    cout << "Es mayor de edad." << endl;
} else {
    cout << "Es menor de edad." << endl;
}
```

Analogía: antes de salir, una persona observa el clima. Si está lloviendo, lleva paraguas; de lo contrario, sale sin él.

53.3.3 Selección múltiple con `switch`

`switch` es útil cuando una variable puede representar varias opciones concretas.

```
switch (opcion) {  
    case 1:  
        cout << "Consultar saldo" << endl;  
        break;  
  
    case 2:  
        cout << "Depositar" << endl;  
        break;  
  
    case 3:  
        cout << "Retirar" << endl;  
        break;  
  
    default:  
        cout << "Opcion no valida" << endl;  
}
```

¿Para qué sirve `break`?

`break` detiene el `switch` después de ejecutar el caso seleccionado. Sin él, el programa puede continuar ejecutando los casos siguientes.

53.3.4 Repetición con `while`

`while` repite un bloque mientras una condición sea verdadera.

```
int contador = 1;  
  
while (contador <= 5) {  
    cout << contador << endl;  
    contador++;  
}
```

Analogía: mientras todavía haya platos sucios, se continúa lavando.

53.4 Ciclo for

El ciclo `for` se utiliza principalmente cuando conocemos cuántas veces debe repetirse una acción.

```
for (inicializacion; condicion; actualizacion) {  
    // instrucciones que se repiten  
}
```

En el siguiente ejemplo:

```
for (int i = 1; i <= 5; i++) {  
    cout << i << endl;  
}
```

ocurre lo siguiente:

1. `int i = 1` crea e inicializa el contador.
2. `i <= 5` determina si el ciclo continúa.
3. `i++` aumenta el contador después de cada repetición.

Analogía: recorrer cinco escritorios para entregar una hoja. Se conoce desde el inicio cuántos escritorios deben visitarse.

53.4.1 Ejercicio 1: mostrar los números del 1 al 10

Complete y ejecute el siguiente programa:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    for (int i = 1; i <= 10; i++) {  
        cout << i << " ";  
    }  
  
    cout << endl;  
    return 0;  
}
```

53.4.2 Ejercicio 2: tabla de multiplicar

Construya un programa que solicite un número y muestre su tabla de multiplicar del 1 al 10.


```
#include <iostream>
using namespace std;

int main() {
    int numero;

    cout << "Ingrese un numero: ";
    cin >> numero;

    for (int i = 1; i <= 10; i++) {
        cout << numero << " x " << i
            << " = " << numero * i << endl;
    }

    return 0;
}
```

💡 Pregunta para el grupo

¿Qué parte del ciclo habría que modificar para mostrar la tabla únicamente del 1 al 5?

53.5 Uso de enum

Un enum permite asignar nombres claros a un conjunto de valores relacionados.

```
enum Dia {
    LUNES = 1,
    MARTES,
    MIERCOLES,
    JUEVES,
    VIERNES
};
```

Internamente, los valores continúan siendo números, pero el código resulta más fácil de leer.

Analogía: en lugar de identificar las luces de un semáforo como 0, 1 y 2, podemos llamarlas ROJO, AMARILLO y VERDE.

```
#include <iostream>
using namespace std;

enum EstadoSemaforo {
    ROJO = 1,
```

```
    AMARILLO,  
    VERDE  
};  
  
int main() {  
    int opcion;  
  
    cout << "1. Rojo" << endl;  
    cout << "2. Amarillo" << endl;  
    cout << "3. Verde" << endl;  
    cout << "Seleccione el estado: ";  
    cin >> opcion;  
  
    switch (opcion) {  
        case ROJO:  
            cout << "Detenerse." << endl;  
            break;  
  
        case AMARILLO:  
            cout << "Prepararse para detenerse." << endl;  
            break;  
  
        case VERDE:  
            cout << "Puede avanzar." << endl;  
            break;  
  
        default:  
            cout << "Estado no valido." << endl;  
    }  
  
    return 0;  
}
```

53.5.1 Ventaja principal

Compare estas dos expresiones:

```
if (opcion == 3)
```

```
if (opcion == VERDE)
```

La segunda comunica mejor la intención del programa.

53.6 Proyecto del laboratorio: cajero automático

53.6.1 Situación

Un cajero automático presenta un menú y espera que el usuario seleccione una operación. Después de consultar, depositar o retirar, el menú vuelve a mostrarse. El programa termina únicamente cuando se selecciona **Salir**.

53.6.2 Operaciones requeridas

1. Consultar saldo.
2. Depositar dinero.
3. Retirar dinero.
4. Salir.

El saldo inicial será de **Q1,000.00**.

53.6.3 Reglas

- Los depósitos deben ser mayores que cero.
- Los retiros deben ser mayores que cero.
- No se puede retirar una cantidad superior al saldo disponible.
- El menú debe repetirse hasta seleccionar la opción de salida.
- Las opciones se representarán mediante `enum`.
- El menú se controlará con `switch`.
- La repetición se realizará con `do...while`.

53.7 4.1 ¿Por qué utilizar `do...while`?

El cajero debe mostrar el menú por lo menos una vez. Por ello, `do...while` es apropiado: primero ejecuta las instrucciones y luego evalúa si debe repetirlas.

```
do {  
    // instrucciones  
} while (condicion);
```

Analogía: al llegar a un cajero, primero se observa el menú. Después de realizar una operación se decide si se hará otra o si se terminará.

53.8 Diseño previo

Antes de programar, identifique las partes principales:

```
Inicio
  Definir saldo inicial
  Definir las opciones del menu

  Hacer
    Mostrar menu
    Leer opcion

    Segun la opcion
      Consultar saldo
      Depositar
      Retirar
      Salir
      Informar opcion invalida
  Mientras la opcion no sea Salir
Fin
```

53.9 Diagrama de flujo

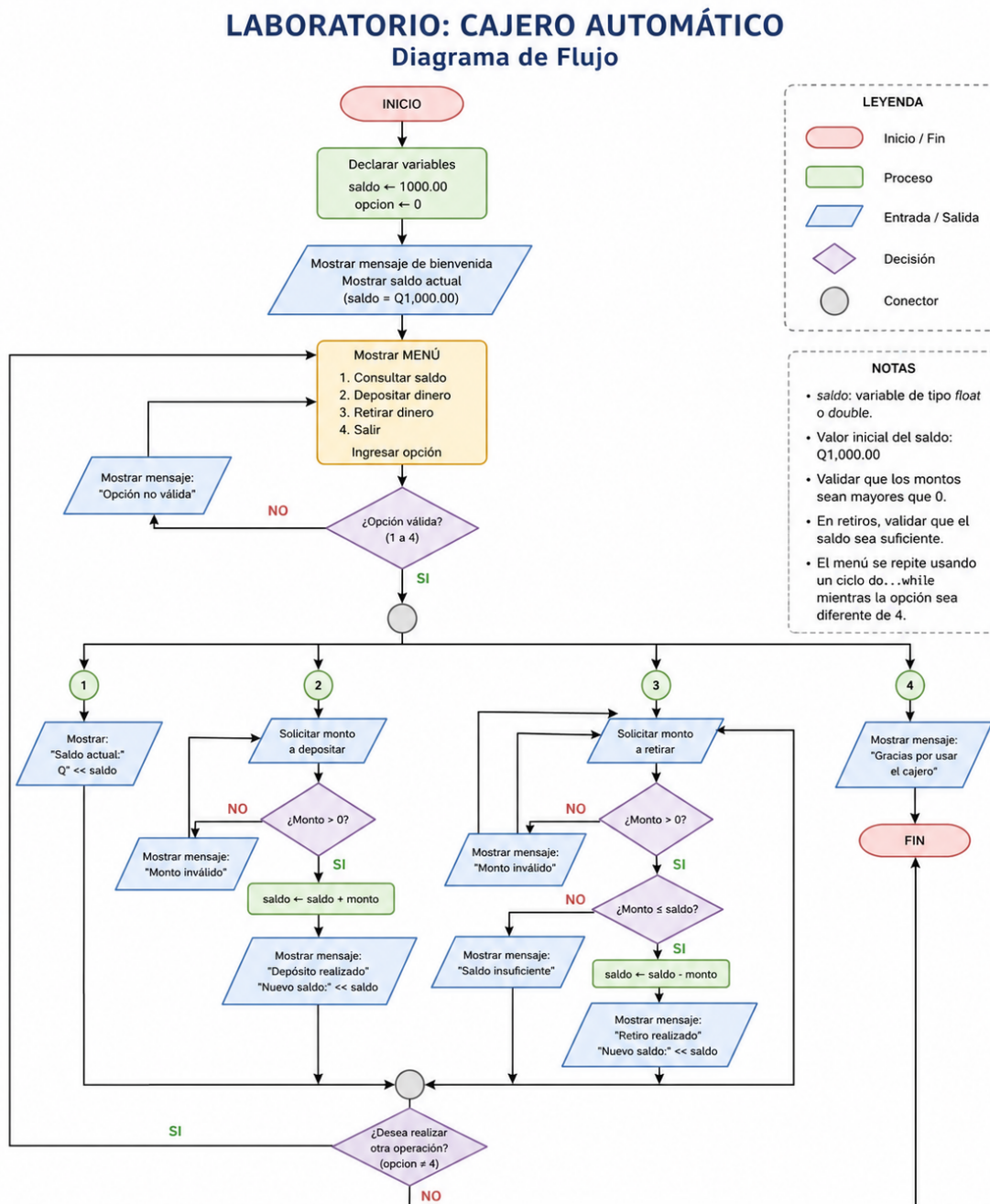


Figura 53.1: Diagrama de flujo del cajero automático

53.10 Definición de las opciones con enum

```
enum OpcionCajero {  
    CONSULTAR_SALDO = 1,  
    DEPOSITAR,  
    RETIRAR,  
    SALIR  
};
```

Como el primer valor es 1, los siguientes toman automáticamente los valores 2, 3 y 4.

53.11 Construcción paso a paso

53.11.1 Paso 1: variables principales

```
double saldo = 1000.00;  
double monto;  
int opcion;
```

53.11.2 Paso 2: mostrar el menú

```
cout << "\n==== CAJERO AUTOMATICO =====" << endl;  
cout << "1. Consultar saldo" << endl;  
cout << "2. Depositar dinero" << endl;  
cout << "3. Retirar dinero" << endl;  
cout << "4. Salir" << endl;  
cout << "Seleccione una opcion: ";  
cin >> opcion;
```

53.11.3 Paso 3: procesar la opción

```
switch (opcion) {  
    case CONSULTAR_SALDO:  
        cout << "Saldo disponible: Q" << saldo << endl;  
        break;  
  
    case DEPOSITAR:  
        // solicitar y validar el deposito
```

```
        break;

    case RETIRAR:
        // solicitar y validar el retiro
        break;

    case SALIR:
        cout << "Gracias por utilizar el cajero." << endl;
        break;

    default:
        cout << "Opcion no valida." << endl;
}

```

53.11.4 Paso 4: repetir el menú

```
do {
    // mostrar menu y procesar opcion
} while (opcion != SALIR);

```

53.12 5. Programa completo

```
#include <iostream>
#include <iomanip>
using namespace std;

enum OpcionCajero {
    CONSULTAR_SALDO = 1,
    DEPOSITAR,
    RETIRAR,
    SALIR
};

int main() {
    double saldo = 1000.00;
    double monto;
    int opcion;

    cout << fixed << setprecision(2);

```

```
do {
    cout << "\n==== CAJERO AUTOMATICO =====> endl;
    cout << "1. Consultar saldo" << endl;
    cout << "2. Depositar dinero" << endl;
    cout << "3. Retirar dinero" << endl;
    cout << "4. Salir" << endl;
    cout << "Seleccione una opcion: ";
    cin >> opcion;

    switch (opcion) {
        case CONSULTAR_SALDO:
            cout << "Saldo disponible: Q" << saldo << endl;
            break;

        case DEPOSITAR:
            cout << "Ingrese el monto a depositar: Q";
            cin >> monto;

            if (monto > 0) {
                saldo = saldo + monto;
                cout << "Deposito realizado correctamente." << endl;
                cout << "Nuevo saldo: Q" << saldo << endl;
            } else {
                cout << "El monto debe ser mayor que cero." << endl;
            }
            break;

        case RETIRAR:
            cout << "Ingrese el monto a retirar: Q";
            cin >> monto;

            if (monto <= 0) {
                cout << "El monto debe ser mayor que cero." << endl;
            } else if (monto > saldo) {
                cout << "Fondos insuficientes." << endl;
            } else {
                saldo = saldo - monto;
                cout << "Retiro realizado correctamente." << endl;
                cout << "Nuevo saldo: Q" << saldo << endl;
            }
            break;

        case SALIR:
            cout << "Gracias por utilizar el cajero automatico." << endl;
            break;

        default:
```



```

        cout << "Opcion no valida. Intente nuevamente." << endl;
    }

    } while (opcion != SALIR);

    return 0;
}

```

53.13 Pruebas del programa

Antes de considerar terminado el ejercicio, pruebe por lo menos los siguientes casos:

Prueba	Acción	Resultado esperado
1	Consultar al iniciar	Mostrar Q1,000.00
2	Depositar Q500.00	Nuevo saldo Q1,500.00
3	Retirar Q300.00	Nuevo saldo Q1,200.00
4	Retirar más que el saldo	Mostrar fondos insuficientes
5	Depositar un número negativo	Rechazar el monto
6	Seleccionar una opción inexistente	Mostrar opción no válida
7	Seleccionar Salir	Finalizar el ciclo

! Idea clave

Un programa no está terminado solamente porque compila. También debe probarse con datos correctos, incorrectos y casos límite.

53.14 Reto opcional

Cuando el programa básico funcione, agregue **una** de las siguientes mejoras:

- solicitar un PIN antes de mostrar el menú;
- contar cuántas operaciones se realizaron;
- establecer un retiro máximo por operación;
- mostrar un mensaje cuando el saldo sea menor que Q100.00.

53.15 Cierre

En el cajero automático se combinaron varias estructuras:

- `enum` dio nombres a las opciones;
- `switch` seleccionó la operación;
- `if` validó depósitos y retiros;

- `do...while` mantuvo activo el menú;
- las variables almacenaron el saldo y los montos.

El valor del ejercicio no está únicamente en hacer funcionar un cajero, sino en comprender cómo varias estructuras pequeñas pueden integrarse para resolver un problema completo.

54 Capítulo 5. Integración de conceptos: construyendo un programa completo

Caso de estudio: Sistema de Control de Parqueos

55 Introducción

Este capítulo integra los conceptos estudiados hasta el momento para resolver un problema completo en C++.

56 Objetivos

- Integrar variables, operadores y estructuras de control.
- Analizar un problema antes de programar.
- Diseñar una solución mediante pseudocódigo y Mermaid.

57 Caso de estudio

57.1 Sistema de Control de Parqueos

58 Análisis del problema

Entradas	Procesos	Salidas
Nombre	Determinar tarifa	Total
Placa	Calcular	Resumen
Tipo	Validar	Mensajes
Horas	Operaciones	Información

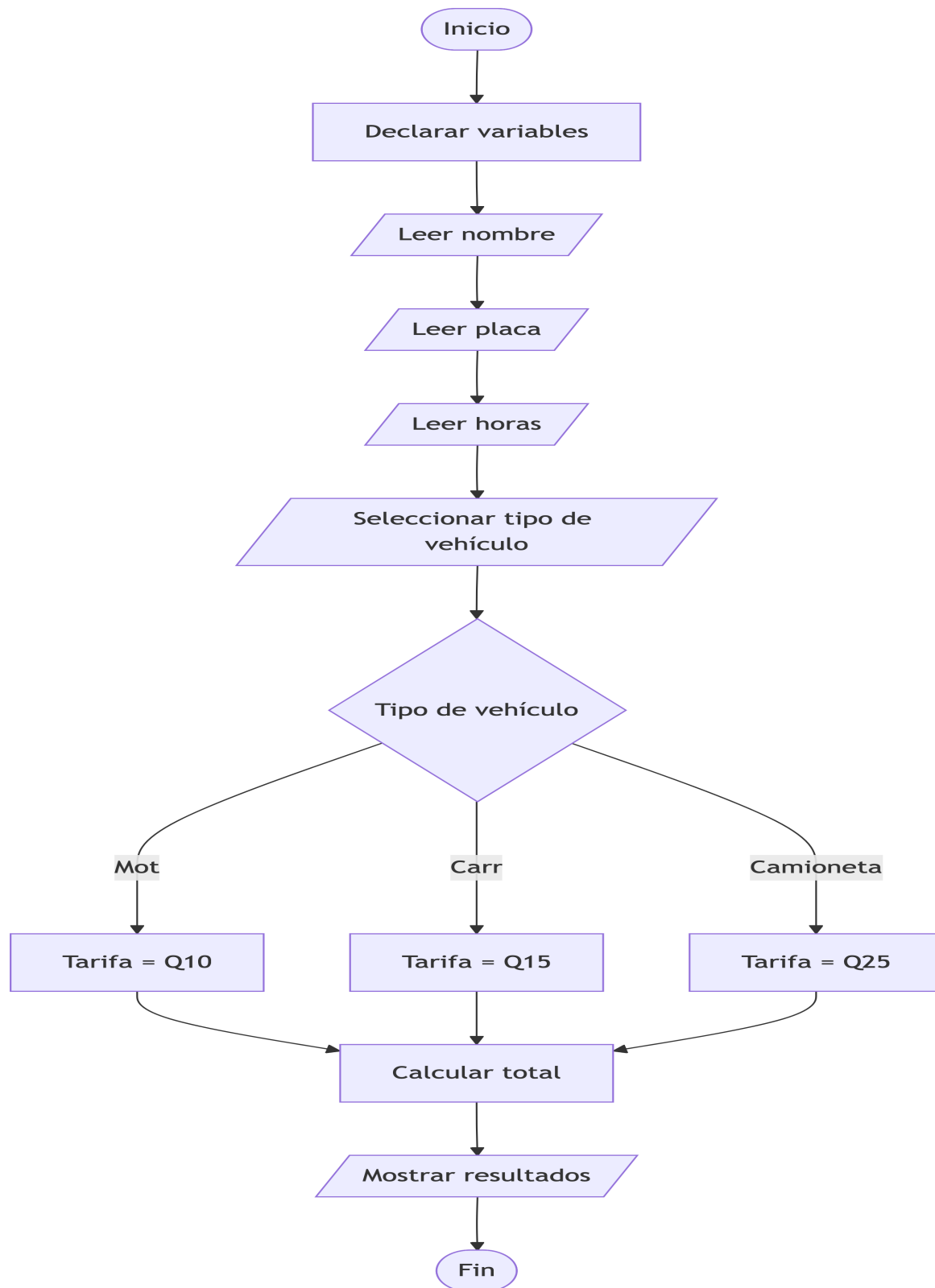
59 Variables

Variable	Tipo
nombreConductor	string
placa	string
tipoVehiculo	int
horas	int
tarifa	double
totalPagar	double

60 Pseudocódigo

```
Inicio  
  Declarar variables  
  Leer datos  
  Determinar tarifa  
  Calcular total  
  Mostrar resultados  
Fin
```


61 Diagrama: Sistema de control de parqueos



62 Paso 1

```
#include <iostream>
#include <string>
using namespace std;
```


63 Paso 2

```
//-----  
// Enumeración de los tipos de vehículos  
//-----  
  
enum TipoVehiculo  
{  
    MOTO = 1,  
    CARRO,  
    CAMIONETA  
};  
  
//-----  
// Declaración de variables  
//-----  
  
// En esta sección declaramos todas las variables que  
// utilizaremos durante el programa, aunque algunas  
// todavía no se empleen en los pasos siguientes.  
// Esto mejora la organización y facilita la lectura  
// del código.  
  
string nombreConductor;  
string placa;  
  
int tipoVehiculo;  
int horas;  
  
double tarifa = 0.0;  
double totalPagar = 0.0;
```


64 Paso 3. Entrada

```
cout<<"Nombre: ";
getline(cin,nombreConductor);
cout<<"Placa: ";
getline(cin,placa);

cout<<"Horas: ";
cin>>horas;

cout << "Tipo de vehículo" << endl;
cout << "1. Moto" << endl;
cout << "2. Carro" << endl;
cout << "3. Camioneta" << endl;
cout << "\nSeleccione una opción: ";
cin >> tipoVehiculo;
```


65 Paso 4. Tarifa

```
switch(tipoVehiculo){  
case MOTO: tarifa=10; break;  
case CARRO: tarifa=15; break;  
case CAMIONETA: tarifa=25; break;  
default: tarifa=0;  
}
```


66 Paso 5

```
totalPagar=tarifa*horas;
```


67 Paso 6

```
cout<<"Total: Q"<<totalPagar<<endl;
```


68 Reflexión

- ¿Por qué tarifa es double?
- ¿Qué hace switch?

69 Resumen

Tema	Aplicado
Variables	☐
Operadores	☐
switch	☐
if	☐
while	☐
do...while	☐
for	☐

70 Paso 7. Mostrando los resultados

Una vez calculado el total a pagar, es momento de presentar la información al usuario de forma clara y ordenada.

```
//-----  
//  Mostrar resultados  
//-----  
  
cout << "\n===== \n";  
cout << "        SISTEMA DE CONTROL DE PARQUEOS\n";  
cout << "===== \n\n";  
  
cout << "Conductor : " << nombreConductor << endl;  
cout << "Placa      : " << placa << endl;  
  
cout << "Tipo de vehículo" << endl;  
cout << "1. Moto" << endl;  
cout << "2. Carro" << endl;  
cout << "3. Camioneta" << endl;  
cout << "\nSeleccione una opción: ";  
cin >> tipoVehiculo;  
  
switch(tipoVehiculo)  
{  
    case MOTO: cout << "Moto"; break;  
    case CARRO: cout << "Carro"; break;  
    case CAMIONETA: cout << "Camioneta"; break;  
}  
  
cout << endl;  
cout << "Horas      : " << horas << endl;  
cout << "Tarifa     : Q" << tarifa << endl;  
cout << "-----" << endl;  
cout << "TOTAL A PAGAR : Q" << totalPagar << endl;
```

70.1 Ejemplo de ejecución

Entrada

```
Nombre: Jorge García
Placa: P123ABC
Tipo de vehículo: 2
Horas: 3
```

Salida

```
=====
          SISTEMA DE CONTROL DE PARQUEOS
=====

Conductor  : Jorge García
Placa      : P123ABC
Vehículo   : Carro
Horas      : 3
Tarifa     : Q15

-----
TOTAL A PAGAR : Q45
```

70.2 Reflexionemos

- ¿Por qué es importante presentar la información de forma ordenada?
- ¿Qué ventajas tiene mostrar el nombre del vehículo en lugar del número seleccionado?
- ¿Qué parte del programa realiza el cálculo y cuál solamente muestra información?

71 Paso 8. Analizando el programa

Concepto	¿Dónde aparece?
Variables	Declaración de variables
Operadores	<code>tarifa * horas</code>
<code>switch</code>	Selección de la tarifa
<code>cout</code>	Presentación de resultados

Hasta este punto ya tenemos un programa completamente funcional.

72 Paso 8. ¿Cómo podemos mejorarlo?

Nuestro programa funciona correctamente, pero como ocurre en el desarrollo de software real, siempre podemos incorporar nuevas características.

Cada mejora será la oportunidad perfecta para recordar una estructura de control ya estudiada.

Necesidad del programa	Concepto que utilizaremos
Validar las horas ingresadas	<code>if</code>
Clasificar el tiempo de permanencia	<code>if...else</code>
Aplicar descuentos	<code>if...else if</code>
Registrar varios vehículos	<code>while</code>
Mostrar un menú principal	<code>do...while</code>
Mostrar los espacios disponibles	<code>for</code>

72.1 Lo que construiremos a continuación

En la siguiente versión del programa incorporaremos:

1. Validación de datos.
2. Descuentos según el tiempo de permanencia.
3. Registro de varios vehículos.
4. Menú principal.
5. Visualización de espacios del parqueo.

Recuerda

Un buen programador no memoriza estructuras de control.
Las utiliza para resolver problemas reales.

72.2 Preparándonos para la siguiente clase

En la próxima sesión reorganizaremos este mismo programa utilizando:

- Funciones
- Procedimientos
- Parámetros

El objetivo será comprobar cómo un programa que ya funciona puede organizarse de una manera más profesional.

73 Capítulo 6. Organizando nuestros programas mediante funciones y procedimientos

73.1 Caso de estudio

73.1.1 Evolucionando el Sistema de Control de Parqueos

74 Introducción

En el capítulo anterior desarrollamos un programa completo para calcular el cobro de un parqueo. El programa resolvía correctamente el problema planteado y nos permitió integrar variables, operadores y estructuras de control.

Ahora el programa funciona, pero ha crecido. Cuando un programa aumenta de tamaño también aumenta la dificultad para leerlo, comprenderlo y modificarlo.

En el desarrollo profesional del software es poco común escribir todo dentro de `main()`. En su lugar, el programa se divide en pequeños módulos donde cada uno realiza una única responsabilidad.

75 Objetivos

- Comprender por qué organizamos un programa.
- Diferenciar funciones y procedimientos.
- Comprender el uso de parámetros.
- Reorganizar el Sistema de Control de Parqueos.

76 ¿Por qué necesitamos organizar nuestros programas?

Antes de escribir una sola línea de código, observa nuevamente el programa de la clase anterior.

 Pregunta para discutir

¿Les parece fácil leer un programa donde todo está escrito dentro del `main()` ?

Permite que los estudiantes expresen sus ideas y luego introduce el concepto de modularidad.

77 La analogía de la construcción de una casa

Una casa no es construida por una sola persona.

Profesional	Responsabilidad
Arquitecto	Diseñar la casa
Albañil	Construir la estructura
Electricista	Instalar el sistema eléctrico
Plomero	Instalar las tuberías
Pintor	Realizar el acabado final

Cada uno tiene una única responsabilidad.

Un programa bien diseñado funciona exactamente igual: cada módulo realiza una tarea específica.

78 Procedimientos y funciones

79 ¿Qué es un procedimiento?

Un procedimiento es un bloque de código que realiza una tarea específica y **no devuelve un valor**.

Las siguientes tareas de nuestro sistema pueden convertirse en procedimientos:

- Mostrar el título.
- Leer información.
- Mostrar resultados.

Recuerda

Un procedimiento ejecuta acciones.

80 ¿Qué es una función?

Algunas tareas necesitan calcular un resultado.

Por ejemplo:

- Determinar la tarifa.
- Calcular el total a pagar.

En estos casos utilizaremos una función.

Una función realiza una tarea específica y devuelve un valor al programa que la llamó.

81 Procedimiento o función

Característica	Procedimiento	Función
Realiza una tarea	☐	☐
Devuelve un valor	☐	☐
Utiliza <code>return</code>	No	Sí

Reflexiona

¿Cuáles tareas del Sistema de Control de Parques implementarías mediante procedimientos y cuáles mediante funciones?

82 ¿Qué son los parámetros?

Una función sabe **qué debe hacer**, pero necesita información para poder trabajar.
A esa información la llamamos **parámetros**.

83 La analogía de la casa

Cada trabajador necesita materiales para realizar su trabajo.

Profesional	¿Qué necesita?
Electricista	Cables e interruptores
Albañil	Ladrillos y cemento
Pintor	Pintura

De la misma manera, una función necesita recibir datos para realizar su tarea.

Un parámetro es un dato que se envía a una función o procedimiento para que pueda realizar su trabajo.

i Idea importante

Las funciones siempre realizan la misma tarea.
Lo único que cambia son los datos que reciben.

La siguiente sección transformará paso a paso nuestro Sistema de Control de Parques utilizando procedimientos, funciones y parámetros.

84 Transformando nuestro programa

Hasta este momento ya conocemos los conceptos de **procedimientos, funciones y parámetros**.

Ahora llegó el momento de aplicarlos sobre el programa desarrollado en la clase anterior.

Nuestro objetivo **no es cambiar el funcionamiento del Sistema de Control de Parqueos**.

El objetivo es **organizar mejor el código**.

Al finalizar esta sección, el programa realizará exactamente las mismas tareas, pero será mucho más fácil de leer, comprender y mantener.

85 Primera mejora: mover el título del programa

85.1 Objetivo

Crear un procedimiento que sea el único responsable de mostrar el encabezado del sistema.

Piensa

Si mañana deseamos cambiar el nombre del sistema o mejorar el diseño del encabezado, ¿en cuántos lugares tendríamos que modificar el programa?

Después de crear el procedimiento, elimina ese código del `main()` y reemplázalo por una única llamada al procedimiento.

Analiza el resultado.

El programa continúa funcionando exactamente igual.

Sin embargo, el `main()` ya es más corto y fácil de leer.

86 Segunda mejora: separar la lectura de datos

86.1 Objetivo

Crear un procedimiento encargado de solicitar toda la información al usuario.

Este procedimiento será responsable de:

- Leer el nombre del cliente.
- Leer el tipo de vehículo.
- Leer la cantidad de horas.

Una vez terminado, el `main()` únicamente deberá llamar al procedimiento.

Observa cómo comienza a desaparecer el código repetitivo del programa principal.

87 Tercera mejora: crear una función para determinar la tarifa

Hasta este momento el programa determina la tarifa utilizando una estructura de decisión. Ahora esa responsabilidad será trasladada a una función.

87.1 Objetivo

Crear una función que reciba el tipo de vehículo y devuelva la tarifa correspondiente.

El programa principal ya no necesitará conocer cómo se calcula la tarifa.

Únicamente solicitará el resultado.

i Importante

Una buena función oculta los detalles del cálculo.

El programa principal solamente utiliza el resultado obtenido.

88 Cuarta mejora: crear una función para calcular el total

Una vez conocida la tarifa, podemos crear otra función cuya única responsabilidad sea calcular el total a pagar.

Esta función recibirá:

- La tarifa.
- La cantidad de horas.

Y devolverá el monto total.

Observa que ahora cada función tiene una única responsabilidad.

89 Quinta mejora: crear un procedimiento para mostrar el reporte

Finalmente trasladaremos toda la salida del programa a un procedimiento.

Este procedimiento mostrará:

- Nombre del cliente.
- Tipo de vehículo.
- Horas.
- Tarifa.
- Total a pagar.

El programa principal dejará de preocuparse por la presentación de la información.

Su única responsabilidad será coordinar la ejecución del programa.

90 Nuestro nuevo programa

Al finalizar las mejoras, el `main()` será mucho más pequeño y mucho más fácil de comprender. Su lectura se parece más a una lista de tareas que a un bloque de instrucciones.



💡 Reflexiona

Observa el diagrama anterior.
Cada bloque realiza una única responsabilidad.

Esta es una de las principales características de un programa bien organizado.

91 ¿Qué ganamos con esta organización?

Al reorganizar nuestro programa obtenemos varias ventajas:

- El código es más fácil de leer.
- Cada módulo tiene una responsabilidad específica.
- Las modificaciones futuras serán más sencillas.
- Es posible reutilizar funciones en otros programas.
- El trabajo en equipo resulta más simple, ya que diferentes programadores pueden trabajar sobre módulos distintos.

En el siguiente apartado compararemos la primera versión del programa con esta nueva versión organizada para identificar las mejoras obtenidas.

92 Primera modificación: crear el procedimiento `mostrarTitulo()`

Hasta este momento, el encabezado del sistema se encuentra escrito directamente dentro de `main()`.

El primer cambio será mover únicamente esas instrucciones a un procedimiento independiente.

93 1. Identificamos el código que vamos a mover

Localiza dentro de `main()` las siguientes instrucciones:

```
cout << "\n===== \n";  
cout << "    SISTEMA DE CONTROL DE PARQUEOS \n";  
cout << "===== \n \n";
```

Estas líneas cumplen una sola responsabilidad: mostrar el título del sistema.

94 2. Creamos el procedimiento

Antes de `main()`, escribe:

```
//-----  
// Muestra el encabezado del sistema  
//-----  
void mostrarTitulo()  
{  
    cout << "\n===== \n";  
    cout << "    SISTEMA DE CONTROL DE PARQUEOS \n";  
    cout << "===== \n \n";  
}
```

La palabra `void` indica que el procedimiento realiza una acción, pero no devuelve ningún valor.

95 3. Modificamos `main()`

Elimina del `main()` las tres instrucciones originales y reemplázalas por:

```
mostrarTitulo();
```


96 4. Código después de la primera modificación

```
#include <iostream>
#include <string>

using namespace std;

//-----
// Enumeración de los tipos de vehículos
//-----
enum TipoVehiculo
{
    MOTO = 1,
    CARRO,
    CAMIONETA
};

//-----
// Declaración de variables
//-----
string nombreConductor;
string placa;

int tipoVehiculo;
int horas;

double tarifa = 0.0;
double totalPagar = 0.0;

//-----
// Muestra el encabezado del sistema
//-----
void mostrarTitulo()
{
    cout << "\n===== \n";
    cout << "    SISTEMA DE CONTROL DE PARQUEOS \n";
}
```

```
    cout << "=====\n\n";
}

int main()
{
    cout << "Nombre: ";
    getline(cin, nombreConductor);

    cout << "Placa: ";
    getline(cin, placa);

    cout << "Horas: ";
    cin >> horas;

    cout << "\nTipo de vehículo" << endl;
    cout << "1. Moto" << endl;
    cout << "2. Carro" << endl;
    cout << "3. Camioneta" << endl;
    cout << "\nSeleccione una opción: ";
    cin >> tipoVehiculo;

    switch (tipoVehiculo)
    {
        case MOTO:
            tarifa = 10;
            break;

        case CARRO:
            tarifa = 15;
            break;

        case CAMIONETA:
            tarifa = 25;
            break;

        default:
            tarifa = 0;
    }

    totalPagar = tarifa * horas;

    mostrarTitulo();

    cout << "Conductor : " << nombreConductor << endl;
    cout << "Placa      : " << placa << endl;

    cout << "Vehículo   : ";
```



```
switch (tipoVehiculo)
{
    case MOTO:
        cout << "Moto";
        break;

    case CARRO:
        cout << "Carro";
        break;

    case CAMIONETA:
        cout << "Camioneta";
        break;

    default:
        cout << "No válido";
}

cout << endl;
cout << "Horas      : " << horas << endl;
cout << "Tarifa      : Q" << tarifa << endl;
cout << "-----" << endl;
cout << "TOTAL A PAGAR: Q" << totalPagar << endl;

return 0;
}
```

Comprobación

Compila y ejecuta el programa.
La salida debe ser exactamente la misma que antes.

¿Qué cambió?

El encabezado ya no está escrito directamente dentro de `main()`.
Ahora esa tarea pertenece al procedimiento `mostrarTitulo()`.

97 Segunda modificación: crear el procedimiento `LeerDatos()`

En la modificación anterior organizamos el encabezado del sistema.

Ahora haremos lo mismo con la entrada de datos.

Nuestro objetivo será que **todo el proceso de captura de información** quede dentro de un solo procedimiento.

98 1. Identificamos el código que vamos a mover

Dentro de `main()` localiza todas las instrucciones utilizadas para leer:

- Nombre del conductor.
- Placa.
- Horas.
- Tipo de vehículo.

Todo ese bloque pertenece a una misma responsabilidad: **leer los datos del usuario.**

99 2. Creamos el procedimiento

Antes de `main()` agrega el siguiente procedimiento:

```
//-----  
// Solicita la información del conductor  
//-----  
void leerDatos()  
{  
    cout << "Nombre: ";  
    getline(cin, nombreConductor);  
  
    cout << "Placa: ";  
    getline(cin, placa);  
  
    cout << "Horas: ";  
    cin >> horas;  
  
    cout << "\nTipo de vehículo" << endl;  
    cout << "1. Moto" << endl;  
    cout << "2. Carro" << endl;  
    cout << "3. Camioneta" << endl;  
    cout << "\nSeleccione una opción: ";  
    cin >> tipoVehiculo;  
}
```

Observa que el procedimiento únicamente realiza una tarea: solicitar información al usuario.

100 3. Modificamos `main()`

Ahora elimina todas las instrucciones de entrada de datos y reemplázalas por una sola línea:

```
leerDatos();
```

El inicio del programa principal ahora queda mucho más limpio.

```
int main()
{
    leerDatos();

    // El resto del programa continúa aquí...
}
```


101 4. ¿Qué ganamos?

Comparemos ambas versiones.

101.1 Antes

```
main()
| Leer nombre
| Leer placa
| Leer horas
| Mostrar menú
| Leer tipo de vehículo
| ...
```

101.2 Después

```
main()
| leerDatos()
| ...
```

El programa sigue haciendo exactamente lo mismo, pero ahora el nombre del procedimiento describe claramente la tarea que realiza.

Comprobación

Compila nuevamente el programa.
Si todo está correcto, el comportamiento será exactamente el mismo que antes.

¿Qué aprendimos?

Un procedimiento puede agrupar varias instrucciones relacionadas en una sola unidad lógica. Esto hace que el programa sea más fácil de leer y mantener.

101.3 Preparándonos para la siguiente modificación

En la siguiente sección moveremos el `switch` encargado de determinar la tarifa hacia una **función**.

Será la primera vez que nuestro programa devolverá un valor utilizando `return`.

102 Tercera modificación: crear la función `determinarTarifa()`

Hasta este momento nuestro programa utiliza un bloque `switch` dentro de `main()` para determinar la tarifa según el tipo de vehículo.

En esta modificación moveremos esa responsabilidad hacia una función.

103 1. Identificamos el código que vamos a mover

Localiza el siguiente bloque dentro de `main()`:

```
switch (tipoVehiculo)
{
    case MOTO:
        tarifa = 10;
        break;

    case CARRO:
        tarifa = 15;
        break;

    case CAMIONETA:
        tarifa = 25;
        break;

    default:
        tarifa = 0;
}
```

Observa que este bloque realiza una única tarea: **determinar la tarifa**.

104 2. Creamos la función

Antes de `main()` agrega la siguiente función:

```
//-----  
// Determina la tarifa según el tipo de vehículo  
//-----  
double determinarTarifa(int tipo)  
{  
    switch (tipo)  
    {  
        case MOTO:  
            return 10;  
  
        case CARRO:  
            return 15;  
  
        case CAMIONETA:  
            return 25;  
  
        default:  
            return 0;  
    }  
}
```

104.1 Analicemos el encabezado

```
double determinarTarifa(int tipo)
```

Elemento	Significado
<code>double</code>	La función devolverá un valor decimal.
<code>determinarTarifa</code>	Nombre de la función.
<code>int tipo</code>	Parámetro que recibe el tipo de vehículo.

A diferencia de un procedimiento, esta función **devuelve un resultado** mediante la instrucción `return`.

105 3. Modificamos `main()`

Elimina el bloque `switch` y reemplázalo por una única línea:

```
tarifa = determinarTarifa(tipoVehiculo);
```

Lee esa instrucción en voz alta:

“La variable **tarifa** recibirá el valor que devuelva la función **determinarTarifa()**.”

106 4. ¿Qué sucede internamente?

```
main()
  ↓ envía tipoVehiculo
determinarTarifa()
  ↓ calcula la tarifa
return tarifa
  ↓
main() guarda el resultado en la variable tarifa
```



107 5. ¿Qué ganamos?

107.1 Antes

```
main()  
├ switch(...)  
├ case MOTO  
├ case CARRO  
└ case CAMIONETA
```

107.2 Después

```
main()  
└ tarifa = determinarTarifa(tipoVehiculo);
```

Ahora el programa principal ya no necesita conocer cómo se calcula la tarifa.

Esa responsabilidad quedó encapsulada dentro de la función.

Comprobación

Compila y ejecuta nuevamente el programa.
El resultado debe ser exactamente el mismo que en las modificaciones anteriores.

¿Qué aprendimos?

Una función permite ocultar los detalles de un cálculo.
El programa principal únicamente solicita el resultado y continúa con la siguiente tarea.

107.3 Preparándonos para la siguiente modificación

En la siguiente modificación construiremos la función `calcularTotal()`.

Verás que el `main()` comienza a parecer una lista de tareas de alto nivel.

108 Cuarta modificación: crear la función `calcularTotal()`

En la modificación anterior logramos que el programa determinara la tarifa utilizando una función. Ahora haremos exactamente lo mismo con el cálculo del total a pagar.

109 1. Identificamos el código que vamos a mover

Dentro de `main()` localiza la siguiente instrucción:

```
totalPagar = tarifa * horas;
```

Aunque únicamente es una línea de código, representa una responsabilidad muy importante del programa: **calcular el monto que deberá pagar el cliente.**

110 2. Creamos la función

Antes de `main()` agrega la siguiente función:

```
//-----  
// Calcula el total a pagar  
//-----  
double calcularTotal(double tarifaHora, int cantidadHoras)  
{  
    return tarifaHora * cantidadHoras;  
}
```

Observa que esta función recibe dos parámetros:

Parámetro	Descripción
<code>tarifaHora</code>	Tarifa correspondiente al vehículo
<code>cantidadHoras</code>	Horas que permaneció estacionado

Después realiza el cálculo y devuelve el resultado mediante `return`.

111 3. Modificamos `main()`

Elimina la instrucción original y reemplázala por:

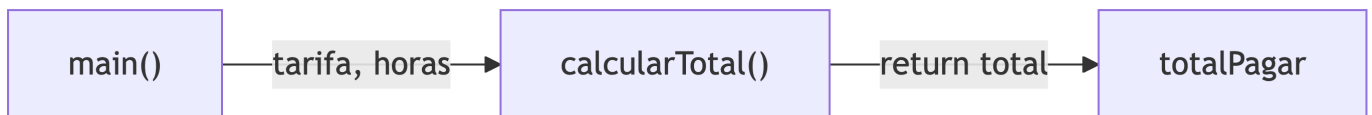
```
totalPagar = calcularTotal(tarifa, horas);
```

Lee la instrucción como si fuera una frase:

“TotalPagar recibirá el valor que devuelva la función `calcularTotal()` utilizando la tarifa y las horas.”

112 4. ¿Qué sucede internamente?

```
main()
  ↓ envía tarifa y horas
calcularTotal()
  ↓ realiza el cálculo
return total
  ↓
main() guarda el resultado
```



113 5. ¿Qué ganamos?

113.1 Antes

```
main()
├─ tarifa = determinarTarifa(...)
├─ totalPagar = tarifa * horas
└─ mostrar resultados
```

113.2 Después

```
main()
├─ tarifa = determinarTarifa(...)
├─ totalPagar = calcularTotal(...)
└─ mostrar resultados
```

Ahora el programa principal ya no realiza cálculos directamente.

Únicamente coordina la ejecución de las funciones.

Comprobación

Compila nuevamente el programa.
La salida debe ser exactamente igual que en las versiones anteriores.

¿Qué aprendimos?

Cada función debe tener una única responsabilidad.
`determinarTarifa()` determina la tarifa.
`calcularTotal()` calcula el total.
Separar responsabilidades hace que el programa sea más fácil de comprender y mantener.

113.3 Preparándonos para la siguiente modificación

Solo nos queda un paso.

Moveremos toda la presentación de resultados al procedimiento `mostrarResultados()`.

Cuando terminemos, el `main()` quedará reducido a unas pocas líneas y describirá claramente el flujo del programa.

114 Quinta modificación: crear el procedimiento `mostrarResultados()`

Ya organizamos la entrada de datos y los cálculos del programa.

Solo nos queda una última responsabilidad: presentar la información al usuario.

En esta modificación moveremos todo el reporte final hacia un procedimiento independiente.

115 1. Identificamos el código que vamos a mover

Localiza dentro de `main()` todas las instrucciones utilizadas para mostrar:

- El encabezado.
- El nombre del conductor.
- La placa.
- El tipo de vehículo.
- Las horas.
- La tarifa.
- El total a pagar.

Todo ese bloque tiene una única responsabilidad: **mostrar el resultado del proceso.**

116 2. Creamos el procedimiento

Antes de `main()` agrega el siguiente procedimiento.

```
//-----  
// Muestra el reporte final  
//-----  
void mostrarResultados()  
{  
    mostrarTitulo();  
  
    cout << "Conductor : " << nombreConductor << endl;  
    cout << "Placa      : " << placa << endl;  
  
    cout << "Vehículo  : ";  
  
    switch (tipoVehiculo)  
    {  
        case MOTO:  
            cout << "Moto";  
            break;  
  
        case CARRO:  
            cout << "Carro";  
            break;  
  
        case CAMIONETA:  
            cout << "Camioneta";  
            break;  
  
        default:  
            cout << "No válido";  
    }  
  
    cout << endl;  
    cout << "Horas      : " << horas << endl;  
    cout << "Tarifa      : Q" << tarifa << endl;  
    cout << "-----" << endl;
```

```
    cout << "TOTAL A PAGAR: Q" << totalPagar << endl;  
}
```

Observa algo interesante.

Dentro de este procedimiento estamos llamando nuevamente a:

```
mostrarTitulo();
```

Esto demuestra que un procedimiento puede utilizar otro procedimiento.

117 3. Modificamos `main()`

Elimina todo el bloque de salida y reemplázalo por una sola instrucción.

```
mostrarResultados();
```


118 4. Nuestro nuevo main()

Después de todas las modificaciones, el programa principal queda reducido a unas pocas líneas.

```
int main()
{
    leerDatos();

    tarifa = determinarTarifa(tipoVehiculo);

    totalPagar = calcularTotal(tarifa, horas);

    mostrarResultados();

    return 0;
}
```

Ahora el `main()` puede leerse casi como un algoritmo escrito en lenguaje natural.

1. Leer datos.
2. Determinar la tarifa.
3. Calcular el total.
4. Mostrar resultados.

119 5. ¿Qué cambió?

Comparemos ambas versiones.

119.1 Antes

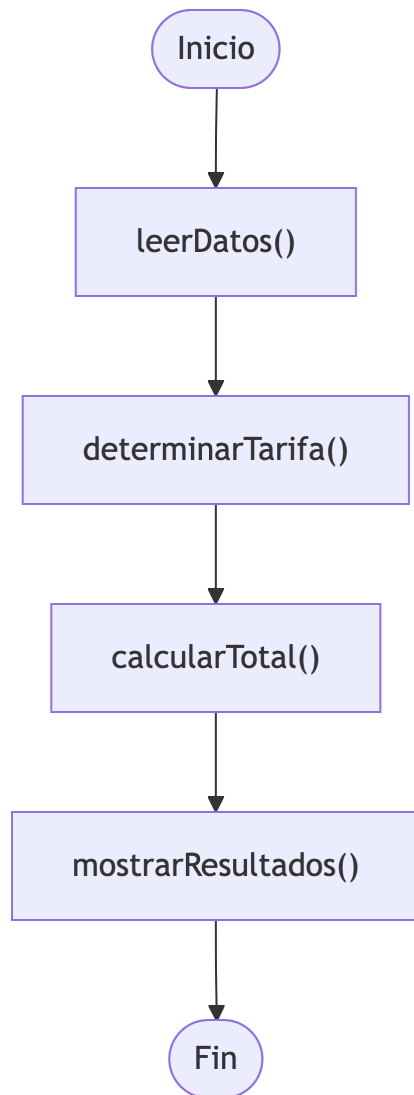
```
main()  
├─ Leer datos  
├─ Determinar tarifa  
├─ Calcular total  
├─ Mostrar encabezado  
├─ Mostrar conductor  
├─ Mostrar vehículo  
├─ Mostrar horas  
├─ Mostrar tarifa  
└─ Mostrar total
```

119.2 Después

```
main()  
├─ leerDatos()  
├─ determinarTarifa()  
├─ calcularTotal()  
└─ mostrarResultados()
```

Observa que el programa realiza exactamente las mismas tareas.

La diferencia es que ahora cada responsabilidad fue trasladada a un módulo independiente.



120 Comprobación final

Compila nuevamente el programa.

Utiliza los mismos datos empleados en el capítulo anterior.

El resultado obtenido debe ser exactamente el mismo.

Si el comportamiento no cambió, significa que la reorganización fue realizada correctamente.

Felicitaciones

Has transformado un programa funcional en un programa mucho mejor organizado.
Eso es precisamente lo que ocurre durante el desarrollo profesional del software.

121 Lo que aprendimos

Durante este capítulo reorganizamos un programa completo utilizando procedimientos, funciones y parámetros.

Responsabilidad	Módulo
Mostrar el encabezado	<code>mostrarTitulo()</code>
Leer información	<code>leerDatos()</code>
Determinar la tarifa	<code>determinarTarifa()</code>
Calcular el total	<code>calcularTotal()</code>
Mostrar resultados	<code>mostrarResultados()</code>

Observa que cada módulo tiene una única responsabilidad.

Gracias a ello, el programa principal quedó reducido a unas pocas líneas fáciles de leer.

Idea clave

Programar mejor no significa escribir más código.

Programar mejor significa organizar el código de manera que cualquier persona pueda comprenderlo y mantenerlo.

122 Preparándonos para el siguiente capítulo

Nuestro Sistema de Control de Parques ya está organizado.

En el siguiente capítulo comenzaremos a trabajar con **arreglos**, lo que permitirá registrar y administrar la información de varios vehículos en lugar de procesar únicamente uno.

123 Laboratorio guiado: desarrollo de una solución

Repaso integrador: variables, decisiones, ciclos, procedimientos, funciones, parámetros y validaciones

124 El reto de hoy

Una pequeña empresa necesita un programa en C++ para **controlar el acceso a un sistema y registrar información básica de una persona**.

Para ingresar, el usuario deberá proporcionar un **nombre de usuario y una contraseña**. El sistema deberá comprobar que las credenciales sean correctas y permitirá un máximo de **tres intentos**. Si los tres intentos son incorrectos, el acceso será bloqueado y el programa finalizará.

Cuando el acceso sea correcto, el sistema mostrará un menú con las siguientes opciones:

1. Registrar datos.
2. Mostrar los datos registrados.
3. Calcular un bono.
4. Consultar el estado de la persona.
5. Salir.

Para registrar a una persona se solicitará:

- nombre completo;
- edad;
- género;
- salario;
- estado activo o inactivo.

El programa deberá **validar las entradas** para evitar datos incorrectos. Por ejemplo, no deberá aceptar texto cuando se espera un número ni valores diferentes a los permitidos en las opciones que correspondan.

Para calcular el bono se utilizarán las siguientes reglas:

- si la persona tiene menos de 50 años, el bono será del **5 % del salario**;
- si tiene 50 años o más, el bono será del **10 % del salario**.

El menú deberá permanecer disponible hasta que el usuario decida salir.

! Antes de escribir código

Leamos el problema y respondamos entre todos:

1. ¿Cuáles son las **entradas**?
2. ¿Cuáles son los **procesos**?
3. ¿Cuáles son las **salidas**?

4. ¿Qué **tipos de variables** necesitaremos?
5. ¿Dónde tendremos que tomar **decisiones**?
6. ¿Qué partes deberán **repetirse**?

Todavía no programemos. Primero entendamos qué debemos construir.

125 1. Comprobemos nuestro análisis

Ahora sí, comparemos nuestras respuestas con una representación general de la solución.

¿Contemplamos todo? ¿Qué nos hizo falta?

125.1 Veamos el problema completo



ALGORITMOS

Laboratorio Guiado

Construyamos juntos un programa completo en C++



Tiempo estimado
40 minutos

Repasaremos lo visto hasta
Procedimientos, Funciones y
Parámetros



El reto de hoy

Construiremos un sistema sencillo que permite autenticar a un usuario y realizar varias operaciones sobre los datos de una persona.

Vista previa del programa funcionando

Así se verá cuando terminemos:

```
=====
      SISTEMA DE ACCESO
=====
Usuario: jorgeg4
Contraseña: *****

Bienvenido al sistema, jorgeg4

=====
      MENU PRINCIPAL
=====
1. Registrar datos
2. Mostrar datos
3. Realizar calculo (bono)
4. Ver estado
5. Salir

Seleccione una opcion: 3

Su bono calculado es: Q225.00

Presione ENTER para continuar...
```



Nuestra meta

En 40 minutos construiremos este programa paso a paso. Empezaremos con lo más simple y poco a poco iremos agregando más funcionalidad.
¡Pregunta, participa y construye tu código!



1. Antes de programar: analicemos

Respondamos juntos antes de escribir código.



1.1 Entradas

- Usuario
- Contraseña
- Opción del menú
- Nombre, edad, género
- Salario
- Estado (activo/inactivo)



1.2 Procesos

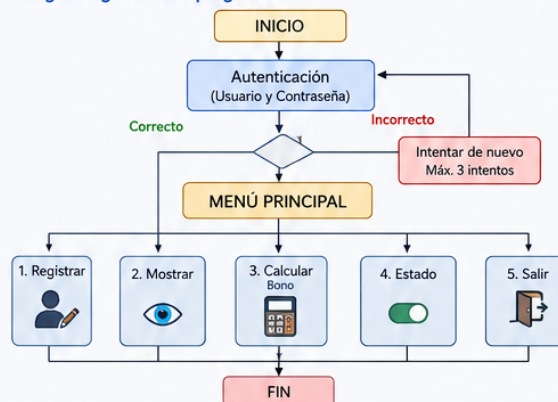
- Validar usuario
- Validar contraseña
- Controlar intentos
- Mostrar menú
- Registrar datos
- Mostrar datos
- Calcular bono
- Determinar estado



1.3 Salidas

- Mensajes de autenticación
- Datos registrados
- Bono calculado
- Estado del usuario
- Mensajes de error
- Confirmaciones

Diagrama general del programa



2. Primer código: ¿qué necesitamos?

¿Qué variable necesitamos como mínimo para controlar nuestro menú?

```
// 2.1 Bloque de variables para el menú
int opcion; // Opción seleccionada por el usuario
```

Pregunta para el grupo

¿Qué tipo de estructura utilizaremos para repetir el menú y permitir varias opciones?

Usaremos un ciclo + switch para seleccionar.

Probemos este primer bloque

Compilamos y ejecutamos... Aún no hace nada, pero ya tenemos la base para empezar.



Recordemos

- int → números enteros
- double → números con decimales
- char → un solo carácter
- string → texto o cadenas de caracteres
- bool → verdadero (true) o falso (false)

3. Construyamos el menú

Mostraremos el menú en un ciclo do-while y usaremos switch.

```
1 // 3.1 Bloque del funcionamiento o Menú
2 do {
3     cout << "\n===== \n";
4     cout << "      MENU PRINCIPAL \n";
5     cout << "===== \n";
6     cout << "1. Registrar datos \n";
7     cout << "2. Mostrar datos \n";
8     cout << "3. Realizar calculo (bono) \n";
9     cout << "4. Ver estado \n";
10    cout << "5. Salir \n";
11    cout << "Seleccione una opcion: ";
12    cin >> opcion;
13    cin.ignore(numeric_limits<streamsize>::max(), '\n');
14
15    switch (opcion) {
16        case 1: cout << "\nOpcion en construccion... \n"; break;
17        case 2: cout << "\nOpcion en construccion... \n"; break;
18        case 3: cout << "\nOpcion en construccion... \n"; break;
19        case 4: cout << "\nOpcion en construccion... \n"; break;
20
21        case 5: cout << "\nGracias por usar el sistema. \n"; break;
22        default: cout << "Opcion invalida. Intente nuevamente. \n";
23    } while (opcion != 5);
```

Por ahora las opciones solo muestran un mensaje. Luego agregaremos su funcionalidad.



¡Cada bloque que construyamos lo probaremos!

La idea es ver cómo poco a poco nuestro programa toma forma.



Avancemos juntos paso a paso y con confianza.

Comparemos

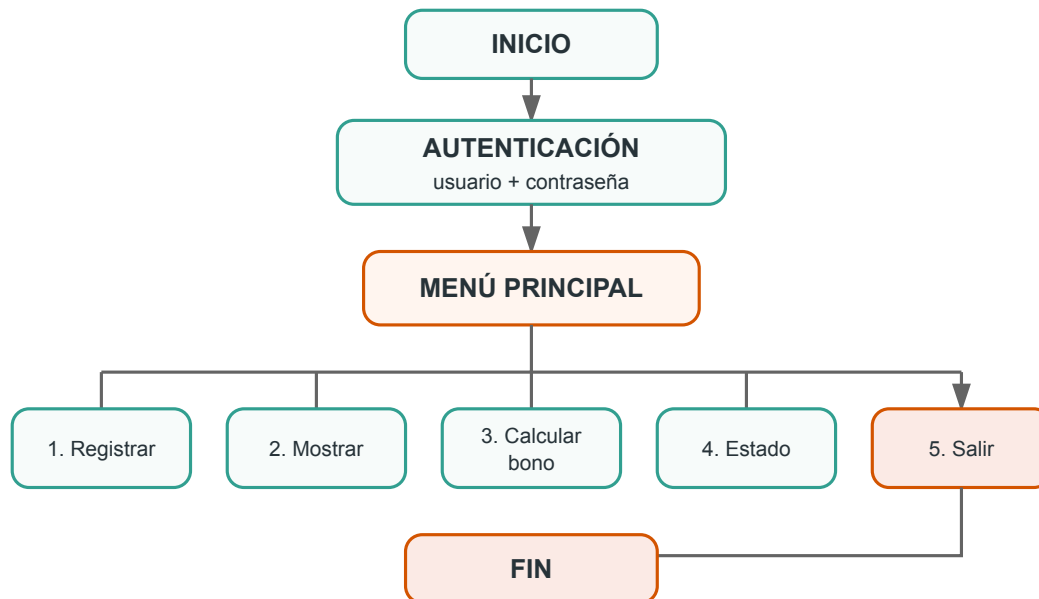
Observemos la infografía y revisemos nuestras respuestas:

- ¿Identificamos todos los datos que necesita el programa?
- ¿Reconocimos que necesitaremos validaciones?
- ¿Detectamos que habrá un menú?
- ¿Pensamos en funciones y procedimientos?
- ¿Qué elemento no habíamos considerado?

125.2 Ahora veamos el flujo de la solución

La infografía nos muestra **qué construiremos**.

El diagrama de flujo nos ayudará ahora a comprender **cómo funcionará**.



Observemos el diagrama

Antes de continuar, identifiquemos:

- dónde inicia el programa;
- qué debe ocurrir antes de mostrar el menú;
- dónde existen decisiones;
- qué partes se repiten;
- cuándo termina el programa.

Con el problema entendido, **ahora sí comenzamos a construir código**.

126 2. Primer código: ¿qué necesitamos para controlar el menú?

i Pregunta para el grupo

Si todavía no hemos construido ninguna funcionalidad, ¿qué variable necesitamos como mínimo para saber qué opción seleccionó el usuario?

```
int opcion;
```

Recordemos:

Tipo	Uso
int	números enteros
double	números con decimales
char	un carácter
string	texto
bool	verdadero o falso

127 3. Construyamos el menú

Preguntemos:

1. ¿Qué estructura permite repetir instrucciones?
2. ¿Qué estructura permite elegir entre varias opciones?
3. ¿Cuándo debe terminar nuestro menú?

Primera versión:

```
#include <iostream>
using namespace std;

int main() {

    int opcion;

    do {
        cout << "\n===== " << endl;
        cout << "          MENU PRINCIPAL" << endl;
        cout << "===== " << endl;
        cout << "1. Registrar datos" << endl;
        cout << "2. Mostrar datos" << endl;
        cout << "3. Realizar calculo (bono)" << endl;
        cout << "4. Ver estado" << endl;
        cout << "5. Salir" << endl;
        cout << "Seleccione una opcion: ";
        cin >> opcion;
        limpiarBuffer();

        switch (opcion) {
            case 1:
                cout << "\nOpcion en construccion..." << endl;
                break;
            case 2:
                cout << "\nOpcion en construccion..." << endl;
                break;
            case 3:
                cout << "\nOpcion en construccion..." << endl;
```

```
        break;
    case 4:
        cout << "\nOpcion en construccion..." << endl;
        break;
    case 5:
        cout << "\nGracias por usar el sistema." << endl;
        break;
    default:
        cout << "\nOpcion invalida." << endl;
    }

    while (opcion != 5);

    return 0;
}
```

127.1 Probemos este primer bloque

Compilemos y ejecutemos.

- ¿Se repite el menú?
- ¿Funciona cada case?
- ¿Qué ocurre si escribimos 9?
- ¿Qué ocurre al seleccionar 5?

Todavía no hace mucho, pero **ya tenemos una base funcional**.

128 4. ¿Qué datos necesita la persona?

i Pregunta para el grupo

Si la opción 1 debe registrar información, ¿qué variables necesitamos y de qué tipo debería ser cada una?

Como ahora utilizaremos variables de tipo `string`, agregamos una nueva librería **al inicio del programa**:

```
#include <string>
```

Nuestra cabecera queda así:

```
#include <iostream>
#include <string>

using namespace std;
```

Ahora, **dentro de `main()` y antes del `do...while` del menú**, agregamos las variables de la persona:

```
string nombre;
int edad;
char genero;
double salario;
bool activo;
bool datosRegistrados = false;
```

¿Para qué sirve `datosRegistrados`?

Nos permitirá evitar que el usuario intente mostrar o calcular información antes de haber registrado los datos.

129 5. Provocamos un problema

Supongamos que hacemos esto:

```
cout << "Ingrese edad: ";  
cin >> edad;
```

Ahora escribamos:

```
Ingrese edad: Jorge
```

 ¿Qué ocurrió?

El programa esperaba un entero y recibió texto.

Entonces aparece una nueva necesidad:

no podemos confiar en que todas las entradas serán correctas.

130 6. Construyamos nuestras validaciones

Antes de escribir las funciones de validación debemos preparar correctamente la parte superior de nuestro programa.

130.1 6.1 Actualicemos las librerías

Para las validaciones necesitaremos dos librerías nuevas:

```
#include <limits> // numeric_limits: ayuda a limpiar el buffer
#include <cctype> // toupper: permite convertir caracteres a mayúsculas
```

Hasta este momento, la cabecera de nuestro programa debe quedar así:

```
#include <iostream>
#include <string>
#include <limits>
#include <cctype>

using namespace std;
```

Pregunta para el grupo

Si las funciones de validación estarán escritas **después de main()**, ¿cómo sabrá C++ que existen cuando las llamemos desde el programa principal?

Necesitamos declarar sus **prototipos** antes de `main()`.

Un prototipo le informa al compilador:

- el nombre de la función;
- el tipo de dato que devuelve;
- los parámetros que recibe.

Agregamos debajo de `using namespace std;`

```
// Prototipos de validación
void limpiarBuffer();
int leerEntero(string mensaje);
double leerDecimal(string mensaje);
string leerTexto(string mensaje);
char leerCaracter(string mensaje);
bool leerBooleano(string mensaje);
```

Nuestra estructura, por ahora, debe verse así:

```
#include <iostream>
#include <string>
#include <limits>
#include <cctype>

using namespace std;

// Prototipos de validación
void limpiarBuffer();
int leerEntero(string mensaje);
double leerDecimal(string mensaje);
string leerTexto(string mensaje);
char leerCaracter(string mensaje);
bool leerBooleano(string mensaje);

int main() {

    // Variables y menú...

    return 0;
}

// Aquí escribiremos las funciones completas
```

! Regla que utilizaremos desde ahora

Cada vez que creamos una nueva función o procedimiento que estará escrito después de `main()`, primero agregaremos su **prototipo** en la parte superior. Así podremos **compilar y probar cada bloque** antes de continuar.

130.2 6.2 Procedimiento para limpiar el buffer


```
void limpiarBuffer() {  
    cin.clear();  
    cin.ignore((numeric_limits<streamsize>::max)(), '\n');  
}
```

Preguntemos:

- ¿Recibe parámetros? → No.
- ¿Devuelve un valor? → No.
- ¿Realiza una tarea? → Sí.

Por eso utilizamos `void`.

130.3 6.3 Validar un entero

```
int leerEntero(string mensaje) {  
    int valor;  
  
    while (true) {  
        cout << mensaje;  
  
        if (cin >> valor) {  
            limpiarBuffer();  
            return valor;  
        }  
  
        cout << "Error: debe ingresar un numero entero." << endl;  
        limpiarBuffer();  
    }  
}
```

Preguntemos:

- ¿Qué recibe? → un `string`.
- ¿Cómo se llama ese dato recibido? → **parámetro**.
- ¿Qué devuelve? → un `int`.

130.4 6.4 Validar un decimal

```
double leerDecimal(string mensaje) {
    double valor;

    while (true) {
        cout << mensaje;

        if (cin >> valor) {
            limpiarBuffer();
            return valor;
        }

        cout << "Error: debe ingresar un numero valido." << endl;
        limpiarBuffer();
    }
}
```

130.5 6.5 Validar texto

```
string leerTexto(string mensaje) {
    string texto;

    do {
        cout << mensaje;
        getline(cin, texto);

        if (texto.empty()) {
            cout << "Error: el texto no puede estar vacio." << endl;
        }

    } while (texto.empty());

    return texto;
}
```

130.6 6.6 Validar un carácter

```
char leerCaracter(string mensaje) {
    char valor;

    while (true) {
        cout << mensaje;
        cin >> valor;
    }
}
```

```
    limpiarBuffer();

    valor = toupper(valor);

    if (valor == 'M' || valor == 'F') {
        return valor;
    }

    cout << "Error: ingrese M o F." << endl;
}
}
```

130.7 6.7 Validar un valor lógico

En lugar de pedir al usuario que escriba `true` o `false`, utilizaremos una entrada más natural: **S/N**.

```
bool leerBooleano(string mensaje) {
    char respuesta;

    do {
        cout << mensaje << " (S/N): ";
        cin >> respuesta;
        limpiarBuffer();

        respuesta = toupper(respuesta);

        if (respuesta != 'S' && respuesta != 'N') {
            cout << "Error: ingrese S o N." << endl;
        }

    } while (respuesta != 'S' && respuesta != 'N');

    return respuesta == 'S';
}
```

💡 Observemos algo importante

`leerBooleano()` recibe un `string`, internamente utiliza un `char` y finalmente devuelve un `bool`.

¡Estamos conectando varios conceptos en una sola función!

131 7. Hagamos funcionar la opción Registrar

Ya tenemos:

- las librerías necesarias;
- las variables;
- los prototipos de validación;
- y las funciones completas escritas después de `main()`.

Ahora sí podemos utilizarlas desde el menú sin que el compilador indique que son funciones desconocidas.

Reemplazamos el mensaje "Opcion en construccion..." del case 1.

```
case 1:
    nombre = leerTexto("Nombre completo: ");
    edad = leerEntero("Edad: ");
    genero = leerCaracter("Genero (M/F): ");
    salario = leerDecimal("Salario: Q");
    activo = leerBooleano("Usuario activo");

    datosRegistrados = true;

    cout << "\nDatos registrados correctamente." << endl;
    break;
```

131.1 Probemos otra vez

Intentemos deliberadamente:

- escribir letras en la edad;
- dejar vacío el nombre;
- escribir X en género;
- escribir texto en salario;
- responder algo diferente de S o N.

Nuestro programa ya empieza a defenderse de entradas incorrectas.

132 8. ¿Llenamos main () de instrucciones?

Podríamos colocar todos los `cout` dentro del `case 2`.

Pero acabamos de aprender otra forma de organizar el programa.

Como vamos a crear un nuevo procedimiento y lo escribiremos después de `main()`, primero agregamos su prototipo junto a los anteriores:

```
void mostrarDatos(string nombre, int edad, char genero,
                  double salario, bool activo);
```

💡 Sigamos la misma regla

Primero el prototipo → luego usamos el procedimiento → después escribimos su implementación completa.

i Pregunta para el grupo

Si una tarea solamente debe **mostrar información** y no necesitamos recibir un resultado, ¿podemos convertirla en un procedimiento?

```
void mostrarDatos(string nombre, int edad, char genero,
                  double salario, bool activo) {

    cout << "\n===== " << endl;
    cout << "          DATOS REGISTRADOS" << endl;
    cout << "===== " << endl;
    cout << "Nombre : " << nombre << endl;
    cout << "Edad   : " << edad << endl;
    cout << "Genero : " << genero << endl;
    cout << "Salario: Q" << salario << endl;
    cout << "Activo : " << (activo ? "Si" : "No") << endl;
}
```

Y nuestro `case 2` queda mucho más limpio:

```
case 2:
    if (datosRegistrados) {
        mostrarDatos(nombre, edad, genero, salario, activo);
    }
    else {
        cout << "\nPrimero debe registrar los datos." << endl;
    }
    break;
```


133 9. Ahora necesitamos obtener un resultado

La opción 3 calculará un bono.

Antes de utilizar la nueva función agregamos su prototipo en la parte superior:

```
double calcularBono(double salario, int edad);
```

Regla sencilla:

- edad menor de 50 años → 5 %;
- edad de 50 años o más → 10 %.

i Pregunta para el grupo

¿Procedimiento o función?

Necesitamos que el cálculo nos **devuelva un valor**, por lo que construiremos una función.

```
double calcularBono(double salario, int edad) {  
    double porcentaje;  
  
    if (edad >= 50) {  
        porcentaje = 0.10;  
    }  
    else {  
        porcentaje = 0.05;  
    }  
  
    return salario * porcentaje;  
}
```

Ahora el case 3:

```
case 3:
    if (datosRegistrados) {
        double bono = calcularBono(salario, edad);
        cout << "\nBono calculado: Q" << bono << endl;
    }
    else {
        cout << "\nPrimero debe registrar los datos." << endl;
    }
    break;
```

134 10. Consultemos el estado

Primero declaramos su prototipo:

```
void mostrarEstado(bool activo);
```

Luego construimos el procedimiento completo después de `main()`:

```
void mostrarEstado(bool activo) {  
    if (activo) {  
        cout << "\nEl usuario se encuentra ACTIVO." << endl;  
    }  
    else {  
        cout << "\nEl usuario se encuentra INACTIVO." << endl;  
    }  
}
```

Y completamos:

```
case 4:  
    if (datosRegistrados) {  
        mostrarEstado(activo);  
    }  
    else {  
        cout << "\nPrimero debe registrar los datos." << endl;  
    }  
    break;
```


135 11. Nuestro programa funciona... ¿pero cualquiera puede entrar?

Ahora agregaremos seguridad.

Primero agregamos los prototipos de las funciones de autenticación:

```
bool validarUsuario(string usuario);  
bool validarPassword(string usuario, string password);
```

Luego agregamos, dentro de `main()`, las variables necesarias:

```
string usuario;  
string password;  
int intentos = 0;  
bool acceso = false;
```

135.1 11.1 Validar usuario

```
bool validarUsuario(string usuario) {  
    return usuario == "mariojdi3" ||  
           usuario == "jorgeg4" ||  
           usuario == "raulc13";  
}
```

135.2 11.2 Validar usuario y contraseña juntos

```
bool validarPassword(string usuario, string password) {  
    if (usuario == "mariojdi3" &&  
        password == "myHouseisntalone1289") {  
        return true;  
    }
```

```
}  
  
if (usuario == "jorgeg4" &&  
    password == "aSushiRestaurant3456") {  
    return true;  
}  
  
if (usuario == "raulc13" &&  
    password == "watermelonandOrange") {  
    return true;  
}  
  
return false;  
}
```

! ¿Por qué enviamos usuario y contraseña?

No basta con comprobar que la contraseña existe.

Debemos comprobar que **esa contraseña corresponde a ese usuario**.

Aquí tenemos una función que recibe **dos parámetros**.

136 12. Control de intentos

Antes de agregar la autenticación al flujo principal, nuestra zona de prototipos ya debe verse así:

```
// Prototipos de validación
void limpiarBuffer();
int leerEntero(string mensaje);
double leerDecimal(string mensaje);
string leerTexto(string mensaje);
char leerCaracter(string mensaje);
bool leerBooleano(string mensaje);

// Prototipos de autenticación
bool validarUsuario(string usuario);
bool validarPassword(string usuario, string password);

// Prototipos de procesos
void mostrarDatos(string nombre, int edad, char genero,
                  double salario, bool activo);
double calcularBono(double salario, int edad);
void mostrarEstado(bool activo);
```

Ahora sí, **antes de mostrar el menú**, agregamos el control de intentos:

```
while (intentos < 3 && !acceso) {

    cout << "\n===== " << endl;
    cout << "          SISTEMA DE ACCESO" << endl;
    cout << "===== " << endl;

    usuario = leerTexto("Usuario: ");
    password = leerTexto("Contraseña: ");

    if (validarUsuario(usuario) &&
        validarPassword(usuario, password)) {

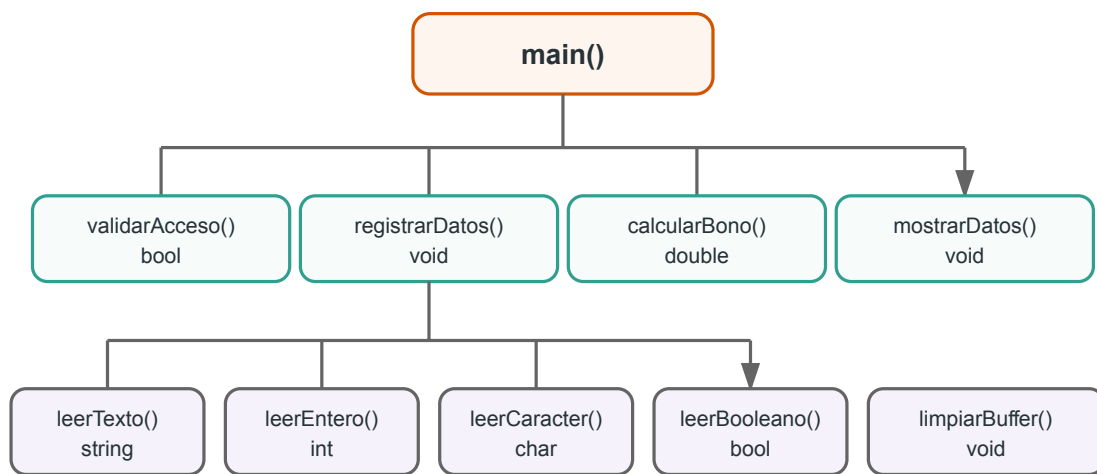
        acceso = true;
        cout << "\nBienvenido al sistema, "
```

```
        << usuario << "." << endl;
    }
    else {
        intentos++;

        cout << "\nUsuario o contraseña incorrectos." << endl;
        cout << "Intento " << intentos << " de 3." << endl;
    }
}

if (!acceso) {
    cout << "\nUsuario bloqueado. Fin del programa." << endl;
    return 0;
}
```


137 13. ¿Cómo quedó organizado nuestro programa?



Un programa grande se construye con pequeñas tareas.

💡 Idea clave

Un programa grande no tiene que escribirse como un enorme bloque de instrucciones. Podemos dividir el problema en **pequeñas tareas**, resolver cada una y luego hacer que trabajen juntas.

138 14. Código completo del laboratorio

Para la versión final agregaremos una última librería:

```
#include <iomanip>
```

La utilizaremos con:

```
cout << fixed << setprecision(2);
```

para mostrar los valores monetarios con **dos decimales**.

Este es el programa integrado para probarlo antes o después de construirlo paso a paso.

```
#include <iostream>
#include <string>
#include <limits>
#include <cctype>
#include <iomanip>

using namespace std;

// Prototipos
void limpiarBuffer();
int leerEntero(string mensaje);
double leerDecimal(string mensaje);
string leerTexto(string mensaje);
char leerCaracter(string mensaje);
bool leerBooleano(string mensaje);

bool validarUsuario(string usuario);
bool validarPassword(string usuario, string password);

void mostrarDatos(string nombre, int edad, char genero,
                  double salario, bool activo);
```

```
double calcularBono(double salario, int edad);
void mostrarEstado(bool activo);

int main() {

    // Variables de autenticacion
    string usuario;
    string password;
    int intentos = 0;
    bool acceso = false;

    // Variables del menu y registro
    int opcion;
    string nombre;
    int edad = 0;
    char genero = ' ';
    double salario = 0.0;
    bool activo = false;
    bool datosRegistrados = false;

    cout << fixed << setprecision(2);

    // Autenticacion
    while (intentos < 3 && !acceso) {

        cout << "\n===== " << endl;
        cout << "          SISTEMA DE ACCESO" << endl;
        cout << "===== " << endl;

        usuario = leerTexto("Usuario: ");
        password = leerTexto("Contraseña: ");

        if (validarUsuario(usuario) &&
            validarPassword(usuario, password)) {

            acceso = true;

            cout << "\nBienvenido al sistema, "
                 << usuario << "." << endl;
        }
        else {
            intentos++;

            cout << "\nUsuario o contraseña incorrectos." << endl;
            cout << "Intento " << intentos << " de 3." << endl;
        }
    }
}
```

```
}

if (!acceso) {
    cout << "\nUsuario bloqueado. Fin del programa." << endl;
    return 0;
}

// Menu principal
do {

    cout << "\n===== " << endl;
    cout << "          MENU PRINCIPAL" << endl;
    cout << "===== " << endl;
    cout << "1. Registrar datos" << endl;
    cout << "2. Mostrar datos" << endl;
    cout << "3. Realizar calculo (bono)" << endl;
    cout << "4. Ver estado" << endl;
    cout << "5. Salir" << endl;

    opcion = leerEntero("Seleccione una opcion: ");

    switch (opcion) {

        case 1:
            cout << "\n--- REGISTRO DE DATOS ---" << endl;

            nombre = leerTexto("Nombre completo: ");
            edad = leerEntero("Edad: ");
            genero = leerCaracter("Genero (M/F): ");
            salario = leerDecimal("Salario: Q");
            activo = leerBooleano("Usuario activo");

            datosRegistrados = true;

            cout << "\nDatos registrados correctamente." << endl;
            break;

        case 2:
            if (datosRegistrados) {
                mostrarDatos(nombre, edad, genero,
                              salario, activo);
            }
            else {
                cout << "\nPrimero debe registrar los datos." << endl;
            }
            break;
```

```
        case 3:
            if (datosRegistrados) {

                double bono = calcularBono(salario, edad);

                cout << "\nBono calculado: Q"
                     << bono << endl;
            }
            else {
                cout << "\nPrimero debe registrar los datos." << endl;
            }
            break;

        case 4:
            if (datosRegistrados) {
                mostrarEstado(activo);
            }
            else {
                cout << "\nPrimero debe registrar los datos." << endl;
            }
            break;

        case 5:
            if (leerBooleano("Realmente desea salir")) {
                cout << "\nGracias por usar el sistema." << endl;
            }
            else {
                opcion = 0;
            }
            break;

        default:
            cout << "\nOpcion invalida. Intente nuevamente." << endl;
    }

    while (opcion != 5);

    return 0;
}

// =====
// VALIDACIONES
// =====

void limpiarBuffer() {
    cin.clear();
}
```

```
    cin.ignore((numeric_limits<streamsize>::max)(), '\n');
}

int leerEntero(string mensaje) {
    int valor;
    while (true) {
        cout << mensaje;

        if (cin >> valor) {
            limpiarBuffer();
            return valor;
        }

        cout << "Error: debe ingresar un numero entero." << endl;
        limpiarBuffer();
    }
}

double leerDecimal(string mensaje) {
    double valor;
    while (true) {
        cout << mensaje;

        if (cin >> valor) {
            limpiarBuffer();
            return valor;
        }

        cout << "Error: debe ingresar un numero valido." << endl;
        limpiarBuffer();
    }
}

string leerTexto(string mensaje) {
    string texto;
    do {
```

```
        cout << mensaje;
        getline(cin, texto);

        if (texto.empty()) {
            cout << "Error: el texto no puede estar vacio." << endl;
        }

    } while (texto.empty());

    return texto;
}

char leerCaracter(string mensaje) {

    char valor;

    while (true) {

        cout << mensaje;
        cin >> valor;
        limpiarBuffer();

        valor = toupper(valor);

        if (valor == 'M' || valor == 'F') {
            return valor;
        }

        cout << "Error: ingrese M o F." << endl;
    }
}

bool leerBooleano(string mensaje) {

    char respuesta;

    do {

        cout << mensaje << " (S/N): ";
        cin >> respuesta;
        limpiarBuffer();

        respuesta = toupper(respuesta);

        if (respuesta != 'S' && respuesta != 'N') {
```



```
        cout << "Error: ingrese S o N." << endl;
    }

    } while (respuesta != 'S' && respuesta != 'N');

    return respuesta == 'S';
}

// =====
// AUTENTICACION
// =====

bool validarUsuario(string usuario) {

    return usuario == "mariojdi3" ||
           usuario == "jorgeg4" ||
           usuario == "raulcl3";
}

bool validarPassword(string usuario, string password) {

    if (usuario == "mariojdi3" &&
        password == "myHouseisntalone1289") {
        return true;
    }

    if (usuario == "jorgeg4" &&
        password == "aSushiRestaurant3456") {
        return true;
    }

    if (usuario == "raulcl3" &&
        password == "watermelonandOrange") {
        return true;
    }

    return false;
}

// =====
// PROCESOS
// =====

void mostrarDatos(string nombre, int edad, char genero,
```

```
double salario, bool activo) {

    cout << "\n===== " << endl;
    cout << "          DATOS REGISTRADOS" << endl;
    cout << "===== " << endl;

    cout << "Nombre : " << nombre << endl;
    cout << "Edad   : " << edad << endl;
    cout << "Genero : " << genero << endl;
    cout << "Salario: Q" << salario << endl;
    cout << "Activo : " << (activo ? "Si" : "No") << endl;
}

double calcularBono(double salario, int edad) {

    double porcentaje;

    if (edad >= 50) {
        porcentaje = 0.10;
    }
    else {
        porcentaje = 0.05;
    }

    return salario * porcentaje;
}

void mostrarEstado(bool activo) {

    if (activo) {
        cout << "\nEl usuario se encuentra ACTIVO." << endl;
    }
    else {
        cout << "\nEl usuario se encuentra INACTIVO." << endl;
    }
}
```

139 15. ¿Qué acabamos de utilizar?

Concepto	Aplicación
int	edad, opción, intentos
double	salario y bono
char	género y respuestas S/N
string	nombre, usuario y contraseña
bool	acceso, estado y datos registrados
if / else	decisiones y validaciones
switch	opciones del menú
while	validaciones y autenticación
do-while	repetición del menú
procedimientos	limpiar, mostrar información y estado
funciones	leer, validar y calcular
parámetros	enviar información entre funciones
return	devolver resultados

140 16. Cierre del laboratorio

140.1 De una variable a un programa completo

Comenzamos preguntándonos qué necesitábamos para controlar un menú:

```
int opcion;
```

Y terminamos construyendo un programa que integra **análisis, variables, decisiones, ciclos, validaciones, procedimientos, funciones, parámetros y valores de retorno.**

El objetivo no era memorizar código. El objetivo era comprobar que podemos **analizar un problema, dividirlo y construir la solución paso a paso.**

140.2 Reto rápido

Si terminaste antes:

1. valida que la edad esté entre 18 y 100 años;
2. evita salarios negativos;
3. muestra cuántos intentos de autenticación le quedaron al usuario;
4. agrega una sexta opción al menú sin modificar las funciones existentes.

No intentes resolver todo al mismo tiempo. Haz **un cambio, compila y prueba** antes de continuar.

141 Arreglos y vectores: muchas casillas, una sola estructura

De una fila de Excel al recorrido de datos en C++

142 El reto de hoy

Hasta ahora hemos trabajado con variables individuales:

```
int nota1;  
int nota2;  
int nota3;
```

Esto funciona cuando tenemos pocos datos.

Pero imaginemos que necesitamos guardar las notas de **10, 50 o 100 estudiantes**.

! Pregunta para el grupo

¿Tendría sentido crear manualmente `nota1, nota2, nota3...` hasta `nota100`?
Hoy aprenderemos una forma diferente de organizar muchos datos del mismo tipo: **los arreglos o vectores**.

143 1. Antes de programar: imaginemos una fila de Excel

Pensemos en una fila formada por **10 casillas**.

Pero haremos algo especial: en lugar de numerarlas del 1 al 10, las numeraremos como lo hace un arreglo en programación:

0, 1, 2, 3, 4, 5, 6, 7, 8 y 9.

Una fila de “Excel” convertida en vector

Índice	0	1	2	3	4	5	6	7	8	9
Valor	0	2	4	6	8	10	12	14	16	18

Primera posición = 0

Última posición = 9

💡 Idea clave

Si tenemos **10 casillas**, la primera posición es 0 y la última es 9.
En general, si un arreglo tiene n elementos, sus posiciones van:

$0, 1, 2, \dots, n - 1$

144 2. Dos conceptos que no debemos confundir

En cada casilla existen dos cosas diferentes:

- **Índice:** indica dónde está la casilla.
- **Valor:** es el dato almacenado dentro de esa casilla.

Por ejemplo:

Índice	0	1	2	3	4
Valor	15	20	8	12	30

Entonces:

- en la posición 0 está almacenado 15;
- en la posición 1 está almacenado 20;
- en la posición 4 está almacenado 30.

Preguntemos

Si quiero obtener el valor 12, ¿qué posición debo visitar?
La respuesta es la posición **3**, aunque visualmente sea la cuarta casilla.

145 3. Llenemos las casillas manualmente

Supongamos que queremos almacenar los primeros diez números pares.

Al inicio tenemos:

Índice	0	1	2	3	4	5	6	7	8	9
Valor	?	?	?	?	?	?	?	?	?	?

Ahora llenemos una casilla a la vez:

Paso	Índice visitado	Valor que colocamos
1	0	0
2	1	2
3	2	4
4	3	6
5	4	8
6	5	10
7	6	12
8	7	14
9	8	16
10	9	18

Al terminar:

Índice	0	1	2	3	4	5	6	7	8	9
Valor	0	2	4	6	8	10	12	14	16	18

146 4. Ahora recorramos la fila manualmente

Recorrer un arreglo significa visitar sus posiciones una por una.

Recorrer significa visitar cada posición



El índice i cambia; la misma instrucción trabaja con una casilla diferente.

Podríamos describir el proceso así:

```
Visitar posición 0
Mostrar su valor

Visitar posición 1
Mostrar su valor

Visitar posición 2
Mostrar su valor

...

Visitar posición 9
Mostrar su valor
```

i ¿Observamos un patrón?

La acción siempre es la misma.
Lo único que cambia es la **posición** que estamos visitando.
Eso debería recordarnos algo que ya conocemos: **los ciclos**.

147 5. De las casillas de Excel a un arreglo en C++

En C++ podemos representar nuestras diez casillas así:

```
int numeros[10];
```

Leamos esta instrucción:

Cree una estructura llamada `numeros` capaz de almacenar **10 valores enteros**.

Las posiciones disponibles serán:

```
numeros[0]  
numeros[1]  
numeros[2]  
numeros[3]  
numeros[4]  
numeros[5]  
numeros[6]  
numeros[7]  
numeros[8]  
numeros[9]
```

Atención

`numeros[10]` **no es una posición válida** en un arreglo de 10 elementos. El 10 utilizado al declarar el arreglo representa su **cantidad de elementos**; sus índices válidos son de 0 a 9.

148 6. Hagamos exactamente lo mismo que hicimos a mano

Podemos llenar el arreglo manualmente:

```
numeros[0] = 0;  
numeros[1] = 2;  
numeros[2] = 4;  
numeros[3] = 6;  
numeros[4] = 8;  
numeros[5] = 10;  
numeros[6] = 12;  
numeros[7] = 14;  
numeros[8] = 16;  
numeros[9] = 18;
```

Y también podemos mostrarlo manualmente:

```
cout << numeros[0] << endl;  
cout << numeros[1] << endl;  
cout << numeros[2] << endl;  
cout << numeros[3] << endl;  
cout << numeros[4] << endl;  
cout << numeros[5] << endl;  
cout << numeros[6] << endl;  
cout << numeros[7] << endl;  
cout << numeros[8] << endl;  
cout << numeros[9] << endl;
```

! Hagamos una pausa

El arreglo ya resolvió un problema: no necesitamos crear diez variables con nombres diferentes.

Pero todavía estamos repitiendo muchas instrucciones.

¿Qué estructura conocemos para repetir?

149 7. Aparece el ciclo `for`

Podemos utilizar una variable `i` para representar la posición actual:

```
for (int i = 0; i < 10; i++) {  
    cout << numeros[i] << endl;  
}
```

Analicemos:

Parte	Significado
<code>int i = 0</code>	comenzamos en la primera posición
<code>i < 10</code>	continuamos mientras exista una posición válida
<code>i++</code>	avanzamos a la siguiente posición
<code>numeros[i]</code>	accedemos al elemento que corresponde al índice actual

149.1 La conexión importante

El ciclo `for` no está haciendo algo mágico.

Está automatizando exactamente el recorrido que primero hicimos **manualmente**.

150 8. Ahora llenemos el arreglo con un ciclo

En lugar de escribir diez asignaciones:

```
for (int i = 0; i < 10; i++) {  
    numeros[i] = i * 2;  
}
```

Observemos algunas vueltas:

i	Expresión $i * 2$	Casilla modificada
0	0	numeros[0]
1	2	numeros[1]
2	4	numeros[2]
3	6	numeros[3]
...	...	...
9	18	numeros[9]

151 9. Nuestro primer programa completo

```
#include <iostream>
using namespace std;

int main() {

    int numeros[10];

    // Llenamos el arreglo
    for (int i = 0; i < 10; i++) {
        numeros[i] = i * 2;
    }

    // Recorremos y mostramos el arreglo
    for (int i = 0; i < 10; i++) {
        cout << "Posicion " << i
              << " = " << numeros[i] << endl;
    }

    return 0;
}
```


152 10. Ahora que los datos los ingrese el usuario

Podemos reutilizar el mismo recorrido:

```
#include <iostream>
using namespace std;

int main() {
    int edades[5];

    for (int i = 0; i < 5; i++) {
        cout << "Ingrese la edad para la posicion " << i << ": ";
        cin >> edades[i];
    }

    cout << "\nDatos almacenados:" << endl;

    for (int i = 0; i < 5; i++) {
        cout << "edades[" << i << "] = " << edades[i] << endl;
    }

    return 0;
}
```

i Antes de ejecutarlo

Predigamos:

1. ¿Cuál será la primera posición utilizada?
2. ¿Cuál será la última?
3. ¿Cuántas veces se ejecutará cada `for`?
4. ¿Qué ocurriría si cambiamos `i < 5` por `i <= 5`?

153 11. Hagamos operaciones con todos los elementos

Una vez que sabemos recorrer el arreglo, podemos procesar sus datos.

Por ejemplo, sumarlos:

```
int suma = 0;

for (int i = 0; i < 5; i++) {
    suma = suma + edades[i];
}

cout << "Suma = " << suma << endl;
```

Y calcular el promedio:

```
double promedio = suma / 5.0;

cout << "Promedio = " << promedio << endl;
```

💡 Lo verdaderamente importante

Aprender arreglos no consiste únicamente en memorizar:

```
int datos[10];
```

Lo importante es comprender tres acciones:

1. **crear** el arreglo;
2. **acceder** a una posición mediante su índice;
3. **recorrer** sus elementos para leerlos, modificarlos o procesarlos.

154 12. Actividad guiada en clase

Trabajemos primero sin programar.

Tenemos el siguiente vector:

Índice	0	1	2	3	4	5
Valor	12	7	25	9	18	4

Respondamos:

1. ¿Cuál es el valor de la posición 0?
2. ¿En qué posición está el valor 25?
3. ¿Cuál es el último índice válido?
4. Si recorremos de izquierda a derecha, ¿en qué orden aparecen los valores?
5. ¿Cuál es la suma de todos los elementos?
6. ¿Cuál es el mayor valor?

Después escribamos un programa que almacene estos datos y los muestre usando un `for`.

155 13. Mini reto

Construya un programa que:

1. cree un arreglo para almacenar **5 notas**;
2. solicite las cinco notas al usuario;
3. muestre cada nota junto con su índice;
4. calcule la suma;
5. calcule el promedio.

 Por ahora

No busquemos todavía la solución más sofisticada.

El objetivo es dominar la relación:

casilla → **índice** → **valor** → **recorrido** → **procesamiento**.

156 14. Cierre de la clase

Hoy pasamos de una idea conocida —una fila de casillas— a una estructura fundamental de programación.

Podemos resumirlo así:

```
Muchas variables relacionadas
      ↓
Un arreglo
      ↓
Cada elemento tiene un índice
      ↓
Los índices comienzan en 0
      ↓
Un ciclo permite recorrerlos
      ↓
Podemos leer, modificar y procesar todos los datos
```

156.1 Si comprendimos esto, estamos listos para el laboratorio

En el laboratorio podremos trabajar con problemas donde el arreglo ya no solo almacene datos, sino que tengamos que:

- buscar valores;
- encontrar máximos y mínimos;
- contar elementos que cumplan una condición;
- calcular promedios;
- modificar posiciones;
- combinar arreglos con funciones y procedimientos.

157 15. Para recordar

Un arreglo es un conjunto de elementos del mismo tipo almacenados bajo un mismo nombre y diferenciados por su posición.

Y nuestra regla de oro:

Si hay n elementos, los índices válidos van de 0 a $n - 1$.

158 Síntesis visual del capítulo

A continuación se presenta una síntesis visual de los conceptos fundamentales que hemos trabajado sobre arreglos y vectores.

C++



Algoritmos

Arreglos y Vectores

1.1 De Excel al código: comprendiendo los índices desde 0



Objetivo

Entender cómo funcionan los arreglos (vectores) en C++ a partir de una analogía con una fila de Excel indexada desde 0, aprender a llenarla, recorrerla y realizar operaciones básicas.

1 1. Simulando una fila de Excel con índices desde 0

En programación, un **arreglo** o **vector** almacena elementos en posiciones consecutivas. Cada posición se identifica con un **índice**, y siempre comenzamos a contar desde 0.

Imaginemos una fila de Excel con 10 casillas:

Índice	0	1	2	3	4	5	6	7	8	9
Valor	?	?	?	?	?	?	?	?	?	?

Índices en C++

Siempre empiezan en 0. Si un arreglo tiene n elementos, sus índices van de 0 a $n-1$.

2 Llenado manual paso a paso

Supongamos que queremos almacenar los primeros 10 números pares.

1 Estado inicial Todas las casillas están vacías.	Índice	0	1	2	3	4	5	6	7	8	9
	Valor	?	?	?	?	?	?	?	?	?	?
2 Ingresamos el primer valor Colocamos 0 en el índice 0.	Índice	0	1	2	3	4	5	6	7	8	9
	Valor	0	?	?	?	?	?	?	?	?	?
3 Continuamos llenando Colocamos 2 en el índice 1.	Índice	0	1	2	3	4	5	6	7	8	9
	Valor	0	2	?	?	?	?	?	?	?	?
4 Seguimos llenando Colocamos 4 en el índice 2.	Índice	0	1	2	3	4	5	6	7	8	9
	Valor	0	2	4	?	?	?	?	?	?	?
5 Proceso completo Repetimos hasta llenar todas las casillas.	Índice	0	1	2	3	4	5	6	7	8	9
	Valor	0	2	4	6	8	10	12	14	16	18

**¡Listo!**

Ya tenemos nuestra fila de Excel llena con los 10 primeros números pares.

Visualización del vector

El arreglo se comporta como una fila de casillas numeradas.



Idea clave

El índice identifica la posición, el valor es el dato almacenado en esa posición.



Nota

En Excel normalmente contamos desde 1, pero en programación contamos desde 0.

3 Recorrido manual del vector

Ahora recorramos la fila para ver cada valor en orden, mostrando el índice y el valor.

Dirección del recorrido

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Recorriendo...

Índice 0 → Valor 0
 Índice 1 → Valor 2
 Índice 2 → Valor 4
 ...
 Índice 9 → Valor 18



Siempre que recorremos un arreglo, usamos un índice que va desde 0 hasta $n-1$.

4 De Excel al código en C++

Lo que acabamos de hacer manualmente, lo podemos representar en C++ con un arreglo (vector).

```
1 int numeros[10]; // Arreglo de 10 enteros
2 // índices: 0 a 9
3 numeros[0] = 0;
4 numeros[1] = 2;
5 numeros[2] = 4;
6 ...
7 numeros[9] = 18;
```



Pregunta importante

¿Te imaginas si tuviéramos que escribir cada instrucción para llenar o recorrer muchos elementos? ¡Sería muy tedioso! Aquí es donde los ciclos harán nuestro trabajo mucho más fácil.

Nota: para los programas de clase en C++, especialmente en windows, puedes colocar:

```
#include <iostream>
#include <clocale>
using namespace std;
int main() {
    setlocale(LC_ALL, "");
    cout << "Ingrese la calificación del estudiante:" << endl;
    cout << "Número de posición: 0" << endl;
    cout << "Promedio académico" << endl;
    return 0;
}
```


159 Laboratorio: Arreglos y Vectores

Del registro de notas al análisis de resultados

160 Laboratorio: Analizador de notas

Meta del laboratorio

En aproximadamente **30 minutos** desarrollarás un programa en C++ utilizando un **arreglo de 5 notas**.

Esta vez el programa **no será construido paso a paso por el docente**.

La meta es comprobar que ya puedes transformar por tu cuenta la idea:

casilla → **índice** → **valor** → **recorrido** → **procesamiento**

161 1. Situación

Un docente necesita un pequeño programa para registrar las notas de **5 estudiantes** y obtener información básica del grupo.

El programa deberá almacenar todas las notas en un **arreglo** y utilizar ciclos para recorrerlo.

! Regla principal

No debes crear variables como:

```
nota1, nota2, nota3, nota4, nota5
```

Las cinco notas deben almacenarse en **un solo arreglo**.

162 2. Requerimiento

Construye un programa en C++ que permita:

1. Crear un arreglo para almacenar **5 notas**.
2. Solicitar al usuario las cinco notas.
3. Mostrar todas las notas almacenadas junto con su **índice**.
4. Calcular la **suma** de las notas.
5. Calcular el **promedio** del grupo.
6. Identificar la **nota mayor**.
7. Identificar la **nota menor**.
8. Contar cuántos estudiantes **aprobaron**.
9. Contar cuántos estudiantes **reprobaron**.

Considere que:

```
Aprobado    → nota >= 61  
Reprobado   → nota < 61
```


163 3. Ejemplo de funcionamiento

El resultado de tu programa podría verse aproximadamente así:

```
=====
          ANALIZADOR DE NOTAS
=====

Ingrese nota [0]: 75
Ingrese nota [1]: 60
Ingrese nota [2]: 90
Ingrese nota [3]: 45
Ingrese nota [4]: 80

=====
          NOTAS REGISTRADAS
=====

Indice 0 -> 75
Indice 1 -> 60
Indice 2 -> 90
Indice 3 -> 45
Indice 4 -> 80

=====
          RESULTADOS
=====

Suma: 350
Promedio: 70
Nota mayor: 90
Nota menor: 45
Aprobados: 3
Reprobados: 2
```

i Importante

El ejemplo muestra **cómo debería comportarse el programa**, pero no indica cómo debes programarlo.

Esa parte te corresponde resolverla.

164 4. Antes de programar

Dedica unos minutos a pensar.

164.1 ¿Qué necesitamos almacenar?

Completa mentalmente o en tu cuaderno:

Elemento	¿Variable o arreglo?	Tipo sugerido
5 notas		
suma		
promedio		
nota mayor		
nota menor		
cantidad de aprobados		
cantidad de reprobados		

164.2 ¿Cuántos recorridos necesitamos?

Piensa:

- ¿Podemos ingresar las notas utilizando un `for`?
- ¿Podemos mostrar las notas con otro `for`?
- ¿Podemos calcular varios resultados durante un mismo recorrido?

 No comiences escribiendo todo el programa

Primero identifica:

Entradas → **Procesos** → **Salidas**

Después programa una parte, compila, prueba y continúa.

165 5. Plan de trabajo sugerido

165.1 Etapa 1 — Registrar las notas

Primero consigue que tu programa:

- declare el arreglo;
- recorra sus posiciones;
- solicite las cinco notas;
- almacene correctamente cada valor.

Cuando funcione, continúa.

165.2 Etapa 2 — Mostrar el arreglo

Recorre nuevamente las posiciones y muestra:

```
Indice 0 -> ...  
Indice 1 -> ...  
Indice 2 -> ...
```

Comprueba que los datos coincidan con lo que ingresaste.

165.3 Etapa 3 — Suma y promedio

Utiliza los valores almacenados para obtener:

- suma;
- promedio.

165.4 Etapa 4 — Mayor y menor

Ahora determina:

- la nota más alta;
- la nota más baja.

💡 Una pista, no la solución

Ya tienes cinco valores almacenados.

Pregúntate:

¿Con qué valor existente del arreglo podría comenzar la variable que guardará el mayor y el menor?

Evita inventar valores arbitrarios.

165.5 Etapa 5 — Aprobados y reprobados

Durante el recorrido analiza cada nota:

```
¿La nota es mayor o igual a 61?
```

Necesitarás llevar dos contadores.

166 6. Tiempo de desarrollo

Tiempo	Actividad
5 min	Analizar el problema
20 min	Programar y probar
5 min	Revisar resultados
30 min	Total

! Durante estos 30 minutos

Intenta resolver el problema utilizando lo aprendido en clase.
Antes de pedir ayuda:

1. lee el mensaje del compilador;
2. identifica en qué parte ocurre;
3. revisa índices, ciclos y condiciones;
4. prueba nuevamente.

167 7. Casos de prueba

Antes de considerar terminado el programa, realiza al menos estas pruebas.

167.1 Prueba 1

```
75
60
90
45
80
```

Comprueba manualmente los resultados.

167.2 Prueba 2

Utiliza cinco notas elegidas por ti.

Antes de ejecutar el programa, calcula en papel:

- suma;
- promedio;
- mayor;
- menor;
- aprobados;
- reprobados.

Luego compara tus resultados con los del programa.

168 8. Preparémonos para explicar

Al finalizar el tiempo de desarrollo, algunos estudiantes presentarán su solución.

No basta con que el programa funcione.

Debes poder explicar:

1. ¿Cómo declaraste el arreglo?
2. ¿Por qué los índices comienzan en 0?
3. ¿Cómo ingresaste las notas?
4. ¿Cómo recorriste el arreglo?
5. ¿Cómo calculaste la suma?
6. ¿Cómo encontraste la nota mayor?
7. ¿Cómo encontraste la nota menor?
8. ¿Cómo contaste aprobados y reprobados?

Regla de la exposición

El código puede compilar perfectamente y aun así existir algo que mejorar.

Durante las exposiciones compararemos **formas de pensar y resolver el problema**, no solamente resultados.

169 9. Reto adicional

Si terminaste antes de los 30 minutos, agrega una búsqueda.

El programa deberá solicitar una nota:

```
Ingresa la nota que desea buscar: 90
```

Luego deberá recorrer el arreglo.

Si la encuentra:

```
Nota encontrada en el indice 2
```

Si no existe:

```
La nota no se encuentra registrada.
```

Pregunta

¿Y qué ocurriría si la misma nota aparece más de una vez?

¿Tu programa mostraría solamente la primera posición o todas las posiciones?

170 10. Alto aquí

! No continúes todavía

La siguiente sección contiene una posible solución.

Primero:

programa → **prueba** → **explica** → **compara**

Después de las exposiciones analizaremos juntos la solución.

171 Solución propuesta

 ☐ Ver solución

Aquí colocamos la explicación de la solución.

```

#include <iostream>
using namespace std;

int main() {

    const int TAM = 5;
    int notas[TAM];

    int suma = 0;
    double promedio;

    int aprobados = 0;
    int reprobados = 0;

    // Ingreso de las notas
    for (int i = 0; i < TAM; i++) {
        cout << "Ingrese nota [" << i << "]: ";
        cin >> notas[i];
    }

    // Inicializamos mayor y menor con un valor real del arreglo
    int mayor = notas[0];
    int menor = notas[0];

    // Procesamiento
    for (int i = 0; i < TAM; i++) {

        suma = suma + notas[i];

        if (notas[i] > mayor) {
            mayor = notas[i];
        }

        if (notas[i] < menor) {
            menor = notas[i];
        }

        if (notas[i] >= 61) {
            aprobados++;
        }
        else {
            reprobados++;
        }
    }

    promedio = suma / static_cast<double>(TAM);

    // Mostrar las notas
    cout << "\n===== " << endl;
    cout << "          NOTAS REGISTRADAS " << endl;
    cout << "===== " << endl;

    for (int i = 0; i < TAM; i++) {

```

172 11. Analicemos la solución

172.1 El arreglo

```
const int TAM = 5;  
int notas[TAM];
```

```
notas[0]  
notas[1]  
notas[2]  
notas[3]  
notas[4]
```

172.2 El ingreso

```
for (int i = 0; i < TAM; i++) {  
    cin >> notas[i];  
}
```

172.3 Mayor y menor

```
int mayor = notas[0];  
int menor = notas[0];
```

```
if (notas[i] > mayor) {  
    mayor = notas[i];  
}
```

```
if (notas[i] < menor) {  
    menor = notas[i];  
}
```

172.4 Aprobados y reprobados

```
if (notas[i] >= 61) {  
    aprobados++;  
}  
else {  
    reprobados++;  
}
```


173 12. Pregunta final

💡 ¿Qué cambió realmente?

Antes necesitábamos pensar en:

```
nota1  
nota2  
nota3  
nota4  
nota5
```

Ahora pensamos en:

```
notas[i]
```

La variable `i` nos permite trabajar con **cualquier posición**.
Esa es una de las ideas más importantes de los arreglos.

174 13. Cierre del laboratorio

```
Arreglo
+
Ciclo
+
Condiciones
```

▪
▪
▪
▪
▪

174.1 Misión cumplida

175 Laboratorio: Modularización del analizador de notas

176 1. El reto de hoy

-
-
-
-
-
-
-
-

Meta del laboratorio

Reestructurar el **Analizador de notas** utilizando:

- arreglos;
- funciones;
- procedimientos;
- parámetros;
- valores de retorno.

El resultado del programa deberá ser el mismo. Lo que cambiará será la **organización del código**.

177 2. Del programa completo a un programa modular

i Nuestro punto de partida

Este programa **ya funciona** y resuelve correctamente el problema.

No vamos a cambiar lo que hace.

Nuestro objetivo será descubrir cómo podemos **dividirlo en módulos más pequeños**, utilizando procedimientos, funciones y parámetros.

```
// código anterior...
#include <iostream>
using namespace std;

int main()
{
    // muchas instrucciones...
    const int TAM = 5;
    int notas[TAM];
    int suma = 0;
    double promedio;
    int aprobados = 0;
    int reprobados = 0;

    // Ingreso de las notas
    cout << "\n===== " << endl;
    cout << " RESULTADOS " << endl;
    cout << "===== " << endl;
    for (int i = 0; i < TAM; i++)
    {
        cout << "Ingrese nota [" << (1 + i) << "]: ";
        cin >> notas[i];
    }
}
```

```

// Inicializamos mayor y menor con un valor real del arreglo
int mayor = notas[0];
int menor = notas[0];

// Procesamiento
for (int i = 0; i < TAM; i++)
{
    suma = suma + notas[i];
    if (notas[i] > mayor) mayor = notas[i];
    if (notas[i] < menor) menor = notas[i];
    if (notas[i] >= 61) aprobados++;
    else reprobados++;
}

promedio = suma / static_cast<double>(TAM);
// Mostrar las notas
cout << "\n===== " << endl;
cout << " NOTAS REGISTRADAS " << endl;
cout << "===== " << endl;
for (int i = 0; i < TAM; i++)
{
    cout << "Indice " << (1+i)
        << " -> " << notas[i] << endl;
}

// Resultados
cout << "\n===== " << endl;
cout << " RESULTADOS " << endl;
cout << "===== " << endl;
cout << "Suma: " << suma << endl;
cout << "Promedio: " << promedio << endl;
cout << "Nota mayor: " << mayor << endl;
cout << "Nota menor: " << menor << endl;
cout << "Aprobados: " << aprobados << endl;
cout << "Reprobados: " << reprobados << endl;

return 0;
}

```

Ingresar notas
 Mostrar notas
 Calcular promedio
 Buscar mayor
 Buscar menor
 Contar aprobados

Contar reprobados
Mostrar resultados

i Idea central

Modularizar no significa cambiar el algoritmo.

Significa **dividir un problema grande en tareas más pequeñas**, de manera que el programa resulte más fácil de leer, comprender, probar y mantener.

178 3. La estructura que queremos alcanzar

```
int main()
{
    const int TAM = 5;
    int notas[TAM];

    double promedio;
    int mayor;
    int menor;
    int aprobados;
    int reprobados;

    ingresarNotas(notas, TAM);
    mostrarNotas(notas, TAM);

    promedio = calcularPromedio(notas, TAM);
    mayor = buscarMayor(notas, TAM);
    menor = buscarMenor(notas, TAM);
    aprobados = contarAprobados(notas, TAM);
    reprobados = contarReprobados(notas, TAM);

    mostrarResultados(promedio, mayor, menor,
                      aprobados, reprobados);

    return 0;
}
```

! Observa antes de programar

Todavía no sabemos cómo está construida cada función.
Eso es precisamente lo que desarrollaremos juntos.
Por ahora observa algo importante:

¿Puedes comprender qué hace el programa únicamente leyendo `main()` ?

179 4. Resultado esperado

```
51
85
95
64
100
```

```
=====
REGISTRO DE NOTAS
=====
Ingrese nota [1]: 51
Ingrese nota [2]: 85
Ingrese nota [3]: 95
Ingrese nota [4]: 64
Ingrese nota [5]: 100

=====
NOTAS REGISTRADAS
=====
Indice 1 -> 51
Indice 2 -> 85
Indice 3 -> 95
Indice 4 -> 64
Indice 5 -> 100

=====
RESULTADOS
=====
Promedio...: 79
Mayor.....: 100
Menor.....: 51
Aprobaron..: 4
Reprobaron: 1
```

i Importante

Ya conocemos el resultado que debe producir el programa.
El reto de hoy no consiste en descubrir nuevamente el algoritmo, sino en **reorganizarlo utilizando módulos**.

180 5. Antes de escribir código

 No pienses todavía en toda la sintaxis

Primero responde para cada módulo:

1. ¿Qué tarea realiza?
2. ¿Qué información necesita?
3. ¿Debe devolver algún resultado?

Después decidiremos si necesitamos un **procedimiento** o una **función**.

181 6. Procedimientos y funciones

181.1 Procedimiento

```
void
```

181.2 Función

```
calcular algo  
    ↓  
devolver resultado  
    ↓  
guardar resultado en una variable
```

Pregunta para el grupo

Observa:

```
promedio = calcularPromedio(notas, TAM);
```

¿Por qué `calcularPromedio()` parece comportarse diferente de:

```
mostrarNotas(notas, TAM);
```

¿Qué está devolviendo una y qué está haciendo la otra?

182 7. ¿Qué información debe viajar entre los módulos?

```
const int TAM = 5;  
int notas[TAM];
```

```
ingresarNotas(notas, TAM);
```

```
notas → arreglo con las notas  
TAM   → cantidad de elementos
```

! Recuerda

Pasar una variable como parámetro **no la convierte en una variable global**.
La función recibe la información que necesita para realizar su tarea.

183 8. Plan de construcción

183.1 Etapa 1 — `ingresarNotas()`

-
-
-
-
-

183.2 Etapa 2 — `mostrarNotas()`

```
=====
NOTAS REGISTRADAS
=====
Indice 1 -> ...
Indice 2 -> ...
Indice 3 -> ...
Indice 4 -> ...
Indice 5 -> ...
```

 Pregunta

¿`mostrarNotas()` necesita devolver un resultado?
¿O solamente necesita realizar una tarea?

183.3 Etapa 3 — `calcularPromedio()`

- 1.
- 2.
- 3.
- 4.
- 5.

```
promedio = calcularPromedio(notas, TAM);
```

183.4 Etapa 4 — `buscarMayor()`

```
mayor = buscarMayor(notas, TAM);
```

 Pista

Ya resolvimos este problema en el laboratorio anterior.
No inventes un valor inicial para `mayor`.
Utiliza un valor que realmente exista dentro del arreglo.

183.5 Etapa 5 — `buscarMenor()`

```
menor = buscarMenor(notas, TAM);
```


183.6 Etapa 6 — contarAprobados ()

```
Aprobado → nota >= 61
```

```
aprobados = contarAprobados(notas, TAM);
```

183.7 Etapa 7 — contarReprobados ()

```
Reprobado → nota < 61
```

```
reprobados = contarReprobados(notas, TAM);
```

183.8 Etapa 8 — mostrarResultados ()

```
Promedio  
Mayor  
Menor  
Aprobados  
Reprobados
```


184 9. Prototipos: el mapa de nuestros módulos

```
_____ ingresarNotas ( _____ ) ;  
_____ mostrarNotas ( _____ ) ;  
_____ calcularPromedio ( _____ ) ;  
_____ buscarMayor ( _____ ) ;  
_____ buscarMenor ( _____ ) ;  
_____ contarAprobados ( _____ ) ;  
_____ contarReprobados ( _____ ) ;  
_____ mostrarResultados ( _____ ) ;
```

Atención

En una función deben ser coherentes:

PROTOTIPO

↓

LLAMADA

↓

IMPLEMENTACIÓN

El nombre, el tipo de retorno y los parámetros deben corresponder.

185 10. Construyamos y probemos

Construir un módulo

↓

Compilar

↓

Probar

↓

Verificar

↓

Continuar

! Si aparece un error

Antes de modificar código al azar:

1. lee el mensaje del compilador;
2. identifica el nombre de la función involucrada;
3. compara el prototipo;
4. compara la llamada;
5. compara la implementación;
6. verifica los parámetros;
7. vuelve a compilar.

186 11. Caso de prueba

187 12. Preparémonos para explicar

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

Regla de la exposición

No basta con decir:

“La función sirve porque funciona.”

Debes poder explicar:

qué recibe → qué procesa → qué devuelve o qué tarea realiza.

188 13. Compare los dos programas

```
¿Cambió el problema?  
¿Cambió el resultado?  
¿Cambió el algoritmo?  
¿Cambió la organización?
```

188.1 La idea que buscamos descubrir

189 14. Alto aquí

! No abras la solución todavía

La siguiente sección contiene una posible implementación completa.

Primero construye el programa junto con el grupo:

analiza → **modulariza** → **programa** → **compila** → **prueba** → **explica**

Utiliza la solución únicamente después de haber trabajado el ejercicio.

190 15. Solución propuesta

Solución de referencia

En la versión HTML la solución se encuentra oculta y puede desplegarse al finalizar el laboratorio.

En la versión PDF se presenta a continuación como material de consulta posterior.

```
#include <iostream>
using namespace std;

// Prototipos
void ingresarNotas(int notas[], int TAM);
void mostrarNotas(int notas[], int TAM);
double calcularPromedio(int notas[], int TAM);
int buscarMayor(int notas[], int TAM);
int buscarMenor(int notas[], int TAM);
int contarAprobados(int notas[], int TAM);
int contarReprobados(int notas[], int TAM);
void mostrarResultados(double promedio, int mayor, int menor,
                      int aprobados, int reprobados);

int main()
{
    const int TAM = 5;
    int notas[TAM];

    double promedio = 0;
    int mayor, menor;
    int aprobados = 0;
    int reprobados = 0;

    ingresarNotas(notas, TAM);
    mostrarNotas(notas, TAM);

    promedio = calcularPromedio(notas, TAM);
    mayor = buscarMayor(notas, TAM);
    menor = buscarMenor(notas, TAM);
```

```
    aprobados = contarAprobados(notas, TAM);
    reprobados = contarReprobados(notas, TAM);

    mostrarResultados(promedio, mayor, menor,
                      aprobados, reprobados);

    return 0;
}

void ingresarNotas(int notas[], int TAM)
{
    for (int i = 0; i < TAM; i++)
    {
        cout << "Ingrese nota [" << (i + 1) << "]: ";
        cin >> notas[i];
    }
}

void mostrarNotas(int notas[], int TAM)
{
    cout << "\n===== " << endl;
    cout << " NOTAS REGISTRADAS " << endl;
    cout << "===== " << endl;

    for (int i = 0; i < TAM; i++)
    {
        cout << "Indice " << (i + 1)
              << " -> " << notas[i] << endl;
    }
}

double calcularPromedio(int notas[], int TAM)
{
    int suma = 0;

    for (int i = 0; i < TAM; i++)
        suma = suma + notas[i];

    return suma / static_cast<double>(TAM);
}

int buscarMayor(int notas[], int TAM)
{
    int mayor = notas[0];

    for (int i = 0; i < TAM; i++)
        if (notas[i] > mayor)
```



```
        mayor = notas[i];

    return mayor;
}

int buscarMenor(int notas[], int TAM)
{
    int menor = notas[0];

    for (int i = 0; i < TAM; i++)
        if (notas[i] < menor)
            menor = notas[i];

    return menor;
}

int contarAprobados(int notas[], int TAM)
{
    int aprobados = 0;

    for (int i = 0; i < TAM; i++)
        if (notas[i] >= 61)
            aprobados++;

    return aprobados;
}

int contarReprobados(int notas[], int TAM)
{
    int reprobados = 0;

    for (int i = 0; i < TAM; i++)
        if (notas[i] < 61)
            reprobados++;

    return reprobados;
}

void mostrarResultados(double promedio, int mayor, int menor,
                      int aprobados, int reprobados)
{
    cout << "\n===== " << endl;
    cout << " RESULTADOS " << endl;
    cout << "===== " << endl;
    cout << "Promedio..: " << promedio << endl;
    cout << "Mayor.....: " << mayor << endl;
    cout << "Menor.....: " << menor << endl;
```

```
cout << "Aprobaron.: " << aprobados << endl;  
cout << "Reprobaron: " << reprobados << endl;  
}
```

191 16. Analicemos la solución

191.1 Procedimientos

```
void ingresarNotas(...);  
void mostrarNotas(...);  
void mostrarResultados(...);
```

191.2 Funciones

```
double calcularPromedio(...);  
  
int buscarMayor(...);  
int buscarMenor(...);  
int contarAprobados(...);  
int contarReprobados(...);
```

```
promedio = calcularPromedio(notas, TAM);  
mayor = buscarMayor(notas, TAM);
```

191.3 Parámetros

```
int notas[], int TAM
```

```
▪  
▪
```


193 Vectores en C++

194 El reto de hoy

! Pregunta para el grupo

¿Qué ocurriría si **no sabemos de antemano cuántos estudiantes** tendremos?
Hoy pasaremos de un arreglo de tamaño fijo a una estructura que puede crecer durante la ejecución:
vector

195 1. Recordemos el arreglo tradicional

```
const int TAM = 5;  
int notas[TAM];
```

💡 Lo que ya sabemos no se pierde

Seguimos trabajando con:

casilla → **índice** → **valor** → **recorrido** → **procesamiento**

Lo nuevo será administrar una cantidad variable de elementos.

196 2. ¿Por qué necesitamos otra estructura?

```
int notas[5];
```


197 3. ¿Qué es vector?

```
#include <vector>
```

```
vector<int> notas;
```

```
notas = [ ]
```


198 4. Arreglo y vector

! Diferencia fundamental

El arreglo tradicional parte de un tamaño establecido.

El `vector` puede **crecer o reducirse durante la ejecución**.

199 5. push_back(): agregar elementos

```
vector<int> notas;  
  
notas.push_back(51);  
notas.push_back(85);  
notas.push_back(95);
```

Índice	0	1	2
Valor	51	85	95

```
notas.push_back(64);
```


200 6. `size()`: conocer la cantidad

```
cout << notas.size();
```

```
size() = 5  
índices = 0, 1, 2, 3, 4
```

i Recuerda

El tamaño no es el último índice.
El último índice válido es:

```
size() - 1
```


201 7. Acceder y recorrer

```
cout << notas[0];
```

```
for (int i = 0; i < notas.size(); i++)  
{  
    cout << notas[i] << endl;  
}
```


202 8. Agregar datos ingresados por el usuario

```
vector<int> notas;  
int cantidad;  
  
cout << "Cuántas notas desea ingresar: ";  
cin >> cantidad;  
  
for (int i = 0; i < cantidad; i++)  
{  
    int nota;  
  
    cout << "Ingrese nota [" << (i + 1) << "]: ";  
    cin >> nota;  
  
    notas.push_back(nota);  
}
```


203 9. Primera prueba

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> notas;
    int cantidad;

    cout << "Cuántas notas desea ingresar: ";
    cin >> cantidad;

    for (int i = 0; i < cantidad; i++)
    {
        int nota;
        cout << "Ingrese nota [" << (i + 1) << "]: ";
        cin >> nota;
        notas.push_back(nota);
    }

    cout << "\nNotas registradas:" << endl;

    for (int i = 0; i < notas.size(); i++)
    {
        cout << "Índice " << i
              << " -> " << notas[i] << endl;
    }

    return 0;
}
```

! Problemas

Ejecuta el programa con 3, luego 5 y finalmente 8 notas.
¿Qué línea del código tuvimos que modificar entre ejecuciones?
Ninguna.

204 10. Regresemos al Analizador de notas modularizado

```
int main()
{
    const int TAM = 5;
    int notas[TAM];

    ingresarNotas(notas, TAM);
    mostrarNotas(notas, TAM);

    promedio = calcularPromedio(notas, TAM);
    mayor = buscarMayor(notas, TAM);
    menor = buscarMenor(notas, TAM);
    aprobados = contarAprobados(notas, TAM);
    reprobados = contarReprobados(notas, TAM);

    mostrarResultados(promedio, mayor, menor,
                      aprobados, reprobados);
}
```

i ¿Tenemos que comenzar nuevamente?

No.

La modularización permite cambiar la forma de almacenar los datos sin destruir toda la solución.

205 11. El arreglo se convierte en vector

```
const int TAM = 5;  
int notas[TAM];
```

```
vector<int> notas;
```


206 12. Pasar el vector a una función

```
void ingresarNotas (vector<int>& notas)
```

! ¿Qué significa &?

Por ahora leeremos & de esta manera:

La función trabajará con el **vector original**, no con una copia.

Así, los elementos agregados dentro de la función permanecen disponibles en `main()`.
Más adelante profundizaremos en el paso por referencia.

```
void ingresarNotas (vector<int>& notas)
{
    int cantidad;

    cout << "Cuantas notas desea ingresar: ";
    cin >> cantidad;

    for (int i = 0; i < cantidad; i++)
    {
        int nota;

        cout << "Ingrese nota [" << (i + 1) << "]: ";
        cin >> nota;

        notas.push_back(nota);
    }
}
```


207 13. Funciones que solamente consultan

```
const vector<int>& notas
```

```
void mostrarNotas(const vector<int>& notas)
{
    for (int i = 0; i < notas.size(); i++)
    {
        cout << "Indice " << (i + 1)
              << " -> " << notas[i] << endl;
    }
}
```


208 14. Transformemos las funciones

208.1 Promedio

```
double calcularPromedio(const vector<int>& notas)
{
    int suma = 0;

    for (int i = 0; i < notas.size(); i++)
        suma = suma + notas[i];

    return suma / static_cast<double>(notas.size());
}
```

208.2 Mayor

```
int buscarMayor(const vector<int>& notas)
{
    int mayor = notas[0];

    for (int i = 1; i < notas.size(); i++)
        if (notas[i] > mayor)
            mayor = notas[i];

    return mayor;
}
```

208.3 Menor

```
int buscarMenor(const vector<int>& notas)
{
    int menor = notas[0];
```

```
    for (int i = 1; i < notas.size(); i++)
        if (notas[i] < menor)
            menor = notas[i];

    return menor;
}
```

208.4 Aprobados

```
int contarAprobados(const vector<int>& notas)
{
    int aprobados = 0;

    for (int i = 0; i < notas.size(); i++)
        if (notas[i] >= 61)
            aprobados++;

    return aprobados;
}
```

208.5 Reprobados

```
int contarReprobados(const vector<int>& notas)
{
    int reprobados = 0;

    for (int i = 0; i < notas.size(); i++)
        if (notas[i] < 61)
            reprobados++;

    return reprobados;
}
```

209 15. El nuevo main ()

```
int main()
{
    vector<int> notas;

    double promedio;
    int mayor, menor;
    int aprobados, reprobados;

    ingresarNotas(notas);
    mostrarNotas(notas);

    promedio = calcularPromedio(notas);
    mayor = buscarMayor(notas);
    menor = buscarMenor(notas);
    aprobados = contarAprobados(notas);
    reprobados = contarReprobados(notas);

    mostrarResultados(promedio, mayor, menor,
                      aprobados, reprobados);

    return 0;
}
```

209.1 Observa

```
ingresarNotas(notas, TAM);
```

```
ingresarNotas(notas);
```


210 16. Resultado esperado

```
Cuantas notas desea ingresar: 5
```

```
Ingrese nota [1]: 51
Ingrese nota [2]: 85
Ingrese nota [3]: 95
Ingrese nota [4]: 64
Ingrese nota [5]: 100
```

```
=====
NOTAS REGISTRADAS
=====
```

```
Indice 1 -> 51
Indice 2 -> 85
Indice 3 -> 95
Indice 4 -> 64
Indice 5 -> 100
```

```
=====
RESULTADOS
=====
```

```
Promedio..: 79
Mayor.....: 100
Menor.....: 51
Aprobaron.: 4
Reprobaron: 1
```


211 17. Solución completa

```
#include <iostream>
#include <vector>
using namespace std;

void ingresarNotas(vector<int>& notas);
void mostrarNotas(const vector<int>& notas);
double calcularPromedio(const vector<int>& notas);
int buscarMayor(const vector<int>& notas);
int buscarMenor(const vector<int>& notas);
int contarAprobados(const vector<int>& notas);
int contarReprobados(const vector<int>& notas);
void mostrarResultados(double promedio, int mayor, int menor,
                      int aprobados, int reprobados);

int main()
{
    vector<int> notas;

    double promedio;
    int mayor, menor;
    int aprobados, reprobados;

    ingresarNotas(notas);
    mostrarNotas(notas);

    promedio = calcularPromedio(notas);
    mayor = buscarMayor(notas);
    menor = buscarMenor(notas);
    aprobados = contarAprobados(notas);
    reprobados = contarReprobados(notas);

    mostrarResultados(promedio, mayor, menor,
                      aprobados, reprobados);

    return 0;
```

```
}

void ingresarNotas(vector<int>& notas)
{
    int cantidad;
    cout << "Cuántas notas desea ingresar: ";
    cin >> cantidad;

    for (int i = 0; i < cantidad; i++)
    {
        int nota;
        cout << "Ingrese nota [" << (i + 1) << "]: ";
        cin >> nota;
        notas.push_back(nota);
    }
}

void mostrarNotas(const vector<int>& notas)
{
    cout << "\n===== " << endl;
    cout << " NOTAS REGISTRADAS " << endl;
    cout << "===== " << endl;

    for (int i = 0; i < notas.size(); i++)
        cout << "Índice " << (i + 1)
            << " -> " << notas[i] << endl;
}

double calcularPromedio(const vector<int>& notas)
{
    int suma = 0;

    for (int i = 0; i < notas.size(); i++)
        suma += notas[i];

    return suma / static_cast<double>(notas.size());
}

int buscarMayor(const vector<int>& notas)
{
    int mayor = notas[0];

    for (int i = 1; i < notas.size(); i++)
        if (notas[i] > mayor)
            mayor = notas[i];

    return mayor;
}
```

```
}

int buscarMenor(const vector<int>& notas)
{
    int menor = notas[0];

    for (int i = 1; i < notas.size(); i++)
        if (notas[i] < menor)
            menor = notas[i];

    return menor;
}

int contarAprobados(const vector<int>& notas)
{
    int aprobados = 0;

    for (int i = 0; i < notas.size(); i++)
        if (notas[i] >= 61)
            aprobados++;

    return aprobados;
}

int contarReprobados(const vector<int>& notas)
{
    int reprobados = 0;

    for (int i = 0; i < notas.size(); i++)
        if (notas[i] < 61)
            reprobados++;

    return reprobados;
}

void mostrarResultados(double promedio, int mayor, int menor,
                      int aprobados, int reprobados)
{
    cout << "\n===== " << endl;
    cout << " RESULTADOS " << endl;
    cout << "===== " << endl;
    cout << "Promedio..: " << promedio << endl;
    cout << "Mayor.....: " << mayor << endl;
    cout << "Menor.....: " << menor << endl;
    cout << "Aprobaron.: " << aprobados << endl;
    cout << "Reprobaron: " << reprobados << endl;
}
```


212 18. La evolución de nuestra solución

```
Variables individuales
      ↓
Arreglo de tamaño fijo
      ↓
Arreglo + ciclos
      ↓
Arreglo + funciones
      ↓
Vector + funciones
      ↓
Cantidad dinámica de datos
```


213 19. Cinco instrucciones para recordar

```
vector<int> notas;           // declarar
notas.push_back(75);        // agregar
notas.size();               // cantidad
notas[i];                   // acceder
for (int i = 0; i < notas.size(); i++)
{
    // recorrer
}
```


214 20. Mini reto

Cantidad de notas registradas: 5

215 21. Tarea para el viernes

215.1 Sistema de registro de temperaturas

```
vector<double>
```

215.1.1 Requerimientos

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

! Condición

No se permite colocar toda la lógica dentro de `main()`.
Utiliza como mínimo:

```
ingresarTemperaturas()  
mostrarTemperaturas()  
calcularPromedio()  
buscarMayor()  
buscarMenor()  
contarMayoresPromedio()  
contarMenoresPromedio()  
mostrarResultados()
```

215.2 Entregables

- 1.
- 2.
- 3.
- 4.

-
-
-
-

216 22. Cierre

216.1 De tamaño fijo a tamaño dinámico

217 Vectores paralelos: del control de estudiantes a las matrices

218 Antes de comenzar

```
float notas[5] = {85, 62, 94, 71, 78};
```

-
-
-

Idea central de la clase

No aprenderemos una estructura completamente nueva. Utilizaremos varios vectores y descubriremos que **la posición o índice puede relacionar información almacenada en estructuras diferentes.**

219 **Objetivos de aprendizaje**

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

220 El problema: control de estudiantes

▪
▪
▪

221 Primera solución: vectores paralelos

```
string carnet[5];  
string nombre[5];  
float nota[5];
```

Índice	0	1	2	3	4
Carnet	2026001	2026002	2026003	2026004	2026005
Nombre	Ana	Carlos	María	José	Andrea
Nota	85	62	94	71	78

```
carnet[2] // 2026003  
nombre[2] // María  
nota[2] // 94
```

! Regla fundamental

Cuando trabajamos con vectores paralelos, **el índice establece la relación entre los datos**. Si modificamos el orden de uno de los vectores sin modificar los demás, perdemos esa relación.

222 Paso 1. Registrar estudiantes

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int CANTIDAD = 5;

    string carnet[CANTIDAD];
    string nombre[CANTIDAD];
    float nota[CANTIDAD];

    for (int i = 0; i < CANTIDAD; i++) {
        cout << "\nEstudiante " << i + 1 << endl;

        cout << "Carnet: ";
        cin >> carnet[i];

        cout << "Nombre: ";
        cin >> nombre[i];

        cout << "Nota: ";
        cin >> nota[i];
    }

    return 0;
}
```

222.1 ¿Qué está ocurriendo?

```
carnet[0]  
nombre[0]  
nota[0]
```

```
carnet[1]  
nombre[1]  
nota[1]
```

223 Paso 2. Mostrar el listado completo

```
cout << "\nLISTADO DE ESTUDIANTES\n";
cout << "-----\n";

for (int i = 0; i < CANTIDAD; i++) {
    cout << carnet[i] << " "
        << nombre[i] << " "
        << nota[i] << endl;
}
```


224 Paso 3. Buscar un estudiante por carnet

- 1.
- 2.
- 3.
- 4.
- 5.

```
string carnetBuscado;
int posicion = -1;

cout << "\nIngrese el carnet que desea buscar: ";
cin >> carnetBuscado;

for (int i = 0; i < CANTIDAD; i++) {
    if (carnet[i] == carnetBuscado) {
        posicion = i;
        break;
    }
}

if (posicion != -1) {
    cout << "\nEstudiante encontrado\n";
    cout << "Carnet: " << carnet[posicion] << endl;
    cout << "Nombre: " << nombre[posicion] << endl;
    cout << "Nota: " << nota[posicion] << endl;
} else {
    cout << "\nEl carnet no fue encontrado.\n";
}
```

224.1 El detalle más importante

```
carnet[i]
```

```
nombre[posicion]  
nota[posicion]
```

i Pregunta para discutir en clase

Si encontramos el carnet en la posición 3, ¿en qué posición debemos buscar el nombre y la nota del estudiante?

225 Paso 4. Mostrar estudiantes aprobados

```
cout << "\nESTUDIANTES APROBADOS\n";

for (int i = 0; i < CANTIDAD; i++) {
    if (nota[i] >= 61) {
        cout << carnet[i] << " "
              << nombre[i] << " "
              << nota[i] << endl;
    }
}
```


226 Paso 5. Calcular el promedio del grupo

```
float suma = 0;
float promedio;

for (int i = 0; i < CANTIDAD; i++) {
    suma = suma + nota[i];
}

promedio = suma / CANTIDAD;

cout << "\nPromedio del grupo: " << promedio << endl;
```


227 Paso 6. Encontrar al estudiante con la nota más alta

```
int posicionMayor = 0;

for (int i = 1; i < CANTIDAD; i++) {
    if (nota[i] > nota[posicionMayor]) {
        posicionMayor = i;
    }
}
```

```
cout << "\nMEJOR RESULTADO DEL GRUPO\n";
cout << "Carnet: " << carnet[posicionMayor] << endl;
cout << "Nombre: " << nombre[posicionMayor] << endl;
cout << "Nota: " << nota[posicionMayor] << endl;
```

Comprensión antes que memorización

No necesitamos buscar nuevamente el nombre del estudiante. Si conocemos la posición de la nota mayor, esa misma posición identifica sus datos en los demás vectores.

228 Programa integrador

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    const int CANTIDAD = 5;

    string carnet[CANTIDAD];
    string nombre[CANTIDAD];
    float nota[CANTIDAD];

    // 1. Registro de estudiantes
    for (int i = 0; i < CANTIDAD; i++) {
        cout << "\nEstudiante " << i + 1 << endl;

        cout << "Carnet: ";
        cin >> carnet[i];

        cout << "Nombre: ";
        cin >> nombre[i];

        cout << "Nota: ";
        cin >> nota[i];
    }

    // 2. Mostrar estudiantes
    cout << "\n\nLISTADO DE ESTUDIANTES\n";
    cout << "-----\n";

    for (int i = 0; i < CANTIDAD; i++) {
        cout << carnet[i] << " "
            << nombre[i] << " "
            << nota[i] << endl;
    }
}
```

```
// 3. Buscar estudiante por carnet
string carnetBuscado;
int posicion = -1;

cout << "\nIngrese carnet a buscar: ";
cin >> carnetBuscado;

for (int i = 0; i < CANTIDAD; i++) {
    if (carnet[i] == carnetBuscado) {
        posicion = i;
        break;
    }
}

if (posicion != -1) {
    cout << "Nombre: " << nombre[posicion] << endl;
    cout << "Nota: " << nota[posicion] << endl;
} else {
    cout << "Carnet no encontrado.\n";
}

// 4. Estudiantes aprobados
cout << "\nESTUDIANTES APROBADOS\n";

for (int i = 0; i < CANTIDAD; i++) {
    if (nota[i] >= 61) {
        cout << carnet[i] << " "
            << nombre[i] << " "
            << nota[i] << endl;
    }
}

// 5. Promedio
float suma = 0;

for (int i = 0; i < CANTIDAD; i++) {
    suma += nota[i];
}

float promedio = suma / CANTIDAD;
cout << "\nPromedio del grupo: " << promedio << endl;

// 6. Nota más alta
int posicionMayor = 0;

for (int i = 1; i < CANTIDAD; i++) {
    if (nota[i] > nota[posicionMayor]) {
```



```
        posicionMayor = i;
    }
}

cout << "\nMEJOR RESULTADO\n";
cout << "Carnet: " << carnet[posicionMayor] << endl;
cout << "Nombre: " << nombre[posicionMayor] << endl;
cout << "Nota: " << nota[posicionMayor] << endl;

return 0;
}
```


229 Detengámonos a pensar

```
string carnet[5];  
string nombre[5];  
float nota[5];
```

```
float notaAlgoritmos[5];  
float notaMatematica[5];  
float notaFisica[5];
```

```
float notaProgramacion[5];
```

```
float notaEstadistica[5];
```


230 El problema cambia de forma

- 1.
- 2.

```
nota[2]
```

```
nota[estudiante][curso]
```

```
nota[2][0]
```

 Aquí nace una nueva necesidad

Nuestros vectores nos han servido muy bien, pero el problema ahora tiene **dos dimensiones**: estudiantes y cursos.

Necesitamos una estructura capaz de organizar información utilizando **filas y columnas**.

231 De una dimensión a dos dimensiones

Vector de notas

	0	1	2	3	4
nota	85	62	94	71	78

una dimensión

		CURSOS		
		0	1	2
		Algorit.	Matem.	Física
EST.	0	85	78	90
	1	62	71	68
	2	94	89	92
	3	71	65	77
	4	78	84	80

[fila][columna]

232 La puerta hacia las matrices

```
float notas[5][3];
```

```
float nota[5];           // una dimensión  
float notas[5][3];      // dos dimensiones
```

```
notas[2][1]
```

▪
▪

i No adelantemos el laboratorio

Por ahora nuestro objetivo no es dominar las matrices. Es comprender **por qué las necesitamos**.

En el próximo laboratorio aprenderemos a declararlas, llenarlas, recorrerlas y procesarlas utilizando ciclos anidados.

233 Conexión con los vectores paralelos

```
string carnet[5];  
string nombre[5];  
float notas[5][3];
```

VECTOR		MATRIZ DE NOTAS		
Índice	Nombre	Algoritmos	Matemática	Física
0	Ana	85	78	90
1	Carlos	62	71	68
2	María	94	89	92
3	José	71	65	77
4	Andrea	78	84	80

```
nombre[2]  
notas[2][0]  
notas[2][1]  
notas[2][2]
```


234 Actividad de comprensión

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

235 Reto final de la clase

```
float curso1[40];  
float curso2[40];  
float curso3[40];  
// ...  
float curso10[40];
```

```
float notas[40][10];
```


236 Lo que hemos construido

```
Variable
|
v
Arreglo / Vector
|
v
Varios vectores relacionados
|
v
Vectores paralelos
|
v
Necesidad de representar filas y columnas
|
v
MATRICES
```

Idea para llevarse de esta clase

Una matriz no aparece porque queramos aprender una nueva sintaxis.

Aparece porque algunos problemas necesitan organizar datos en más de una dimensión.

237 Próxima clase: matrices

▪
▪
▪
▪
▪
▪
▪

238 Cierre

Laboratorio de Integración I

Del problema al programa

! Regla principal del laboratorio

No comenzamos escribiendo código.

Antes de abrir el compilador debemos ser capaces de explicar con nuestras propias palabras:

1. ¿Qué nos están pidiendo?
2. ¿Qué información conocemos o debemos solicitar?
3. ¿Qué resultados necesitamos obtener?
4. ¿Qué operaciones o decisiones debemos realizar?
5. ¿Qué estructura de programación podría ayudarnos?
6. ¿Cómo comprobaremos que nuestra solución funciona?

1. La ruta que seguiremos

1.1 Antes de programar

Consejo del Ingeniero

Si todavía no puede explicar la solución sin utilizar C++, **todavía no es momento de programar**.

Primero resolvemos el problema nosotros. Después le enseñamos a la computadora cómo hacerlo.

Ejercicio 1. Empecemos por lo esencial: variables y operaciones

El requerimiento

-
-
-

-
-

Antes de programar

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Análisis

-
-
-

-
-

Pensemos en las variables

Pseudocódigo

```
Inicio
    Definir producto
    Definir precio, subtotal
    Definir cantidad

    Leer producto
    Leer precio
    Leer cantidad

    subtotal = precio * cantidad

    Mostrar producto
    Mostrar subtotal
Fin
```

Ahora sí: C++

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {  
    string producto;  
    double precio, subtotal;  
    int cantidad;  
  
    cout << "Ingrese el nombre del producto: ";  
    cin >> producto;  
  
    cout << "Ingrese el precio: Q";  
    cin >> precio;  
  
    cout << "Ingrese la cantidad: ";  
    cin >> cantidad;  
  
    subtotal = precio * cantidad;  
  
    cout << "\nProducto: " << producto << endl;  
    cout << "Subtotal: Q" << subtotal << endl;  
  
    return 0;  
}
```

Caso de prueba

i ¿Qué recuperamos aquí?

Variables, tipos de datos, entrada, proceso, salida, operadores aritméticos y asignación.

Ejercicio 2. Ahora debemos tomar una decisión

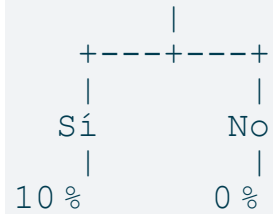
El requerimiento

▪
▪
▪

Analicemos antes de escribir código

¿Qué cambió respecto al ejercicio anterior?

```
¿subtotal > 500?
```



Antes de continuar

Entradas:

Procesos:

Condición:

Salidas:

Pseudocódigo esperado

Reto de programación

```
if (...) {
    ...
} else {
    ...
}
```

Casos de prueba obligatorios

 Observe el límite

El requerimiento dice **mayor a Q500**, no “mayor o igual”.
Por eso Q500 **no** recibe descuento.

Ejercicio 3. Una decisión puede tener varios caminos

El requerimiento

Nota inválida

No programe todavía

- 1.
- 2.
- 3.
- 4.
- 5.

Construya la lógica

```
Si la nota es inválida
```

```
Sino si
```

```
Mostrar "Excelente"
```

```
Sino si
```

```
Mostrar "Muy bueno"
```

```
Sino si
```

```
Mostrar "Bueno"
```

```
Sino si
```

```
Mostrar "Aprobado"
```

```
Sino
```

Pruebas mínimas

```
-5  
0  
60  
61  
75  
89  
90  
100  
105
```

Pregunta clave

¿Por qué probamos precisamente valores cercanos a los límites?
Porque muchos errores de lógica aparecen en los extremos de los rangos.

Ejercicio 4. Cuando una tarea debe repetirse

El requerimiento

- 1.
- 2.
- 3.
- 4.
- 5.

Analicemos el problema

```
Leer nota1  
Leer nota2  
Leer nota3
```

Preguntas de análisis

-
-
-
-
-
-

Simulación manual

```
N = 4
```

```
Notas:
```

```
70
```

```
80
```

```
90
```

```
100
```

Primera solución

Reto adicional

i ¿Qué estamos integrando?

Variables + acumuladores + contadores + decisiones + ciclos.

Ejercicio 5. Construyamos un menú

El requerimiento

```
=====
                CALCULADORA
=====
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Salir
=====
Seleccione una opción:
```

Aquí aparecen dos problemas diferentes

Antes de programar

Diseño general

```
Inicio
Repetir
  Mostrar menú
  Leer opción

  Según opción
    1 -> sumar
    2 -> restar
    3 -> multiplicar
    4 -> dividir
    5 -> salir
    otro -> opción inválida

Mientras opción sea diferente de 5
Fin
```

Pregunta importante

Reto avanzado

6. Potencia

Ejercicio 6. Usemos `for` cuando conozcamos las repeticiones

El requerimiento

-
-
-
-

Pensemos antes de programar

```
7 días
```

```
Para día = 1 hasta 7  
    ...  
Fin Para
```

Identifique las variables necesarias

Pregunta de razonamiento

Reto

Reto avanzado

■

! Alto pedagógico

Esta última pregunta es importante.

A veces comprendemos perfectamente un problema, pero descubrimos que la información disponible en ese momento **no es suficiente** para resolver una parte.

Esta necesidad nos prepara para trabajar posteriormente con **arreglos y vectores**.

Ejercicio 7. Dividamos el problema: procedimientos

El requerimiento

```
void mostrarMenu() {  
    cout << "1. Sumar" << endl;  
    cout << "2. Restar" << endl;  
    cout << "3. Multiplicar" << endl;  
    cout << "4. Dividir" << endl;  
    cout << "5. Salir" << endl;  
}
```

La pregunta cambia

```
mostrarMenu()  
sumar()  
restar()  
multiplicar()  
dividir()
```

Analicemos responsabilidades

Primer objetivo

```
void sumar() {  
    double a, b;  
  
    cout << "Ingrese el primer número: ";  
    cin >> a;  
  
    cout << "Ingrese el segundo número: ";  
    cin >> b;  
  
    cout << "Resultado: " << a + b << endl;  
}
```

```
int main() {  
  
    int opcion;  
  
    do {  
        mostrarMenu();  
        cin >> opcion;  
    }
```

```
        switch (opcion) {
            case 1:
                sumar();
                break;

            // continuar...

        }

    } while (opcion != 5);

    return 0;
}
```

Detengámonos aquí

```
mostrar menú
leer opción
ejecutar operación
repetir
```

Idea fundamental

Un procedimiento nos permite **darle nombre a una tarea**.
En lugar de pensar en veinte instrucciones, podemos pensar:

“Ahora quiero sumar.”

y delegar esa responsabilidad a `sumar()`.

Preguntas de comprensión

- 1.
- 2.
- 3.
- 4.
- 5.

Ejercicio 8. Funciones y parámetros: piezas reutilizables

El problema de nuestra solución anterior

```
void sumar() {  
    double a, b;  
  
    cout << "Ingrese el primer número: ";  
    cin >> a;  
  
    cout << "Ingrese el segundo número: ";  
    cin >> b;  
  
    cout << "Resultado: " << a + b << endl;  
}
```

- 1.
- 2.
- 3.

8.1 De procedimiento cerrado a procedimiento reutilizable

```
void sumar(double a, double b) {  
    cout << "Resultado: " << a + b << endl;  
}
```

```
sumar(10, 20);
```

```
double x = 5;  
double y = 8;  
  
sumar(x, y);
```

¿Qué cambió?

i Parámetros

Los parámetros permiten enviar información a un procedimiento o función. Podemos imaginarlo así:

```
main()  
|  
|  a = 10  
|  b = 20  
v  
sumar(a, b)
```

El procedimiento recibe los datos y realiza su tarea.

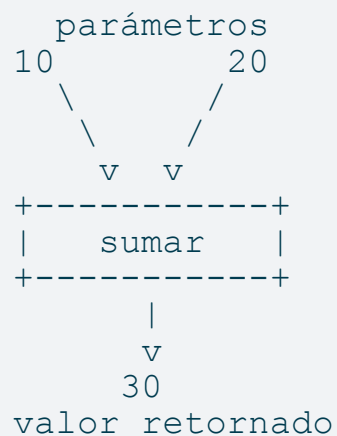
8.2 Ahora necesitamos devolver un resultado

```
resultado = sumar(a, b);
```

```
double sumar(double a, double b) {  
    return a + b;  
}
```

```
double resultado;  
resultado = sumar(10, 20);  
cout << resultado << endl;
```

8.3 Pensemos en la función como una pequeña máquina



8.4 El reto integrador

```
double sumar(double a, double b);  
double restar(double a, double b);  
double multiplicar(double a, double b);  
double dividir(double a, double b);  
void mostrarMenu();
```

Organización propuesta

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

Estructura inicial

```
#include <iostream>  
using namespace std;  
  
void mostrarMenu();  
double sumar(double a, double b);  
double restar(double a, double b);  
double multiplicar(double a, double b);  
double dividir(double a, double b);  
  
int main() {  
    // Diseñe aquí el programa principal.  
    return 0;  
}  
  
void mostrarMenu() {
```

```
    // Mostrar opciones.
}

double sumar(double a, double b) {
    return a + b;
}

double restar(double a, double b) {
    // completar
}

double multiplicar(double a, double b) {
    // completar
}

double dividir(double a, double b) {
    // completar
}
```

 Una función no elimina el análisis

Crear funciones no significa fragmentar el código porque sí.

Cada función debe tener una responsabilidad clara.

Pregúntese:

¿Qué recibe? → ¿Qué hace? → ¿Qué devuelve?

8.5 Segundo reto: función para determinar el mayor

```
int mayor(int a, int b);
```

```
int resultado = mayor(15, 8);
```

Preguntas

-
-
-
-
-

8.6 Tercer reto: función para determinar si un número es par

```
bool esPar(int numero);
```

```
esPar(8)    -> true  
esPar(7)    -> false
```

```
numero % 2
```

Pregunta importante

8.7 Cuarto reto: combine funciones

```
bool esPar(int numero);
```

-

■

```
variables
  +
ciclo
  +
función
  +
parámetros
  +
decisión
```

Antes de escribir código

```
main()
|
| lee número
|
+----> esPar(numero)
          |
          +----> true / false
|
| actualiza contador
|
+----> siguiente número
```

! Este es el objetivo de la clase

No memorizar una función.

Comprender que un programa complejo puede construirse combinando **pequeñas soluciones que ya sabemos resolver**.

Desafío final. Sistema de evaluación de hábitos e IMC

Situación

! Importante

Este ejercicio tiene fines académicos para practicar análisis y programación.

El IMC es un indicador de referencia y **no constituye por sí solo un diagnóstico médico**. Las recomendaciones del programa deben redactarse como orientaciones generales de hábitos saludables, no como tratamientos médicos personalizados.

El requerimiento

-
-
-
-
-

¿Cuántos días por semana consume comida rápida (pizza, hamburguesas, pollo frito, etc.)?

¿Cuántos días por semana consume golosinas, frituras, pastelitos u otras chucherías?

¿Cuántos días por semana consume bebidas azucaradas?
¿Cuántas bebidas energizantes consume por semana?
¿Cuántos días por semana consume bebidas alcohólicas?
¿Cuántos días por semana consume frutas o verduras?
¿Cuántos días por semana realiza al menos
30 minutos de actividad física?

Cálculo del IMC

Validación necesaria

Como esta clasificación corresponde a adultos, el programa debe verificar la edad. Si la persona es menor de 18 años, deberá indicar que esta clasificación no será aplicada y evitar emitir una clasificación de adulto.

Pero el IMC no es todo el problema

1. Resultado del IMC
2. Hábitos reportados

Recomendaciones que puede generar el programa

Si consume comida rápida con mucha frecuencia:
recomendar reducir su frecuencia y preferir preparaciones caseras con alimentos variados.

Si consume muchas golosinas, frituras o bebidas azucaradas:
recomendar disminuir su consumo y preferir agua y alimentos menos procesados.

Si consume bebidas energizantes:
recomendar limitar o evitar su consumo habitual.

Si reporta consumo frecuente de alcohol:
mostrar una recomendación preventiva para reducirlo o evitarlo.

Si consume pocas frutas o verduras:
recomendar aumentar su presencia en las comidas.

Si realiza poca actividad física:
recomendar incorporar actividad física regularmente, de acuerdo con sus posibilidades y condición personal.

Ejemplo del reporte esperado

```
=====
                        REPORTE DE EVALUACIÓN
=====

Nombre: Ana López
Edad: 22 años
Peso: 68 kg
Estatura: 1.65 m

IMC: 24.98
Clasificación: Peso saludable

-----
HÁBITOS REPORTADOS
```

Comida rápida: 4 días/semana
Chucherías: 5 días/semana
Bebidas azucaradas: 3 días/semana
Bebidas energizantes: 2 por semana
Alcohol: 1 día/semana
Frutas o verduras: 2 días/semana
Actividad física: 1 día/semana

RECOMENDACIONES GENERALES

- Su IMC se encuentra en el rango de referencia considerado saludable para adultos.
- Considere disminuir la frecuencia de comida rápida.
- Reduzca golosinas, frituras y bebidas azucaradas.
- Prefiera agua como bebida habitual.
- Aumente gradualmente el consumo de frutas y verduras.
- Procure incorporar actividad física con regularidad.

=====

Antes de programar: analice

A. Comprensión del requerimiento

B. Entradas

C. Validaciones

-
-
-
-

D. Procesos

```
calcular IMC  
clasificar IMC  
analizar hábitos  
generar recomendaciones
```

E. Decisiones

```
¿La persona es adulta?  
¿En qué rango se encuentra el IMC?  
¿Consume comida rápida frecuentemente?  
¿Consume bebidas energizantes?  
¿Consume suficientes frutas o verduras?  
¿Realiza actividad física regularmente?
```

F. Repeticiones

G. Descomposición del problema

```
double calcularIMC(double peso, double estatura);  
string clasificarIMC(double imc);  
bool frecuenciaAlta(int dias);  
void mostrarRecomendaciones(...);
```

Requisitos mínimos de programación

-
-
-
-
-
-
-
-
-
-
-
-
-
-

💡 El objetivo no es hacer el programa más largo

El objetivo es demostrar que puede tomar un requerimiento, analizarlo, dividirlo y construir una solución utilizando las herramientas aprendidas durante el curso.

Entregables

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

! No se calificará únicamente el código

El propósito del desafío es evaluar el proceso completo:

comprender → **analizar** → **diseñar** → **programar** → **probar** → **explicar**

Un programa que funcione pero que el estudiante no pueda explicar no demuestra completamente el aprendizaje esperado.

Preparación para la exposición

```
¿Por qué utilizó esa variable?  
¿Por qué utilizó un if en esa parte?  
¿Qué ocurriría si cambia esta condición?  
¿Por qué eligió while, do-while o for?  
¿Qué parámetros recibe esta función?  
¿Qué devuelve?  
¿Qué ocurriría si eliminamos esta función?  
¿Cómo validó los datos?  
¿Qué caso de prueba utilizó para un valor límite?  
¿Cómo agregaría una nueva recomendación?
```

Importante

No se trata de memorizar el código para la exposición.
Debe ser capaz de **explicar la lógica de su propia solución**.

¿Qué hemos recuperado?

La idea que debe quedar completa

```
Tengo un problema
↓
¿Qué me están pidiendo?
↓
¿Qué sé?
↓
¿Qué necesito?
↓
¿Qué debo calcular o decidir?
↓
¿Necesito repetir?
↓
¿Puedo dividir el problema?
↓
¿Qué funciones necesito?
↓
Escribo el algoritmo
↓
Lo convierto en C++
↓
Lo pruebo
```

💡 Consejo final del Ingeniero

No intente recordar “qué código llevaba aquel ejercicio”.

Pregúntese:

¿Qué necesita hacer este problema?

Cuando comprende la necesidad, puede elegir la herramienta adecuada.

La programación deja de ser una colección de instrucciones y comienza a convertirse en una forma de resolver problemas.

Semáforo de aprendizaje

Próximo paso: arreglos y vectores

```
double nota;  
double venta;  
int numero;
```

```
nota 1  
nota 2  
nota 3  
...  
nota N
```

! Puente hacia la siguiente clase

Primero aprendimos a guardar **un dato**.

Ahora aprenderemos a guardar y procesar **muchos datos relacionados**.

Ese será nuestro siguiente paso.

239 Matrices y arreglos multidimensionales en C++

240 Antes de comenzar

```
string carnet[35];  
string nombre[35];  
float nota[35];
```

Posición	carnet	nombre	nota
0	2026001	Ana	85
1	2026002	Carlos	62
2	2026003	María	94

```
nombre[2]
```

```
nota[2]
```

Idea importante

Un vector nos permite organizar una **lista** de elementos.

Pero, ¿qué sucede cuando necesitamos organizar información utilizando **filas y columnas**?

241 El nuevo requerimiento

```
float algoritmos[35];  
float matematica[35];  
float fisica[35];
```

```
float algoritmos[35];  
float matematica[35];  
float fisica[35];  
float estadistica[35];  
float programacion[35];  
// ...
```


242 Pensemos primero en Excel

▪
▪
▪
▪

```
fila = 2  
columna = 1
```

```
notas[2][1]
```

i El cambio fundamental

En un vector utilizábamos **un índice**:

```
nota[i]
```

En una matriz utilizaremos **dos índices**:

```
nota[i][j]
```

El primero identifica la **fila** y el segundo la **columna**.

243 ¿Qué es una matriz?

```
int matriz[3][3];
```

▪
▪
▪

	columnas		
	0	1	2
fila 0			
fila 1			
fila 2			

```
matriz[filas][columnas]
```


244 Comparación con un vector

```
int numeros[5];
```

```
[ 10 ] [ 20 ] [ 30 ] [ 40 ] [ 50 ]  
  0      1      2      3      4
```

```
numeros[2]
```

```
int numeros[3][3];
```

	0	1	2
0	5	8	6
1	7	9	4
2	3	2	1

```
numeros[1][1]
```


245 Recordemos: los índices comienzan en cero

```
int matriz[3][3];
```

```
Filas:    0, 1, 2  
Columnas: 0, 1, 2
```

```
matriz[0][0]
```

```
matriz[2][2]
```

Cuidado

En una matriz 3 x 3 no existe:

```
matriz[3][3]
```

porque los índices comienzan en cero.

246 Inicialización de una matriz

```
int matriz[3][3] = {  
    {5, 8, 6},  
    {7, 9, 4},  
    {3, 2, 1}  
};
```

```
{5, 8, 6}  -> fila 0  
{7, 9, 4}  -> fila 1  
{3, 2, 1}  -> fila 2
```

```
matriz[1][2] = 10;
```


247 ¿Cómo llenamos una matriz con datos del usuario?

```
for (int i = 0; i < 5; i++) {  
    cin >> notas[i];  
}
```

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        cin >> matriz[i][j];  
    }  
}
```

```
i = controla las filas  
j = controla las columnas
```


248 Sigamos el recorrido paso a paso

```
int tabla[2][3];
```

```
i = 0, j = 0    -> tabla[0][0]
i = 0, j = 1    -> tabla[0][1]
i = 0, j = 2    -> tabla[0][2]

i = 1, j = 0    -> tabla[1][0]
i = 1, j = 1    -> tabla[1][1]
i = 1, j = 2    -> tabla[1][2]
```

Relación con lo que ya sabemos

No estamos aprendiendo un ciclo nuevo.

Seguimos utilizando `for`.

La diferencia es que ahora **un ciclo está dentro de otro**, porque necesitamos controlar dos dimensiones.

249 Mostrar una matriz como una tabla

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        cout << matriz[i][j] << "\t";  
    }  
    cout << endl;  
}
```

```
int matriz[3][3] = {  
    {5, 8, 6},  
    {7, 9, 4},  
    {3, 2, 1}  
};
```

5	8	6
7	9	4
3	2	1

250 Caso de la vida real: registro de calificaciones

```
string nombres[3] = {"Ana", "Carlos", "Maria"};
```

```
float notas[3][3] = {  
    {85, 70, 66},  
    {77, 40, 91},  
    {60, 100, 89}  
};
```

```
nombres[1]
```

```
notas[1][0]  
notas[1][1]  
notas[1][2]
```


251 Ejemplo 1: promedio por estudiante

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    const int ALUMNOS = 3;
    const int CURSOS = 3;

    string nombres[ALUMNOS] = {
        "Ana",
        "Carlos",
        "Maria"
    };

    float notas[ALUMNOS][CURSOS] = {
        {85, 70, 66},
        {77, 40, 91},
        {60, 100, 89}
    };

    for (int i = 0; i < ALUMNOS; i++) {

        float suma = 0;

        for (int j = 0; j < CURSOS; j++) {
            suma += notas[i][j];
        }

        float promedio = suma / CURSOS;
```

```
        cout << nombres[i]
              << " - Promedio: "
              << promedio << endl;
    }

    return 0;
}
```

```
float suma = 0;
```

252 Pensemos antes de continuar

```
int datos[4][5];
```

- 1.
- 2.
- 3.
- 4.
- 5.

Respuesta

- Filas: 4
- Columnas: 5
- Elementos: 20
- Primera posición: `datos[0][0]`
- Última posición: `datos[3][4]`

253 Comparación: arreglo, vector y matriz

254 Ejercicio guiado

▪
▪
▪
▪

254.1 Paso 1: declarar la matriz

```
const int FILAS = 3;  
const int COLUMNAS = 3;  
  
int matriz[FILAS][COLUMNAS];
```

254.2 Paso 2: ingresar los datos

```
for (int i = 0; i < FILAS; i++) {  
    for (int j = 0; j < COLUMNAS; j++) {  
        cout << "Ingrese [" << i << "][" << j << "]: ";  
        cin >> matriz[i][j];  
    }  
}
```

254.3 Paso 3: procesar los datos

```
int suma = 0;
int mayor = matriz[0][0];
int menor = matriz[0][0];

for (int i = 0; i < FILAS; i++) {
    for (int j = 0; j < COLUMNAS; j++) {
        suma += matriz[i][j];

        if (matriz[i][j] > mayor) {
            mayor = matriz[i][j];
        }

        if (matriz[i][j] < menor) {
            menor = matriz[i][j];
        }
    }
}
```

254.4 Paso 4: calcular el promedio

```
float promedio = (float)suma / (FILAS * COLUMNAS);
```

254.5 Paso 5: mostrar resultados

```
cout << "Suma: " << suma << endl;
cout << "Promedio: " << promedio << endl;
cout << "Mayor: " << mayor << endl;
cout << "Menor: " << menor << endl;
```

255 Buenas prácticas

▪
▪
▪
▪
▪

```
int notas[35][5];
```

```
const int ALUMNOS = 35;  
const int CURSOS = 5;  
  
int notas[ALUMNOS][CURSOS];
```


256 Más de dos dimensiones

```
datos[filas][columnas]
```

```
int datos[3][3][3];
```

```
datos[i][j][k]
```

Por ahora...

No necesitamos complicarnos con arreglos de tres o más dimensiones.
Primero debemos dominar completamente:

```
matriz[i][j]
```

y el uso de ciclos anidados.

257 Comprobemos la comprensión

```
int matriz[2][4] = {  
    {10, 20, 30, 40},  
    {50, 60, 70, 80}  
};
```

- 1.
- 2.
- 3.
- 4.
- 5.

Respuestas

1. 30
2. 50
3. `matriz[1][3]`
4. 8
5. Cambiamos la fila por la columna; no estamos señalando necesariamente la misma posición.

258 El reto para el laboratorio

	Curso 0	Curso 1	Curso 2
Estudiante 0	85	70	66
Estudiante 1	77	40	91
Estudiante 2	60	100	89

■
■
■
■
■

```
notas[estudiante][curso]
```


259 Cierre

```
Variable
  ↓
Arreglo / Vector
  ↓
Vectores paralelos
  ↓
Necesidad de filas y columnas
  ↓
Matriz
```

💡 Idea para recordar

Vector: necesito una posición.

```
datos[i]
```

Matriz: necesito dos posiciones.

```
datos[i][j]
```

Primero ubicamos la fila. Después, la columna.

260 Cierre del capítulo

Consejo final del Ingeniero

Un buen programador no es quien escribe código más rápido, sino quien piensa mejor la solución antes de programarla.

260.1 Próximo paso

